

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

THIẾT KẾ BỘ XỬ LÝ RISC-V TÍCH HỢP P-EXTENSION

NGÀNH: CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

SVTH: LÊ ANH QUÁT

MSSV: 22161311

SVTH: TRANG ĐẶNG VĨ THẮNG

MSSV: 22161320

GVHD: TS. PHẠM VĂN KHOA

TP. HỒ CHÍ MINH – 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

THIẾT KẾ BỘ XỬ LÝ RISC-V TÍCH HỢP P-EXTENSION

NGÀNH: CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - VIỄN THÔNG

SVTH: LÊ ANH QUẤT

MSSV: 22161311

SVTH: TRANG ĐẶNG VĨ THẮNG

MSSV: 22161320

GVHD: TS. PHẠM VĂN KHOA

TP. HỒ CHÍ MINH – 06/2026

PHIẾU GIAO NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

1. Thông tin sinh viên

Họ và tên: Lê Anh Quát

MSSV: 22161311

Họ và tên: Trang Đặng Vĩ Thắng

MSSV: 22161320

2. Thông tin đề tài

Tên đề tài: Thiết kế bộ xử lý RISC-V tích hợp P-extension

Mục đích đề tài: Thiết kế và đánh giá vi xử lý RISC-V RV32IMP nhằm tăng tốc các ứng dụng xử lý dữ liệu song song nhưng vẫn đảm bảo tính tối giản của kiến trúc cơ sở.

3. Nội dung thực hiện

- Nghiên cứu kiến trúc RISC-V RV32IM, pipeline 5 tầng và tập lệnh P-extension.
- Dùng SystemVerilog thiết kế lõi RV32IM và RV32IMP.
- Mô phỏng, kiểm chứng chức năng toàn hệ thống trên phần mềm QuestaSim.
- Tổng hợp trên Quartus để đánh giá tài nguyên phần cứng, tần số hoạt động và công suất tiêu thụ.
- Đánh giá hiệu năng thực thi qua 3 thuật toán: bộ lọc FIR, phát hiện biên Sobel và nhân ma trận.

4. Lời cam đoan của sinh viên

Chúng tôi cam đoan đồ án tốt nghiệp này là kết quả nghiên cứu và thực hiện của nhóm dưới sự hướng dẫn của tiến sĩ Phạm Văn Khoa. Các kết quả, số liệu và nội dung trình bày trong đồ án là trung thực, được trích dẫn đầy đủ theo quy định và chưa từng được sử dụng để bảo vệ bất kỳ học vị nào trước đây. Chúng tôi hoàn toàn chịu trách nhiệm về nội dung cam đoan này.

TP.HCM, ngày 23 tháng 6 năm 2026

Sinh viên thực hiện:

Lê Anh Quát

Trang Đặng Vĩ Thắng

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Thiết kế bộ xử lý RISC-V tích hợp L-extension

Sinh viên: + Lê Anh Quát

MSSV: 22161311

+ Trang Đặng Vũ Thắng

MSSV: 22161320

Hướng dẫn: TS. Phạm Văn Khoa

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá/ kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

8. Đánh giá về sự trùng lặp của đề tài

Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Nghịêm túc, tinh thần học hỏi tốt

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Bảo cấp đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Cho bảo vệ (2,0)

Tp. HCM, ngày 22 tháng 05 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)

Phạm Văn Khoa

* Giáo viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CẢM ƠN

Lời đầu tiên, nhóm chúng tôi xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy Phạm Văn Khoa. Trong suốt quá trình thực hiện đồ án, thầy đã luôn tận tình chỉ dẫn và dành thời gian định hướng cho nhóm. Chúng tôi thực sự trân trọng sự kiên nhẫn và những góp ý chi tiết của thầy trong từng buổi họp. Những phân tích của thầy đã giúp nhóm chúng tôi làm rõ được nhiều vấn đề kỹ thuật quan trọng. Qua đó, chúng tôi cũng học hỏi được từ thầy cách tư duy để giải quyết một vấn đề kỹ thuật một cách hệ thống và bài bản hơn.

Nhóm chúng tôi cũng xin gửi lời tri ân sâu sắc đến quý thầy cô tại khoa Điện - Điện tử của Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh. Quý thầy cô đã trang bị cho chúng tôi một nền tảng kiến thức vững chắc, đồng thời tạo ra một môi trường học tập chuyên nghiệp, giúp nhóm có cơ hội tiếp cận với quy trình thiết kế và kiểm chứng vi mạch hiện đại.

Cuối cùng, nhóm xin chân thành cảm ơn gia đình đã luôn động viên, giúp đỡ, quan tâm và tạo điều kiện thuận lợi nhất để nhóm có thể hoàn thành đồ án tốt nghiệp.

TÓM TẮT

Đề tài tập trung thiết kế bộ xử lý RISC-V RV32IM pipeline 5 tầng và tích hợp P-extension nhằm tăng khả năng xử lý song song dữ liệu cho các ứng dụng xử lý tín hiệu số và xử lý ảnh. Thiết kế được xây dựng trên nền kiến trúc RISC-V 32 bit, trong đó kiến trúc RV32IM được phát triển trước, sau đó bổ sung khối xử lý mở rộng để hỗ trợ các lệnh Packed SIMD trên dữ liệu số nguyên 8 bit và 16 bit.

Quá trình thực hiện bao gồm nghiên cứu kiến trúc RV32IM, thiết kế các khối chức năng, xây dựng đường dữ liệu pipeline 5 tầng, tích hợp khối SIMD Pext và kiểm chứng hoạt động bằng mô phỏng RTL trên QuestaSim. Để đánh giá hiệu quả của phần mở rộng, đề tài sử dụng các ứng dụng kiểm thử gồm bộ lọc FIR, thuật toán phát hiện biên Sobel và nhân ma trận. Kết quả của kiến trúc RV32IMP sẽ được so sánh với RV32IM dựa trên số chu kỳ thực thi và số lệnh hoàn tất.

Bên cạnh đánh giá hiệu năng, thiết kế còn được tổng hợp bằng Quartus để xem xét mức sử dụng tài nguyên phần cứng, tần số hoạt động, công suất tiêu thụ và sau đó sẽ đánh giá năng lượng tiêu thụ. Qua đó, đề tài làm rõ khả năng cải thiện hiệu năng của P-extension đối với các bài toán có tính lặp và song song dữ liệu, đồng thời đánh giá mức đánh đổi giữa hiệu năng, tài nguyên phần cứng và công suất cũng như là năng lượng khi tích hợp phần mở rộng vào bộ xử lý RISC-V.

MỤC LỤC

LỜI CẢM ƠN	i
TÓM TẮT	ii
MỤC LỤC	iii
DANH MỤC HÌNH ẢNH	vii
DANH MỤC BẢNG	xi
CÁC TỪ VIẾT TẮT	xiii
CHƯƠNG 1	1
TỔNG QUAN	1
1.1. GIỚI THIỆU	1
1.2. MỤC TIÊU NGHIÊN CỨU	2
1.3. ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU	2
1.4. PHƯƠNG PHÁP NGHIÊN CỨU	3
1.5. BỐ CỤC ĐỒ ÁN	3
CHƯƠNG 2	5
CƠ SỞ LÝ THUYẾT	5
2.1. KIẾN TRÚC TẬP LỆNH RISC-V	5
2.1.1. Tổng quan	5
2.1.2. Kiến trúc tập lệnh số nguyên cơ sở RV32I	6
2.1.3. Phần mở rộng M-extension	11
2.1.4. Các phần mở rộng tiêu chuẩn khác của RISC-V	13
2.2. KỸ THUẬT PIPELINE	13
2.2.1. Tổng quan về kỹ thuật pipeline	13
2.2.2. Hoạt động của pipeline năm tầng	15
2.2.3. Xung đột trong pipeline	16

2.3. KỸ THUẬT SIMD VÀ P-EXTENSION	22
2.3.1. Giới thiệu về SIMD	22
2.3.2. Tổng quan về P-extension	23
2.3.2. Các nhóm lệnh cơ bản của P-extension.....	23
2.3.3. Ưu điểm và hạn chế của P-extension.....	26
CHƯƠNG 3	28
THIẾT KẾ HỆ THỐNG.....	28
3.1. TỔNG QUAN KIẾN TRÚC LỖI RV32IMP	28
3.1.1. Sơ đồ khối.....	28
3.1.2. Hoạt động của toàn hệ thống	30
3.2. THIẾT KẾ LỖI CƠ SỞ RV32IM.....	31
3.2.1. Program Counter.....	31
3.2.2. Instruction Memory	32
3.2.3. Decoder.....	33
3.2.4. Register File.....	34
3.2.5. Immediate Generator	35
3.2.6. Branch Comparator.....	36
3.2.7. Arithmetic Logic Unit (ALU).....	37
3.2.8. Data Memory	38
3.2.9. Control Logic.....	39
3.3.10. Pipeline Register.....	40
3.3.11. Hazard Unit.....	41
3.3. THIẾT KẾ KHỐI THỰC THI SIMD PEXT.....	48
3.3.1. Cấu trúc tổng thể của khối SIMD Pext.....	48
3.3.2. Bộ cộng SIMD 4x9 bit.....	50

3.3.3. Nhóm lệnh giới hạn dữ liệu	55
3.3.4. Nhóm lệnh nhân cộng dồn.....	56
3.4. TÍCH HỢP SIMD PEXT VÀO PIPELINE RV32IM.....	57
3.4.1. Mở rộng tầng giải mã.....	57
3.4.2. Mở rộng Register File.....	58
3.4.3. Xử lý xung đột dữ liệu.....	58
3.5. QUY TRÌNH ĐÁNH GIÁ VÀ KIỂM THỬ CÁC ỨNG DỤNG.....	59
CHƯƠNG 4	61
KẾT QUẢ VÀ ĐÁNH GIÁ	61
4.1. KẾT QUẢ MÔ PHÒNG	61
4.1.1. Mô phỏng thiết kế RISC-V RV32IM.....	61
4.1.2. Mô phỏng thiết kế RISC-V RV32IMP	74
4.2. ĐÁNH GIÁ HIỆU NĂNG QUA CÁC ỨNG DỤNG	83
4.2.1. Nhân ma trận.....	84
4.2.2. Bộ lọc FIR.....	89
4.2.3. Thuật toán phát hiện biên Sobel	92
4.3. ĐÁNH GIÁ TÀI NGUYÊN, TẦN SỐ HOẠT ĐỘNG, CÔNG SUẤT VÀ NĂNG LƯỢNG GIỮA RV32IM VÀ RV32IMP	97
4.3.1. Tài nguyên sử dụng.....	97
4.3.2. Tần số hoạt động.....	98
4.3.3. Công suất tiêu thụ	99
4.3.4. Năng lượng tiêu thụ	100
CHƯƠNG 5	102
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	102
5.1. KẾT LUẬN	102

5.2. HƯỚNG PHÁT TRIỂN	103
TÀI LIỆU THAM KHẢO	104
PHỤ LỤC 1.....	106
PHỤ LỤC 2.....	107
PHỤ LỤC 3.....	108

DANH MỤC HÌNH ẢNH

Hình 2.1: Sơ đồ tập thanh ghi 32 bit của RISC-V RV32I.....	6
Hình 2.2: Các định dạng mã hóa lệnh trong RISC-V RV32I [5]	7
Hình 2.3: Định dạng mã lệnh R-type trong RISC-V [4]	8
Hình 2.4: Định dạng mã lệnh I-type trong RISC-V [4]	9
Hình 2.5: Minh họa mã lệnh dịch bit (Shift) dạng I-type trong RISC-V [4]	9
Hình 2.6: Định dạng mã lệnh Load và Store trong RISC-V [4]	9
Hình 2.7: Định dạng mã lệnh B-type và J-type trong RISC-V [4]	10
Hình 2.8: Định dạng mã lệnh U-type trong RISC-V [4]	11
Hình 2.9: Định dạng mã lệnh nhân trong M-extension [5]	11
Hình 2.10: Định dạng mã lệnh chia trong M-extension [5]	12
Hình 2.11: Thời gian thực thi trong mô hình pipeline 5 tầng	14
Hình 2.12: Sơ đồ 5 giai đoạn pipeline của bộ xử lý RISC-V	15
Hình 2.13: Xung đột cấu trúc khi Instruction Memory và Data Memory dùng chung RAM	17
Hình 2.14: Phân cứng riêng biệt của Instruction Memory và Data Memory	17
Hình 2.15: Minh họa xung đột dữ liệu (Data Hazard)	18
Hình 2.16: Forwarding dữ liệu từ các giai đoạn sau về ALU	18
Hình 2.17: Sơ đồ tổng quát của Forwarding Control Logic	19
Hình 2.18: Minh họa Load-Use hazard	20
Hình 2.19: Stall 1 chu kỳ để xử lý Load-Use	20
Hình 2.20: Lệnh nhánh chỉ quyết định hướng đi sau khi hoàn tất EX	21
Hình 2.21: Pipeline hoạt động bình thường khi dự đoán đúng hướng nhánh	21
Hình 2.22: Flush pipeline khi dự đoán sai hướng nhánh	22
Hình 2.23: Các cách đóng gói dữ liệu trong thanh ghi 32 bit của P-extension ...	23

Hình 3.1: Sơ đồ khối tổng quát của bộ xử lý RV32IMP pipeline năm tầng	29
Hình 3.2: Program Counter	31
Hình 3.3: Instruction Memory	32
Hình 3.4: Decoder	33
Hình 3.5: Register File	34
Hình 3.6: Immediate Generator	35
Hình 3.7: Branch Comparator	36
Hình 3.8: Arithmetic Logic Unit (ALU)	37
Hình 3.9: Data Memory	38
Hình 3.10: Control Logic	39
Hình 3.11: Pipeline Register	40
Hình 3.12: Hazard Unit	42
Hình 3.13: Forwarding Unit	43
Hình 3.14: Hazard Detection	45
Hình 3.15: Branch Control	47
Hình 3.16: Sơ đồ tổng quan khối SIMD Pext [11]	49
Hình 3.17: Cấu trúc bộ cộng SIMD 4x9 bit cho chế độ byte và halfword [11]... 51	
Hình 3.18: Sơ đồ thiết kế khối giới hạn dữ liệu Clip	55
Hình 3.19: Sơ đồ thiết kế khối nhân cộng dồn MAC trong P-extension	56
Hình 3.20: Register File sau khi bổ sung cổng đọc rsR3	58
Hình 3.21: Quy trình đánh giá và kiểm thử các ứng dụng trên RV32IM và RV32IMP	60
Hình 4.1: Dạng sóng mô phỏng của thiết kế RV32IM từ 0 – 65ns.....	63
Hình 4.2: Dạng sóng mô phỏng của thiết kế RV32IM từ 65 – 135ns.....	65
Hình 4.3: Dạng sóng mô phỏng của thiết kế RV32IM từ 135 – 205ns.....	69

Hình 4.4: Dạng sóng mô phỏng của thiết kế RV32IM từ 205 – 275ns.....	72
Hình 4.5: Dạng sóng mô phỏng của thiết kế RV32IMP từ 0 – 75ns.....	76
Hình 4.6: Dạng sóng mô phỏng của thiết kế RV32IMP từ 75 – 155ns.....	79
Hình 4.7: Nhân ma trận sử dụng SIMD [11]	84
Hình 4.8: Nhân ma trận sử dụng SIMD cùng với ma trận chuyển vị <i>BT</i> [11]	85
Hình 4.9: Dạng sóng mô phỏng của RV32IM trên ứng dụng nhân ma trận	86
Hình 4.10: Dạng sóng mô phỏng của RV32IMP trên ứng dụng nhân ma trận	86
Hình 4.11: Dạng sóng mô phỏng của RV32IMP kết hợp chuyển vị ma trận trên ứng dụng nhân ma trận.....	87
Hình 4.12: So sánh hiệu năng phép nhân ma trận giữa RV32IM, RV32IMP và RV32IMP kết hợp chuyển vị ma trận.....	88
Hình 4.13: Dạng sóng mô phỏng của RV32IM trên ứng dụng bộ lọc FIR.....	90
Hình 4.14: Dạng sóng mô phỏng của RV32IMP trên ứng dụng bộ lọc FIR.....	90
Hình 4.15: Tín hiệu gốc, đầu vào có nhiễu và đầu ra khi qua bộ lọc FIR	91
Hình 4.16: So sánh hiệu năng bộ lọc FIR giữa RV32IM và RV32IMP.....	92
Hình 4.17: Tính gradient theo hướng ngang và dọc trong thuật toán Sobel.....	93
Hình 4.18: Dạng sóng mô phỏng của RV32IM trên ứng dụng phát hiện biên Sobel	94
Hình 4.19: Dạng sóng mô phỏng của RV32IMP trên ứng dụng phát hiện biên Sobel	94
Hình 4.20: Ảnh trước và sau khi áp dụng bộ lọc Sobel	95
Hình 4.21: So sánh hiệu năng thuật toán phát hiện biên Sobel giữa RV32IM và RV32IMP	96
Hình 4.22: So sánh tần số hoạt động tối đa trên hai kiến trúc RV32IM và RV32IMP	98
Hình 4.23: So sánh công suất tiêu thụ giữa RV32IM và RV32IMP	99

Hình 4.24: So sánh năng lượng tiêu thụ giữa RV32IM và RV32IMP 101

DANH MỤC BẢNG

Bảng 2.1: Phân loại định dạng lệnh trong RISC-V RV32I	8
Bảng 2.2: Minh họa thời gian cần thiết cho một số loại lệnh	14
Bảng 2.3: Các nhóm lệnh cơ bản của P-extension.....	24
Bảng 3.1: Mô tả các tín hiệu của Program Counter	32
Bảng 3.2: Mô tả các tín hiệu của Instruction Memory	33
Bảng 3.3: Mô tả các tín hiệu của Decoder	34
Bảng 3.4: Mô tả các tín hiệu của Register File	35
Bảng 3.5: Mô tả các tín hiệu của Immediate Generator	36
Bảng 3.6: Mô tả các tín hiệu của Branch Comparator	37
Bảng 3.7: Mô tả các tín hiệu của Arithmetic Logic Unit (ALU)	38
Bảng 3.8: Mô tả các tín hiệu của Data Memory	39
Bảng 3.9: Mô tả các tín hiệu của Control Logic	40
Bảng 3.10: Bảng trạng thái Pipeline Register	41
Bảng 3.11: Mã hóa tín hiệu Forwarding (forwardA, forwardB).....	43
Bảng 3.12: Điều kiện allow_exmem_fwd.....	43
Bảng 3.13: Trạng thái điều khiển forwardA	44
Bảng 3.14: Trạng thái điều khiển forwardB.....	44
Bảng 3.15: Xác định nhu cầu sử dụng thanh ghi nguồn của lệnh ở ID	45
Bảng 3.16: Quan hệ ngõ vào – ngõ ra của Hazard Detection	46
Bảng 3.17: Bảng quan hệ ngõ vào – ngõ ra của Branch Control.....	47
Bảng 3.18: Nhóm lệnh P-extension được hỗ trợ trong SIMD Pext	50
Bảng 3.19: Phát hiện tràn số có dấu (Signed - PSADD, PSSUB)	52
Bảng 3.20: Phát hiện tràn số không dấu (Unsigned - PSADDU, PSSUBU).....	52
Bảng 3.21: Cấu hình trao đổi dữ liệu cho nhóm Cross	52

Bảng 3.22: Kết quả khi A nhỏ hơn B	54
Bảng 3.23: Logic xử lý PSABS theo từng trường hợp	54
Bảng 3.24: Các điểm mở rộng khi tích hợp SIMD Pext vào pipeline RV32IM..	57
Bảng 4.1: Mã Assembly dùng mô phỏng các lệnh của thiết kế RV32IM.....	61
Bảng 4.2: Mã Assembly dùng mô phỏng các lệnh của thiết kế RV32IMP.....	74
Bảng 4.3: So sánh tài nguyên phần cứng của hai kiến trúc RV32IM và RV32IMP trên ứng dụng bộ lọc FIR	97
Bảng 4.4: So sánh công suất tiêu thụ của kiến trúc RV32IM và RV32IMP.....	99
Bảng 4.5: So sánh năng lượng tiêu thụ của kiến trúc RV32IM và RV32IMP ...	100

CÁC TỪ VIẾT TẮT

Ký hiệu	Từ tiếng Anh	Nghĩa tiếng Việt
ALU	Arithmetic Logic Unit	Khối thực hiện các phép toán số học
ARM	Advanced RISC Machines	Kiến trúc vi xử lý RISC của ARM
ASIC	Application-Specific Integrated Circuit	Mạch tích hợp chuyên dụng theo ứng dụng
CLK	Clock	Xung nhịp điều khiển hoạt động đồng bộ của toàn hệ thống
CPU	Central Processing Unit	Khối xử lý trung tâm, thực hiện các lệnh của chương trình.
CSR	Control and Status Register	Thanh ghi điều khiển và trạng thái
DSP	Digital Signal Processing	Xử lý tín hiệu số
EX	Execute	Giai đoạn thực thi
FFs	Flip-Flops	Các phần tử nhớ Flip-Flop
FPGA	Field-Programmable Gate Array	Vi mạch logic lập trình được
ID	Instruction Decode	Giai đoạn giải mã lệnh
IF	Instruction Fetch	Giai đoạn lấy lệnh từ bộ nhớ lệnh
ISA	Instruction Set Architecture	Tập hợp các lệnh mà bộ xử lý có thể hiểu và thực thi.
LUTs	Look-Up Tables	Các bảng tra cứu, là phần tử logic cơ bản trong FPGA
MAC	Multiply–Accumulate	Nhân cộng dồn
MEM	Memory Access	Giai đoạn truy cập bộ nhớ
MUX	Multiplexer	Bộ chọn kênh
NOP	No Operation	Lệnh không thực hiện bất kỳ thao tác nào.

PC	Program Counter	Giữ địa chỉ lệnh hiện tại, tự tăng sau mỗi chu kỳ.
RAM	Random Access Memory	Bộ nhớ truy xuất ngẫu nhiên
RAW	Read After Write	Đọc sau ghi
RISC	Reduced Instruction Set Computing	Máy tính với tập lệnh đơn giản
RISC-V	Reduced Instruction Set Computer Five	Kiến trúc tập lệnh RISC thế hệ thứ năm
RTL	Register-Transfer Level	Mức mô tả phần cứng dựa trên truyền dữ liệu giữa các thanh ghi
RV32I	RISC-V 32-bit Base Integer Instruction Set	Máy tính với tập lệnh đơn giản
RV32IM	RV32I with M Extension	RV32I có thêm phần mở rộng nhân và chia số nguyên
RV32IMP	RV32IM with P Extension	RV32IM có thêm phần mở rộng xử lý song song P-extension
SIMD	Single Instruction Multiple Data	Một lệnh nhiều dữ liệu
SoC	System on a Chip	Hệ thống trên một chip
SRAM	Static Random Access Memory	Bộ nhớ truy cập ngẫu nhiên tĩnh
WB	Write Back	Giai đoạn ghi kết quả về Register File

CHƯƠNG 1

TỔNG QUAN

1.1. GIỚI THIỆU

Trong bối cảnh công nghệ số phát triển mạnh mẽ, các hệ thống điện tử hiện đại ngày càng được ứng dụng rộng rãi trong công nghiệp, thiết bị thông minh, hệ thống nhúng và các thiết bị xử lý tại biên. Các hệ thống này không chỉ đòi hỏi khả năng xử lý nhanh, ổn định mà còn cần đáp ứng các yêu cầu về tiêu thụ năng lượng thấp, chi phí hợp lý và khả năng tùy biến theo từng bài toán cụ thể. Do đó, việc lựa chọn kiến trúc vi xử lý phù hợp giữ vai trò quan trọng trong quá trình thiết kế phần cứng.

Trong xu hướng đó, kiến trúc RISC tiếp tục được quan tâm nhờ triết lý thiết kế tập lệnh đơn giản, tối ưu hóa phần cứng và nâng cao hiệu quả thực thi lệnh. Với đặc điểm giảm độ phức tạp trong xử lý, RISC có thể mang lại nhiều ưu điểm như hiệu suất cao, tiêu thụ năng lượng thấp và khả năng mở rộng linh hoạt. Mặc dù RISC-V có ưu điểm về tính đơn giản và khả năng mở rộng, các lõi xử lý cơ bản vẫn còn hạn chế khi thực hiện những tác vụ có nhiều phép toán lặp trên dữ liệu nhỏ. Do đó, một trong những hướng tiếp cận hiệu quả là sử dụng kỹ thuật SIMD, cho phép một lệnh duy nhất thực hiện cùng một phép toán trên nhiều phần tử dữ liệu khác nhau, từ đó cải thiện hiệu năng đối với các bài toán xử lý dữ liệu song song [1].

Mở rộng này cho phép đóng gói nhiều phần tử dữ liệu nhỏ vào cùng một thanh ghi và thực hiện phép toán trên chúng bằng một lệnh duy nhất. Cách xử lý này giúp khai thác tốt hơn tài nguyên sẵn có của bộ xử lý, tăng mức độ song song dữ liệu và cải thiện hiệu năng mà vẫn giữ được đặc điểm gọn nhẹ của kiến trúc RISC-V [1]. Ngoài ra trong ứng dụng xử lý tín hiệu số cụ thể là bộ lọc FIR thì các nghiên cứu gần đây cho thấy việc triển khai FIR trên các nền tảng phần cứng tăng tốc như FPGA có thể cải thiện đáng kể hiệu năng so với xử lý thuần CPU, nhờ khả năng khai thác song song mức dữ liệu và giảm độ trễ xử lý. Tuy nhiên, các giải

pháp này thường yêu cầu phần cứng chuyên dụng hoặc tài nguyên lớn, làm tăng chi phí và độ phức tạp hệ thống [2].

Do đó, nhóm nhận thấy việc tích hợp P-extension vào bộ xử lý RISC-V là hướng nghiên cứu phù hợp với xu thế thiết kế bộ xử lý mở, linh hoạt và có khả năng tùy biến theo từng nhóm ứng dụng. Vì vậy, nhóm lựa chọn đề tài “Thiết kế bộ xử lý RISC-V tích hợp P-extension” để nghiên cứu và triển khai.

1.2. MỤC TIÊU NGHIÊN CỨU

Mục tiêu của đề tài là thiết kế và đánh giá một bộ xử lý RISC-V RV32IM tích hợp P-extension nhằm hỗ trợ thực thi các phép toán Packed SIMD trên dữ liệu số nguyên. Thiết kế hướng đến việc cải thiện hiệu năng xử lý cho các ứng dụng có tính lặp và song song dữ liệu, đồng thời vẫn giữ được tính gọn nhẹ và khả năng mở rộng của kiến trúc RISC-V.

Các mục tiêu cụ thể của đề tài gồm:

- Tìm hiểu kiến trúc RISC-V RV32IM, mô hình pipeline 5 tầng và đặc điểm của P-extension.
- Thiết kế bộ xử lý RISC-V RV32IM và tích hợp khối SIMD Pext.
- Mô phỏng và kiểm chứng hoạt động của toàn hệ thống bằng QuestaSim.
- Chạy các ứng dụng cụ thể như bộ lọc FIR, thuật toán phát hiện biên Sobel và nhân ma trận, sau đó so sánh với thiết kế RV32IM dựa trên số chu kỳ thực thi và số lệnh hoàn thành.
- Tổng hợp trên Quartus để đánh giá tài nguyên phần cứng, tần số hoạt động, công suất tiêu thụ và sau đó sẽ đánh giá năng lượng tiêu thụ.

1.3. ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Đối tượng nghiên cứu của đề tài là kiến trúc bộ xử lý RISC-V RV32IM và phần mở rộng P-extension. Trong đó, đề tài tập trung vào việc thiết kế bộ vi xử lý 32 bit pipeline 5 tầng, sau đó bổ sung khả năng thực thi các lệnh Packed SIMD, nhằm tăng hiệu quả xử lý cho các dữ liệu số nguyên có độ rộng nhỏ như 8 bit và 16 bit.

Phạm vi nghiên cứu của đề tài được giới hạn ở mức thiết kế và đánh giá phần cứng RTL cho bộ xử lý RISC-V RV32IM pipeline 5 tầng có tích hợp P-extension. Các nội dung thực hiện tập trung vào mô tả phần cứng bằng SystemVerilog, xây dựng testbench, mô phỏng chức năng trên QuestaSim và tổng hợp thiết kế bằng Quartus. Việc đánh giá được thực hiện trên một số bài toán bao gồm bộ lọc FIR, thuật toán phát hiện biên Sobel và nhân ma trận. Đề tài không mở rộng sang hệ điều hành, bộ nhớ cache, ngoại vi phức tạp hoặc triển khai vật lý ở mức chip.

1.4. PHƯƠNG PHÁP NGHIÊN CỨU

Các phương pháp nghiên cứu được sử dụng trong đề tài bao gồm nghiên cứu lý thuyết, mô hình hóa phần cứng, mô phỏng - kiểm chứng và so sánh đánh giá. Trước hết, nhóm tìm hiểu kiến trúc RISC-V RV32IM, mô hình pipeline 5 tầng và đặc điểm của P-extension để xác định các thành phần cần xây dựng và mở rộng trong bộ xử lý.

Tiếp theo, nhóm sẽ thiết kế bộ xử lý RISC-V RV32IM ở mức RTL bằng SystemVerilog, sau đó bổ sung khối SIMD Pext vào đường dữ liệu và khối điều khiển. Thiết kế được xây dựng testbench và mô phỏng bằng QuestaSim để kiểm tra hoạt động của các thiết kế cũng như khả năng thực thi lệnh.

Sau khi hoàn thiện mô phỏng, nhóm sử dụng các ứng dụng bộ lọc FIR, thuật toán phát hiện biên Sobel và nhân ma trận để đánh giá hiệu năng giữa RV32IM và RV32IM tích hợp P-extension. Kết quả được phân tích dựa trên số chu kỳ thực thi và số lệnh hoàn tất. Ngoài ra, thiết kế được tổng hợp bằng Quartus để đánh giá mức sử dụng tài nguyên, tần số hoạt động, công suất tiêu thụ và sau đó sẽ tính toán để đánh giá năng lượng tiêu thụ.

1.5. BỐ CỤC ĐỒ ÁN

Chương 1: Tổng quan

Chương này sẽ trình bày tổng quan về đề tài, mục tiêu đặt ra của đề tài, phương pháp nghiên cứu cũng như là bố cục đề tài.

Chương 2: Cơ sở lý thuyết

Giới thiệu kiến trúc tập lệnh RISC-V, kỹ thuật pipeline, kỹ thuật SIMD và P-extension trong kiến trúc RISC-V.

Chương 3: Thiết kế hệ thống

Chương này nhóm sẽ thiết kế sơ đồ khối toàn hệ thống, chi tiết các khối trong kiến trúc RV32IM, thiết kế khối SIMD Pext và tích hợp vào kiến trúc RV32IM cơ sở.

Chương 4: Kết quả và đánh giá

Chương này nhóm sẽ trình bày các kết quả mô phỏng. Đánh giá hiệu năng của P-extension thông qua các ứng dụng cụ thể và tổng hợp phần cứng thông qua Quartus, đánh giá tần số hoạt động tối đa mà thiết kế có thể chạy được và đánh giá công suất cũng như năng lượng tiêu thụ.

Chương 5: Kết luận và hướng phát triển

Dựa vào những kết quả đạt được ở chương 4, nhóm đưa ra kết luận về những gì đã và chưa đạt được của đề tài. Từ đó có thể đưa ra hướng phát triển cho đề tài.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1. KIẾN TRÚC TẬP LỆNH RISC-V

2.1.1. Tổng quan

RISC-V là một kiến trúc tập lệnh thuộc họ RISC hiện đại, được thiết kế đơn giản, tinh gọn và dễ mở rộng. Điểm khác biệt quan trọng của RISC-V so với các ISA thương mại như x86 hay ARM là tính chất mở và miễn phí bản quyền, cho phép hiện thực hóa và tùy chỉnh tự do [3]. Do đó, RISC-V nhanh chóng được ứng dụng trong nghiên cứu, công nghiệp bán dẫn và các hệ thống SoC/FPGA. Kiến trúc này cũng tạo điều kiện thuận lợi cho việc phát triển các bộ xử lý chuyên biệt, vì người thiết kế có thể bổ sung các phần mở rộng phù hợp với từng nhóm ứng dụng.

RISC-V được xây dựng dựa trên các đặc trưng chính sau:

- Kiến trúc đơn giản và thống nhất: tập lệnh cơ sở RV32I gồm khoảng 40 lệnh với độ dài cố định 32 bit, bao phủ các phép toán số nguyên, điều khiển luồng và truy cập bộ nhớ [4]. Các biến thể RV64I và RV128I mở rộng độ rộng thanh ghi nhưng vẫn giữ cấu trúc lệnh thống nhất, giúp đơn giản hóa giải mã và phù hợp cho pipeline.
- Thiết kế module hóa và khả năng mở rộng linh hoạt RISC-V tổ chức ISA theo dạng mô-đun gồm:
 - Tập lệnh cơ sở: RV32I, RV64I, RV128I.
 - Phần mở rộng tiêu chuẩn: M (nhân/chia), A (nguyên tử), F/D (dấu phẩy động), C (compressed), V (vector).
- Tối ưu cho thiết kế pipeline và phần cứng: RISC-V tuân theo triết lý RISC tức là các lệnh đơn giản, ít chế độ địa chỉ và mô hình load-store. Nhờ vậy, datapath dễ tổ chức theo pipeline 5 giai đoạn (IF-ID-EX-MEM-WB), giảm độ phức tạp phần cứng và tăng hiệu năng thực thi.

2.1.2. Kiến trúc tập lệnh số nguyên cơ sở RV32I

Kiến trúc tập lệnh số nguyên cơ sở RV32I là tập lệnh nền tảng của RISC-V và bắt buộc trong mọi bộ xử lý RISC-V. Nó được thiết kế để đảm bảo hai yếu tố quan trọng. Thứ nhất đó là sự đơn giản dễ triển khai trên phần cứng, phù hợp với FPGA và ASIC nhỏ. Thứ hai đó chính là độ đầy đủ đủ mạnh để làm mục tiêu cho compiler, hỗ trợ hệ điều hành và lập trình hệ thống.

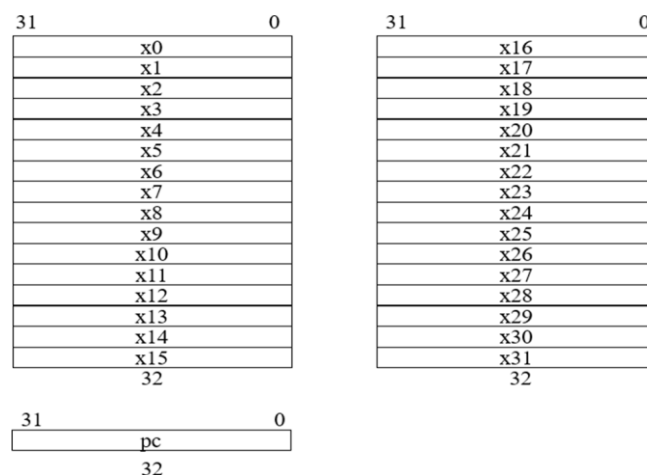
RV32I có các đặc điểm nổi bật:

- Định dạng lệnh đơn giản: RV32I sử dụng một số định dạng lệnh cơ bản như R, I, S, B, U và J, giúp quá trình giải mã lệnh thuận lợi hơn.
- Độ dài lệnh cố định 32 bit: tất cả lệnh của RV32I đều có độ dài 32 bit và được căn chỉnh tự nhiên theo địa chỉ 4 byte [4]. Điều này đơn giản hóa pipeline, đặc biệt trong kiến trúc 5 tầng.
- Hỗ trợ immediate 12 bit hoặc 20 bit, cho phép tạo địa chỉ và hằng số một cách linh hoạt.

Tập thanh ghi (Register File)

Trong kiến trúc RISC-V RV32I, tập thanh ghi là thành phần trung tâm phục vụ lưu trữ toán hạng và giá trị trung gian trong quá trình thực thi lệnh.

Hình 2.1 minh họa sơ đồ tập thanh ghi 32 bit của RV32I, bao gồm 32 thanh ghi số nguyên được đánh số từ x0 đến x31 [3,5].



Hình 2.1: Sơ đồ tập thanh ghi 32 bit của RISC-V RV32I

Trong đó:

- x0 = 0 cố định (hardwired-zero), dùng để tạo hằng 0, bỏ qua giá trị ghi (rd = x0).
- x1 (ra): lưu địa chỉ trở về (return address).
- x2 (sp): con trỏ stack pointer.
- x3 (gp): con trỏ global pointer.
- x4 (tp): con trỏ thread pointer.
- x5-x7 (t0-t2): thanh ghi tạm thời.
- x8 (s0/fp): frame pointer.
- x9-x17 (s1, a0-a7): thanh ghi đối số & kết quả hàm.
- x18-x27 (s2-s11): thanh ghi lưu trữ qua lời gọi hàm.
- x28-x31 (t3-t6): thanh ghi tạm thời bổ sung.
- Ngoài ra còn có thanh ghi đặc biệt pc (program counter), chứa địa chỉ của lệnh hiện tại.

Tổng trạng thái kiến trúc người dùng quan sát được có kích thước: 32×32 bit + 32 bit PC = 1056 bit.

Mã hóa lệnh (Instruction Encoding)

Trong kiến trúc RISC-V RV32I, các lệnh có độ dài cố định 32 bit và được mã hóa theo nhiều định dạng khác nhau tùy theo mục đích sử dụng. Hình 2.2 minh họa các định dạng mã hóa lệnh trong RV32I, thể hiện cách phân bố các trường như opcode, thanh ghi và giá trị tức thời trong từng loại lệnh.

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

Hình 2.2: Các định dạng mã hóa lệnh trong RISC-V RV32I [5]

Dựa trên cấu trúc mã hóa, các lệnh trong RV32I được phân loại theo chức năng và cách sử dụng toán hạng. Bảng 2.1 trình bày 6 định dạng lệnh bao gồm R-type, I-type, S-type, B-type, U-type và J-type của RV32I cùng với mục đích sử dụng tương ứng.

Bảng 2.1: Phân loại định dạng lệnh trong RISC-V RV32I

Định dạng	Mục đích
R-type	Phép toán register–register
I-type	Immediate, load, JALR
S-type	Store
B-type	Branch
U-type	LUI, AUIPC
J-type	JAL (nhảy xa PC-relative)

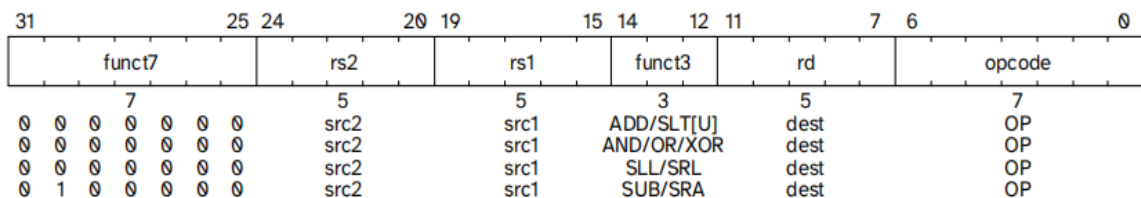
Các trường quan trọng trong lệnh:

- opcode (7 bit): xác định nhóm lệnh.
- funct3 (3 bit) và funct7 (7 bit): xác định biến thể của lệnh.
- rd, rs1, rs2 (5 bit): chỉ định thanh ghi.
- imm: giá trị tức thời (12 bit, 20 bit hoặc ghép từ nhiều bit rải rác trong instruction).

Các loại lệnh của RV32I

❖ Lệnh tính toán (Arithmetic & Logical)

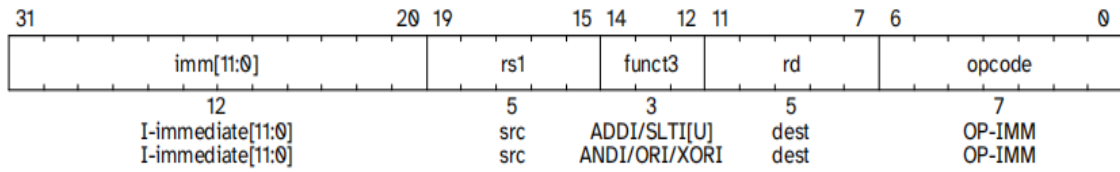
Các lệnh tính toán số học và logic trong RISC-V chủ yếu sử dụng định dạng R-type, trong đó cả hai toán hạng đều lấy từ thanh ghi. Hình 2.3 minh họa cấu trúc mã hóa lệnh R-type trong RISC-V, thể hiện vai trò của các trường như funct7, funct3 và opcode trong việc xác định phép toán cụ thể.

**Hình 2.3:** Định dạng mã lệnh R-type trong RISC-V [4]

RV32I hỗ trợ đầy đủ các phép toán cơ bản:

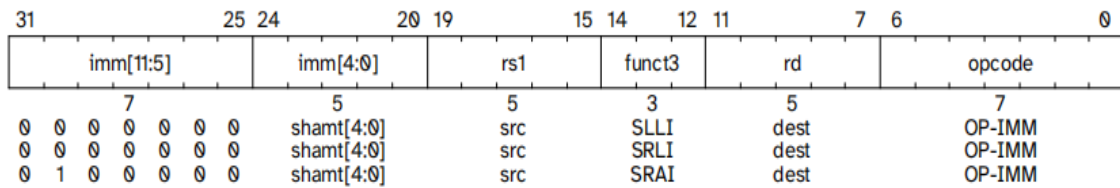
- ADD, SUB, AND, OR, XOR.
- SLT, SLTU - so sánh signed/unsigned.
- SLL, SRL, SRA - dịch trái/phải logic & số học.

Định dạng I-type: Đối với các phép toán cần thực hiện với một hằng số, RISC-V thay thế thanh ghi nguồn thứ hai bằng trường giá trị tức thời là imm 12 bit. Hình 2.4 trình bày định dạng mã hóa I-type dành riêng cho các lệnh số học sử dụng hằng số.



Hình 2.4: Định dạng mã lệnh I-type trong RISC-V [4]

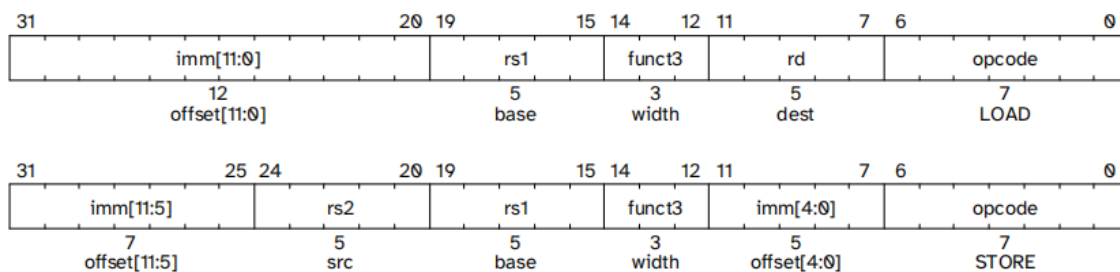
Riêng với các lệnh dịch bit, cấu trúc I-type được tinh chỉnh lại với trường 12 bit được chia nhỏ thành shamt 5 bit chứa lượng dịch và funct7 để phân biệt loại phép dịch như được minh họa trong hình 2.5.



Hình 2.5: Minh họa mã lệnh dịch bit (Shift) dạng I-type trong RISC-V [4]

❖ Lệnh truy cập bộ nhớ (Load–Store)

Trong kiến trúc RISC-V, việc truy cập bộ nhớ tuân theo triết lý load-store, tức chỉ các lệnh load và store mới được truy cập bộ nhớ [4]. Hình 2.6 minh họa định dạng dạng mã hóa lệnh load và store trong RISC-V, ứng với I-type và S-type.



Hình 2.6: Định dạng mã lệnh Load và Store trong RISC-V [4]

RV32I cung cấp:

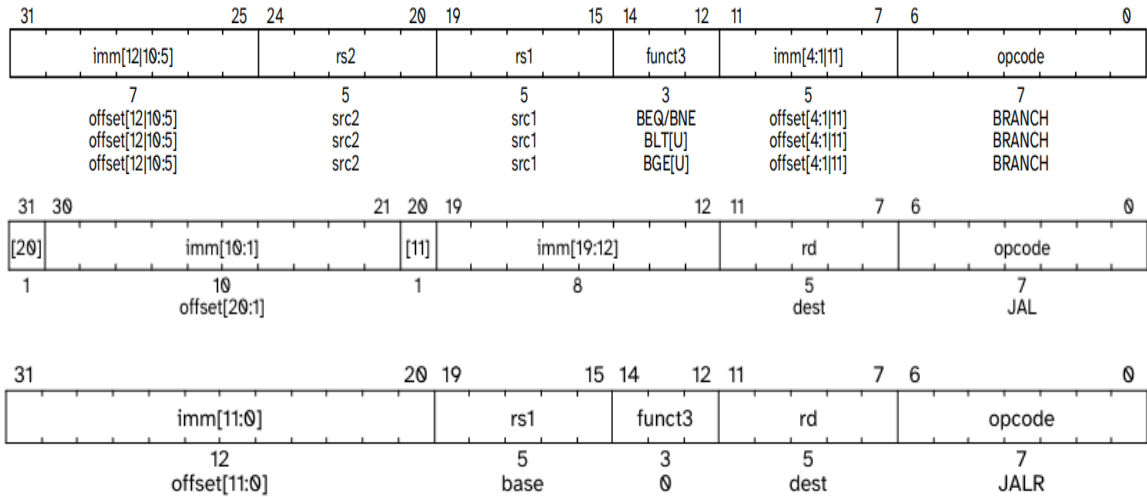
- Load: LB, LH, LW, LBU, LHU.
- Store: SB, SH, SW.

Địa chỉ bộ nhớ được tạo bằng: Address = rs1 + imm.

Kiến trúc load-store giúp đường dữ liệu đơn giản, phù hợp với pipeline 5 tầng IF-ID-EX-MEM-WB.

❖ Lệnh điều khiển luồng (Branch & Jump)

Các lệnh điều khiển trong RISC-V được sử dụng để thay đổi luồng thực thi của chương trình thông qua nhánh có điều kiện và lệnh nhảy. Hình 2.7 minh họa định dạng mã hóa B-type và J-type cho các lệnh branch và jump trong RISC-V.



Hình 2.7: Định dạng mã lệnh B-type và J-type trong RISC-V [4]

Bao gồm:

- BEQ, BNE, BLT, BLTU, BGE, BGEU: các lệnh nhánh có điều kiện, dùng để so sánh `rs1` và `rs2`. Nếu điều kiện đúng thì PC nhảy đến địa chỉ đích.
- JAL: lệnh nhảy không điều kiện theo địa chỉ tương đối PC tức là địa chỉ đích được xác định bằng $PC + \text{offset}$, đồng thời lưu địa chỉ quay lại tức là $PC + 4$ vào `rd`.
- JALR: lệnh nhảy gián tiếp, địa chỉ đích được tính từ $rs1 + \text{offset}$, đồng thời lưu địa chỉ quay lại vào `rd`.

❖ Lệnh tạo hằng số và địa chỉ (U-type)

Hình 2.8 mô tả định dạng mã hóa U-type của hai lệnh LUI và AUIPC trong RISC-V. Trong định dạng này, trường `imm[31:12]` chứa hằng số tức thời 20 bit, trường `rd` xác định thanh ghi đích và trường `opcode` dùng để phân biệt giữa lệnh LUI và AUIPC.



- LUI: dùng để nạp hằng số tức thời 20 bit vào phần cao của thanh ghi đích, hỗ trợ tạo các hằng số có kích thước lớn.
- AUIPC: dùng để tạo địa chỉ tương đối theo PC bằng cách cộng giá trị PC hiện tại với hằng số tức thời.

M-extension là phần mở rộng trong RISC-V dùng cho nhóm lệnh nhân và chia số nguyên. Thay vì đưa các lệnh này vào ngay tập lệnh cơ sở RV32I, RISC-V tách chúng thành một phần mở rộng riêng để người thiết kế có thể lựa chọn triển khai tùy theo yêu cầu của hệ thống [5].

Khi tích hợp M-extension, CPU có thể thực hiện trực tiếp các phép nhân và chia giữa các thanh ghi số nguyên. Vì vậy, tập lệnh RV32IM phù hợp hơn với các ứng dụng có nhiều phép toán số học, đặc biệt là những thuật toán cần lặp lại các thao tác nhân, chia trong quá trình xử lý dữ liệu.

Các loại lệnh của RV32M

❖ **Lệnh nhân**

Các lệnh nhân trong M-extension sử dụng định dạng R-type, với hai toán hạng được lấy từ các thanh ghi rs1 và rs2, kết quả được ghi vào thanh ghi đích rd. Hình 2.9 minh họa cấu trúc mã hóa lệnh nhân trong RISC-V, trong đó các trường funct7, funct3 và opcode được dùng để xác định từng phép nhân cụ thể như MUL, MULH, MULHSU và MULHU.



Các phép tính nhân cụ thể như sau:

- MUL: phép nhân hai số nguyên trong rs1 và rs2, ghi 32 bit thấp của kết quả vào rd.
- MULH: phép nhân có dấu $\times$ có dấu, ghi 32 bit cao của kết quả vào rd.
- MULHSU: phép nhân có dấu $\times$ không dấu, ghi 32 bit cao của kết quả vào rd.
- MULHU: phép nhân không dấu $\times$ không dấu, ghi 32 bit cao của kết quả vào rd.

❖ **Lệnh chia**

Các lệnh chia trong M-extension cũng sử dụng định dạng R-type, với toán hạng bị chia lấy từ thanh ghi rs1, toán hạng chia lấy từ thanh ghi rs2, và kết quả được ghi vào thanh ghi đích rd. Hình 2.10 minh họa cấu trúc mã hóa lệnh chia trong RISC-V, trong đó các trường funct7, funct3 và opcode được dùng để xác định các phép chia và lấy dư như DIV, DIVU, REM và REMU.

31	25 24	20 19	15 14	12 11	7 6	0
funct7	rs2	rs1	funct3	rd	opcode	
7	5	5	3	5	7	
MULDIV	divisor	dividend	DIV[U]/REM[U]	dest	OP	
MULDIV	divisor	dividend	DIV[U]W/REM[U]W	dest	OP-32	

Hình 2.10: Định dạng mã lệnh chia trong M-extension [5]

Các phép tính chia cụ thể như sau:

- DIV: phép chia hai số nguyên có dấu trong rs1 và rs2, ghi thương vào rd.
- DIVU: phép chia hai số nguyên không dấu trong rs1 và rs2, ghi thương vào rd.
- REM: phép chia hai số nguyên có dấu trong rs1 và rs2, ghi phần dư vào rd.
- REMU: phép chia hai số nguyên không dấu trong rs1 và rs2, ghi phần dư vào rd.

2.1.4. Các phần mở rộng tiêu chuẩn khác của RISC-V

RISC-V có kiến trúc mô-đun, trong đó tập lệnh nền tảng như RV32I hoặc RV64I chỉ chứa các lệnh cơ bản cần thiết. Những chức năng nâng cao được bổ sung thông qua các phần mở rộng, giúp bộ xử lý vừa đơn giản khi thiết kế phần cứng, vừa dễ tùy biến cho nhiều ứng dụng khác nhau.

F-extension: hỗ trợ các phép toán dấu phẩy động đơn chính xác, sử dụng 32 thanh ghi dấu phẩy động 32 bit [6].

D-extension: được mở rộng từ F-extension để hỗ trợ các phép toán dấu phẩy động kép chính xác với thanh ghi dấu phẩy động được mở rộng lên 64 bit [6].

Counters-extension: cung cấp cơ chế theo dõi hiệu năng của bộ xử lý thông qua các bộ đếm được ánh xạ vào CSR. Các thanh ghi như CYCLE, TIME và INSTRET cho phép thu thập số chu kỳ thực thi và số lệnh hoàn thành, từ đó hỗ trợ việc đo đạc, phân tích và so sánh hiệu năng giữa các thiết kế bộ xử lý [6].

Ngoài các mở rộng được kể ở trên thì rất còn nhiều mở rộng khác như A-extension, Q-extension, C-extension và còn rất nhiều phần mở rộng khác nữa. Điều này sẽ giúp tăng tính linh hoạt trong thiết kế.

2.2. KỸ THUẬT PIPELINE

2.2.1. Tổng quan về kỹ thuật pipeline

Kỹ thuật pipeline tổ chức CPU theo cách cho phép nhiều lệnh được xử lý đồng thời, mỗi lệnh ở một giai đoạn khác nhau của chu trình thực thi. Mỗi giai đoạn thực hiện một phần công việc riêng và các giai đoạn hoạt động song song, giúp tăng thông lượng mà không cần tăng độ phức tạp phần cứng. Pipeline có thể được hình dung như dây chuyền sản xuất, nơi dữ liệu liên tục đi qua từng công đoạn. Khi hoạt động ổn định và không có xung đột, CPU có thể hoàn thành gần một lệnh mỗi chu kỳ, đạt hiệu năng gần tỷ lệ với số lượng giai đoạn pipeline.

Trong kiến trúc RISC-V, tiến trình thực thi một lệnh được chia thành năm giai đoạn chính [7]:

- Instruction Fetch (IF): lấy lệnh từ bộ nhớ.
- Instruction Decode (ID): giải mã lệnh và đọc thanh ghi.

- Execute (EX): thực hiện phép toán hoặc tính địa chỉ.
- Memory Access (MEM): truy cập bộ nhớ khi cần.
- Write Back (WB): ghi kết quả vào tập thanh ghi.

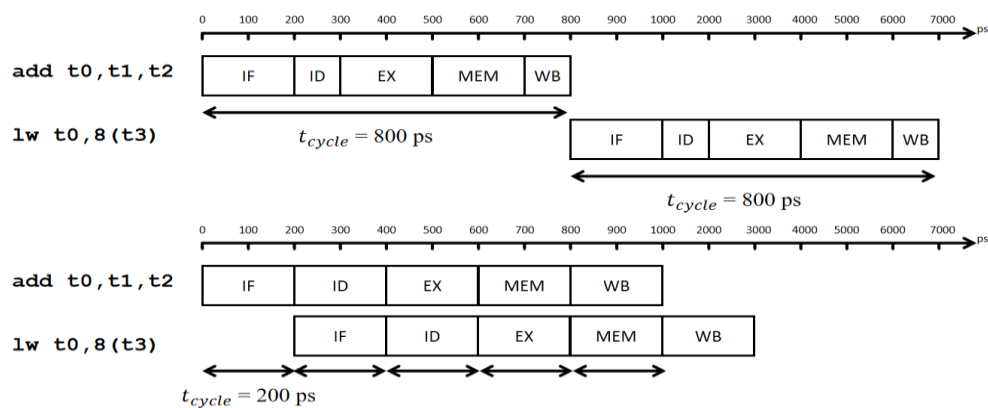
Việc chia nhỏ các bước xử lý giúp rút ngắn đường truyền logic trong mỗi giai đoạn, cho phép tăng tần số hoạt động và tổ chức pipeline hiệu quả hơn. Bảng 2.2 minh họa thời gian cần thiết cho một số loại lệnh trong mô hình pipeline 5 tầng, làm cơ sở để xác định chu kỳ xung của hệ thống.

Bảng 2.2: Minh họa thời gian cần thiết cho một số loại lệnh

Instr	IF = 200ps	ID = 100ps	EX = 200ps	MEM=200ps	WB = 100ps	Total
add	X	X	X		X	600ps
beq	X	X	X			500ps
jal	X	X	X			500ps
lw	X	X	X	X	X	800ps
sw	X	X	X	X		700ps

Lệnh lw có tổng thời gian 800 ps lớn nhất trong các lệnh điều này khiến chu kỳ xung phải tối thiểu bằng thời gian này. Điều đó gây lãng phí vì những lệnh khác cần ít thời gian hơn vẫn phải chờ theo chu kỳ dài nhất.

Để so sánh hiệu quả giữa thực thi tuần tự và thực thi pipeline, Hình 2.11 minh họa thời gian thực thi lệnh trong mô hình pipeline 5 tầng, cho thấy khả năng chồng lấp các giai đoạn nhằm giảm tổng thời gian xử lý.



Hình 2.11: Thời gian thực thi trong mô hình pipeline 5 tầng

Trong mô hình không có pipeline, mỗi lệnh phải hoàn thành tuần tự toàn bộ năm giai đoạn IF-ID-EX-MEM-WB trước khi lệnh tiếp theo bắt đầu. Do lệnh lw

có thời gian thực thi dài nhất (800 ps), chu kỳ xung phải cố định ở mức này, khiến các lệnh nhanh hơn vẫn phải chờ và tổng thời gian xử lý hai lệnh liên tiếp lên tới 1600 ps.

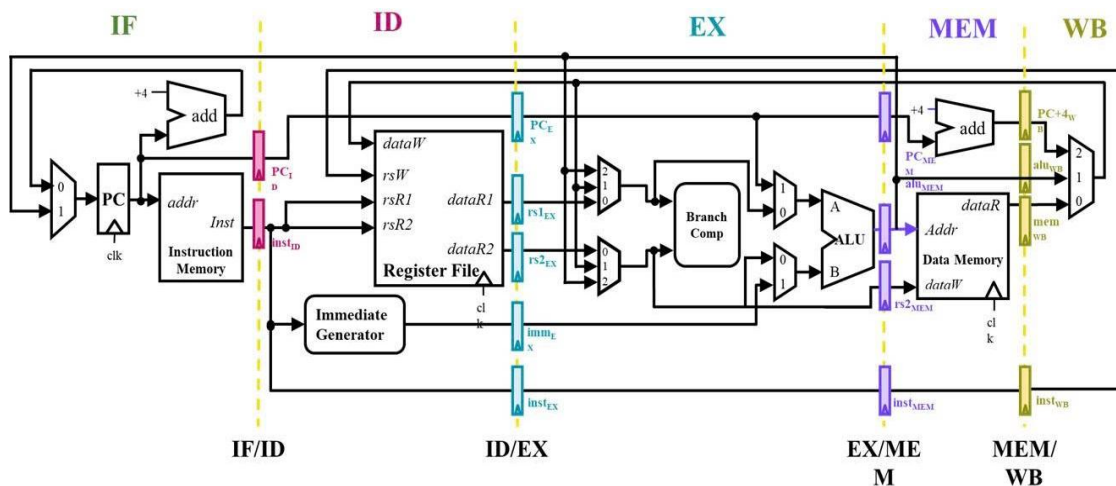
Với pipeline năm giai đoạn, quá trình xử lý được chia nhỏ và các giai đoạn có thể làm việc song song trên các lệnh khác nhau. Chu kỳ xung chỉ còn phụ thuộc vào giai đoạn chậm nhất (200 ps), và lệnh mới có thể được nạp ngay khi giai đoạn IF trống. Nhờ sự chồng lấp giữa các giai đoạn, thời gian hoàn thành chuỗi lệnh giảm đáng kể so với thực thi tuần tự.

Ba lệnh đầu tiên có thể được hoàn tất chỉ trong vài chu kỳ thay vì phải cộng dồn toàn bộ thời gian của từng lệnh. Tuy nhiên, pipeline không mang lại hiệu suất tuyến tính trong mọi trường hợp. Các xung đột dữ liệu, điều khiển và cấu trúc có thể làm gián đoạn pipeline, đòi hỏi các kỹ thuật như forwarding, stall và flush để duy trì tính đúng đắn.

Mặc dù còn tồn tại hạn chế, pipeline lệnh vẫn là yếu tố cốt lõi trong thiết kế RISC-V, cho phép đạt hiệu năng cao với mức phức tạp phần cứng hợp lý.

2.2.2. Hoạt động của pipeline năm tầng

Pipeline năm tầng là mô hình xử lý lệnh tiêu chuẩn trong RISC-V, chia quá trình thực thi thành các giai đoạn IF, ID, EX, MEM và WB nhằm đơn giản hóa phần cứng và cho phép xử lý song song nhiều lệnh. Mô hình này được sử dụng phổ biến trong RV32I và là nền tảng cho các kỹ thuật tối ưu hiệu năng về sau. Hình 2.12 minh họa sơ đồ pipeline năm tầng của bộ xử lý RISC-V.



Hình 2.12: Sơ đồ 5 giai đoạn pipeline của bộ xử lý RISC-V

Giai đoạn IF - Instruction Fetch: giai đoạn IF lấy lệnh từ bộ nhớ chương trình dựa trên giá trị PC và đồng thời tính địa chỉ kế tiếp ($PC + 4$). Các thành phần chính gồm bộ đếm chương trình, bộ nhớ lệnh và mạch tăng PC. IF hoạt động liên tục mỗi chu kỳ, cung cấp dòng lệnh ổn định cho pipeline và phải hỗ trợ cập nhật PC khi xảy ra có các lệnh rẽ nhánh hoặc nhảy.

Giai đoạn ID - Instruction Decode: ID giải mã opcode, đọc các thanh ghi nguồn rs1/rs2 và tạo giá trị immediate cho các định dạng mã lệnh. Giai đoạn này cũng sinh tín hiệu điều khiển và phát hiện hazard, đặc biệt là load-use; nếu cần, pipeline sẽ chèn stall để đảm bảo tính đúng đắn. ID đóng vai trò chuẩn bị đầy đủ dữ liệu trước khi lệnh vào EX.

Giai đoạn EX - Execute / ALU: EX thực hiện các phép tính số học – logic, tính địa chỉ bộ nhớ, kiểm tra điều kiện nhánh và xử lý địa chỉ nhảy. ALU là khối trung tâm của giai đoạn này. Đây cũng là nơi xung đột dữ liệu dễ xảy ra nhất, và cơ chế forwarding/bypass thường được áp dụng để giảm stall. Nếu phân giải nhánh tại EX, pipeline có thể chịu penalty 1-2 chu kỳ.

Giai đoạn MEM - Memory Access: giai đoạn MEM phục vụ các lệnh load/store: load đọc dữ liệu từ địa chỉ tính được, store ghi dữ liệu xuống bộ nhớ. Các lệnh khác chỉ đi qua mà không thực hiện thao tác. Việc tách riêng giai đoạn MEM giúp giảm tranh chấp tài nguyên và cải thiện hiệu năng trong kiến trúc Harvard hoặc hệ thống có cache.

Giai đoạn WB - Write Back: WB ghi kết quả cuối cùng vào thanh ghi đích: dữ liệu từ ALU, dữ liệu load hoặc địa chỉ trả về của lệnh nhảy. Register file thường hỗ trợ đọc sớm - ghi muộn trong cùng chu kỳ để tránh xung đột với giai đoạn ID. Đây là bước hoàn tất vòng đời của một lệnh trong pipeline.

2.2.3. Xung đột trong pipeline

Kỹ thuật pipeline cho phép bộ xử lý thực thi nhiều lệnh song song bằng cách phân chia quá trình xử lý thành các giai đoạn IF-ID-EX-MEM-WB. Mặc dù cải thiện đáng kể thông lượng, pipeline cũng tạo ra những tình huống mà trong đó các lệnh không thể tiến hành đồng thời như mong muốn. Những tình huống này được gọi là xung đột trong pipeline hay còn gọi là pipeline hazards.

Trong kiến trúc vi xử lý, hazards được phân thành ba loại chính [7]:

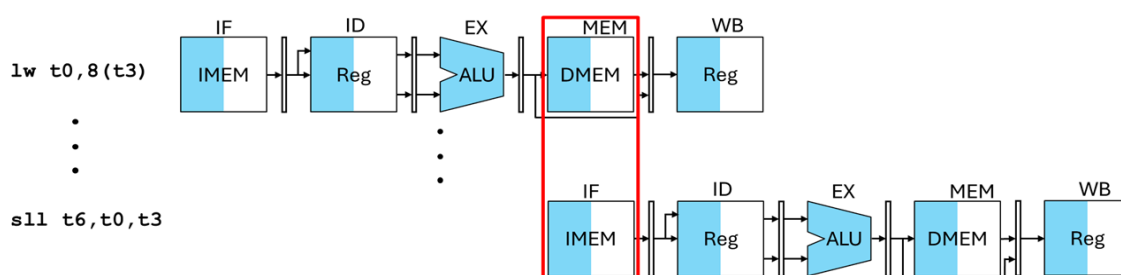
- Xung đột cấu trúc (Structural Hazards)
- Xung đột dữ liệu (Data Hazards)
- Xung đột điều khiển (Control Hazards)

Việc nhận diện và hiểu rõ từng loại hazard là cơ sở quan trọng để thiết kế pipeline RISC-V hiệu quả, giảm số chu kỳ trễ và đảm bảo tính đúng đắn trong thực thi.

Xung đột cấu trúc (Structural Hazards)

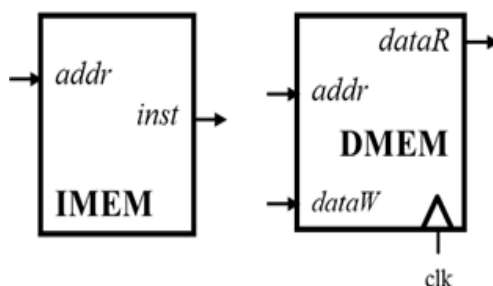
Xung đột cấu trúc xảy ra khi hai hoặc nhiều giai đoạn pipeline yêu cầu truy cập cùng một tài nguyên phần cứng trong cùng một chu kỳ. Ví dụ điển hình là Instruction Memory và Data Memory dùng chung một RAM.

Hình 2.13 dưới minh họa trường hợp lệnh lw đang truy cập bộ nhớ dữ liệu ở giai đoạn MEM, trong khi lệnh kế tiếp lại cần truy cập bộ nhớ lệnh ở giai đoạn IF. Nếu cả hai cùng dùng chung một RAM, tài nguyên này không thể phục vụ đồng thời hai yêu cầu → dẫn đến structural hazard và pipeline phải chèn stall.



Hình 2.13: Xung đột cấu trúc khi Instruction Memory và Data Memory dùng chung RAM

Một cách khắc phục xung đột cấu trúc là tách riêng Instruction Memory và Data Memory theo mô hình Harvard. Hình 2.14 minh họa phần cứng riêng biệt cho hai bộ nhớ, cho phép pipeline truy cập song song mà không gây tranh chấp.



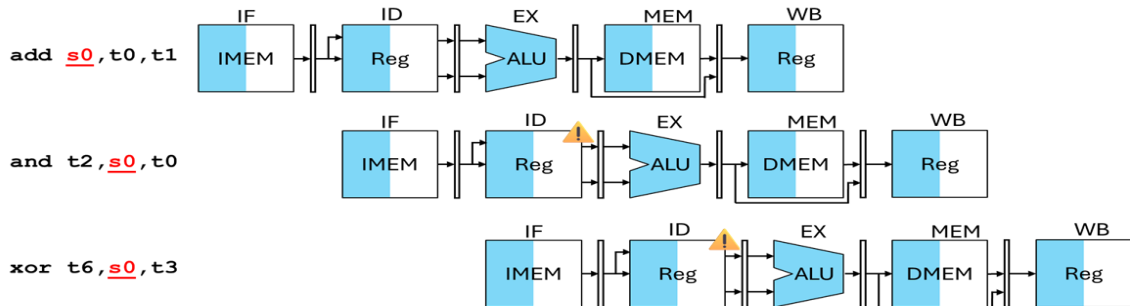
Hình 2.14: Phần cứng riêng biệt của Instruction Memory và Data Memory

Xung đột dữ liệu (Data Hazards)

❖ Data Hazards I

Xung đột dữ liệu xảy ra khi một lệnh cần sử dụng giá trị mà lệnh trước đó chưa kịp tạo ra hoặc chưa kịp ghi vào thanh ghi đích. Trong pipeline 5 tầng, kết quả tính toán của lệnh thường chỉ sẵn sàng ở các giai đoạn sau, nên nếu lệnh kế tiếp cần sử dụng giá trị đó quá sớm sẽ dẫn đến xung đột dữ liệu [3,7].

Hình 2.15 minh họa xung đột dữ liệu trong pipeline 5 tầng khi lệnh sau sử dụng toán hạng được tạo ra bởi lệnh trước.

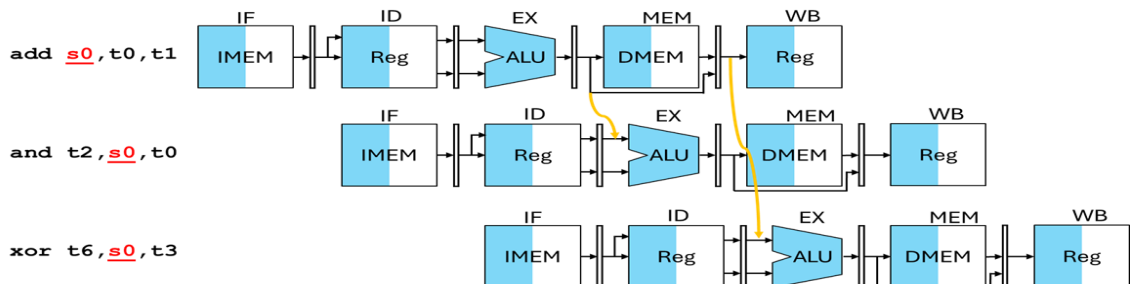


Hình 2.15: Minh họa xung đột dữ liệu (Data Hazard)

Như hình ảnh trên hai lệnh and và xor đều sử dụng giá trị s0 được tạo ra bởi lệnh add. Tuy nhiên, giá trị này chỉ xuất hiện sau khi add đi qua các giai đoạn EX hoặc MEM. Khi lệnh and đi vào giai đoạn ID, dữ liệu s0 vẫn chưa sẵn sàng, gây ra xung đột dữ liệu.

Để tránh pipeline phải dừng lại, kiến trúc pipeline sử dụng một giải pháp kỹ thuật forwarding. Forwarding cho phép đưa trực tiếp kết quả từ các giai đoạn EX/MEM hoặc MEM/WB về đầu vào ALU của lệnh phía sau mà không cần chờ ghi vào thanh ghi. Kỹ thuật này giúp pipeline duy trì tính liên tục và giảm số chu kỳ bị stall.

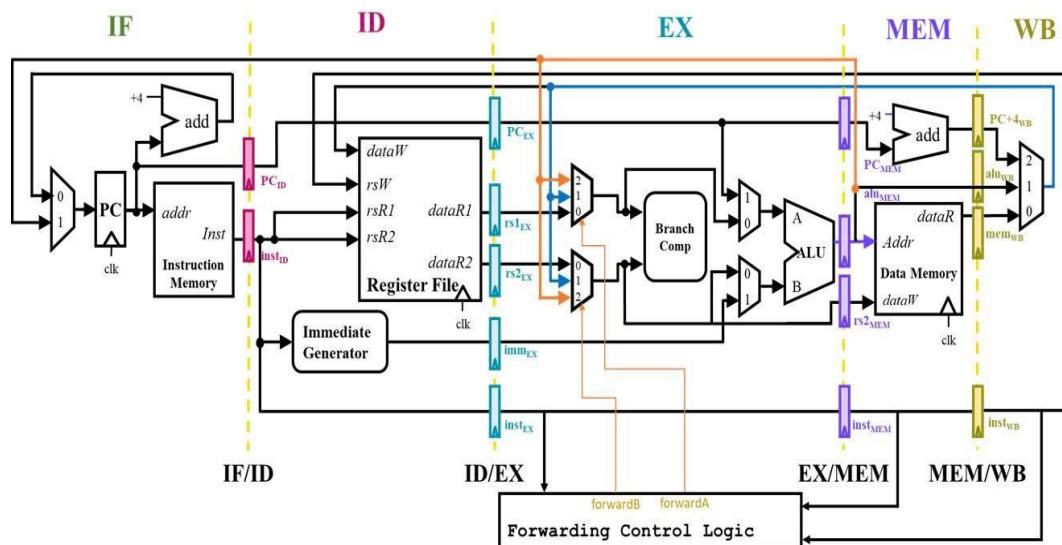
Hình 2.16 minh họa cơ chế forwarding trong pipeline 5 tầng của RISC-V.



Hình 2.16: Forwarding dữ liệu từ các giai đoạn sau về ALU

Để thực hiện kỹ thuật forwarding, pipeline cần một khối điều khiển chuyên trách nhằm phát hiện và xử lý các phụ thuộc dữ liệu.

Hình 2.17 minh họa sơ đồ tổng quát của Forwarding Control Logic trong pipeline 5 tầng, thể hiện các đường chuyển tiếp dữ liệu giữa các giai đoạn.



Hình 2.17: Sơ đồ tổng quát của Forwarding Control Logic

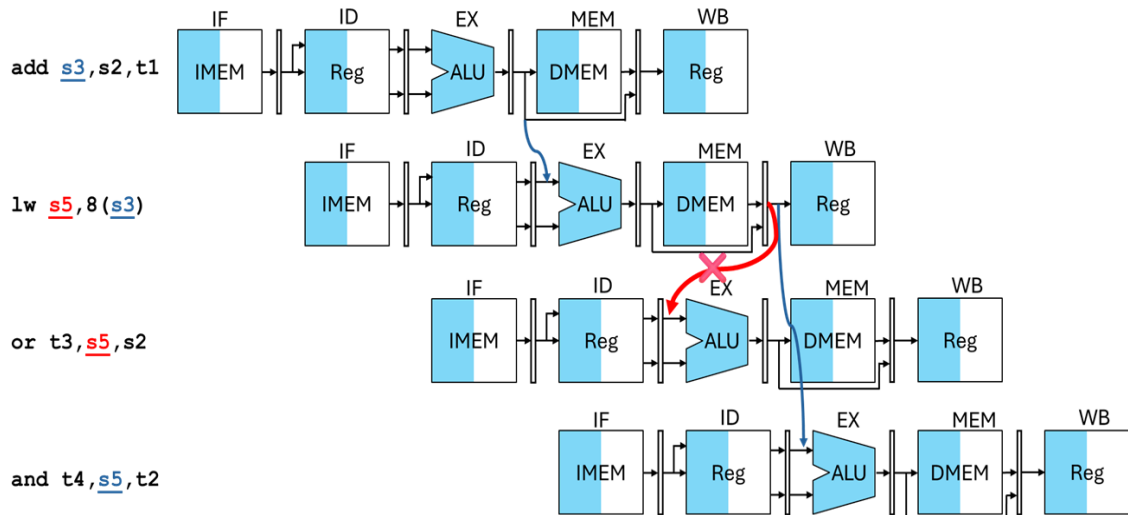
Sơ đồ cho thấy khi lệnh ở giai đoạn EX cần một giá trị mà lệnh phía trước vừa tạo ra nhưng chưa kịp ghi về thanh ghi, Forwarding Control Logic sẽ so sánh các thanh ghi nguồn của lệnh hiện tại với thanh ghi đích của các lệnh ở EX/MEM và MEM/WB.

Nếu phát hiện phụ thuộc dữ liệu, logic sẽ điều khiển các MUX qua tín hiệu forwardA và forwardB để đưa trực tiếp kết quả từ EX/MEM hoặc MEM/WB về đầu vào ALU, giúp lệnh đang EX nhận đúng dữ liệu mà không cần stall.

❖ Data Hazards II: Load-Use Hazard

Load-use hazard xảy ra khi một lệnh cần sử dụng dữ liệu được nạp từ bộ nhớ bởi lệnh load ngay trước đó, trong khi giá trị này chưa sẵn sàng tại giai đoạn EX.

Hình 2.18 minh họa trường hợp load-use hazard trong pipeline 5 tầng, nơi forwarding không thể cung cấp dữ liệu kịp thời và pipeline phải chèn stall.



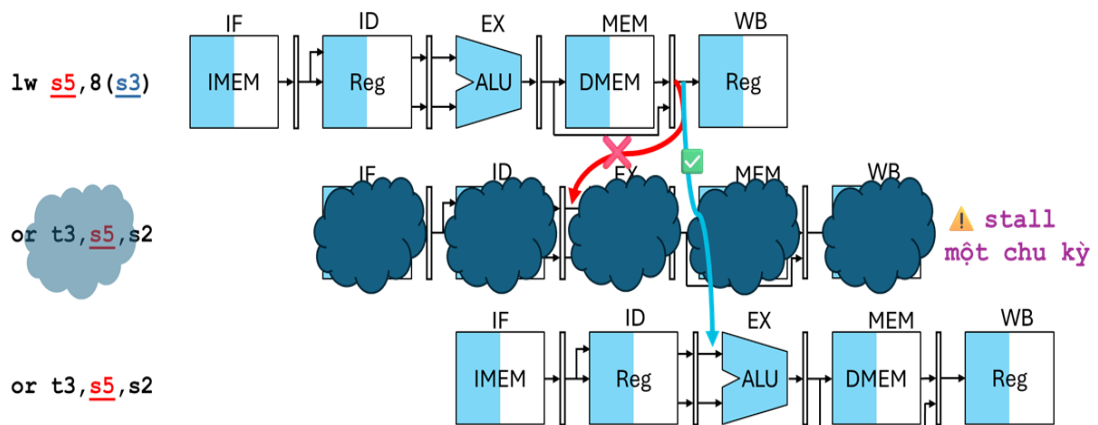
Hình 2.18: Minh họa Load-Use hazard

Trong chu kỳ mà lệnh `or` đi vào giai đoạn EX, giá trị của `s5` vẫn đang được tải từ bộ nhớ và chỉ sẵn sàng ở cuối giai đoạn MEM. Vì vậy, forwarding không thể cung cấp dữ liệu đúng thời điểm.

Giá trị được tải từ bộ nhớ chỉ sẵn sàng ở cuối giai đoạn MEM, trong khi lệnh phụ thuộc cần dữ liệu này ngay tại đầu giai đoạn EX, do đó forwarding không thể cung cấp dữ liệu kịp thời.

Giải pháp: để đảm bảo dữ liệu chính xác, pipeline buộc phải tạm dừng (stall) PC và IF/ID trong một chu kỳ và chèn một bubble vào giai đoạn ID/EX để trì hoãn lệnh phụ thuộc [3,7].

Hình 2.19 minh họa cơ chế chèn stall một chu kỳ nhằm xử lý load-use hazard trong pipeline 5 tầng.

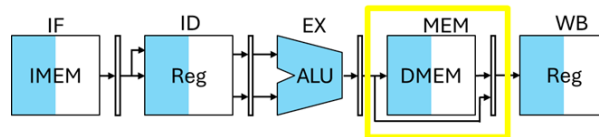


Hình 2.19: Stall 1 chu kỳ để xử lý Load-Use

Xung đột điều khiển (Control Hazards)

Xung đột điều khiển xuất hiện khi pipeline gặp các lệnh có khả năng thay đổi giá trị PC, chẳng hạn như các lệnh nhánh có điều kiện (B-type), lệnh nhảy JAL hoặc JALR. Trong khi pipeline vẫn tiếp tục lấy lệnh theo hướng hiện tại, kết quả điều kiện của nhánh chỉ được xác định ở giai đoạn ID hoặc EX. Nếu dự đoán sai hướng nhánh, các lệnh đã được nạp trước đó sẽ phải bị loại bỏ (flush), gây mất chu kỳ thực thi [3,7].

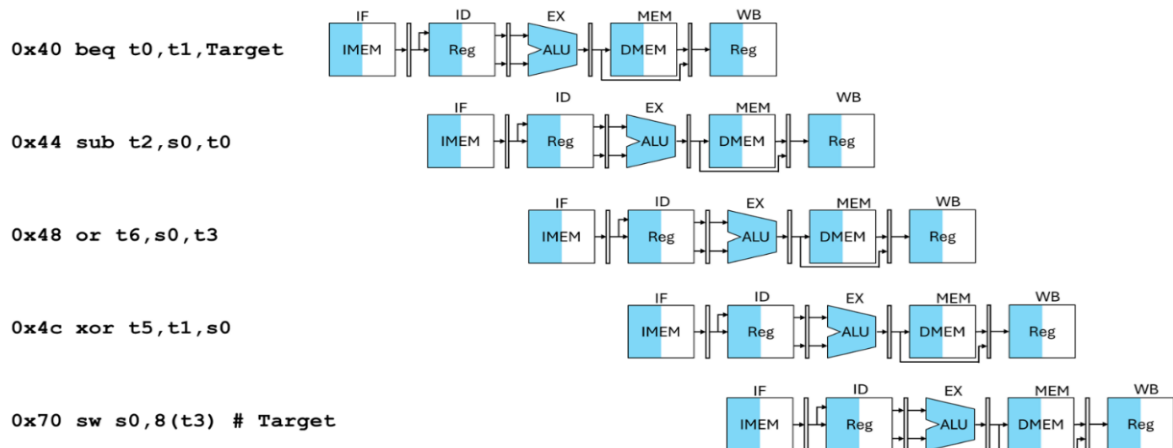
Hình 2.20 minh họa thời điểm hướng nhánh được xác định trong pipeline 5 tầng, dẫn đến nguy cơ phát sinh control hazard.



Hình 2.20: Lệnh nhánh chỉ quyết định hướng đi sau khi hoàn tất EX

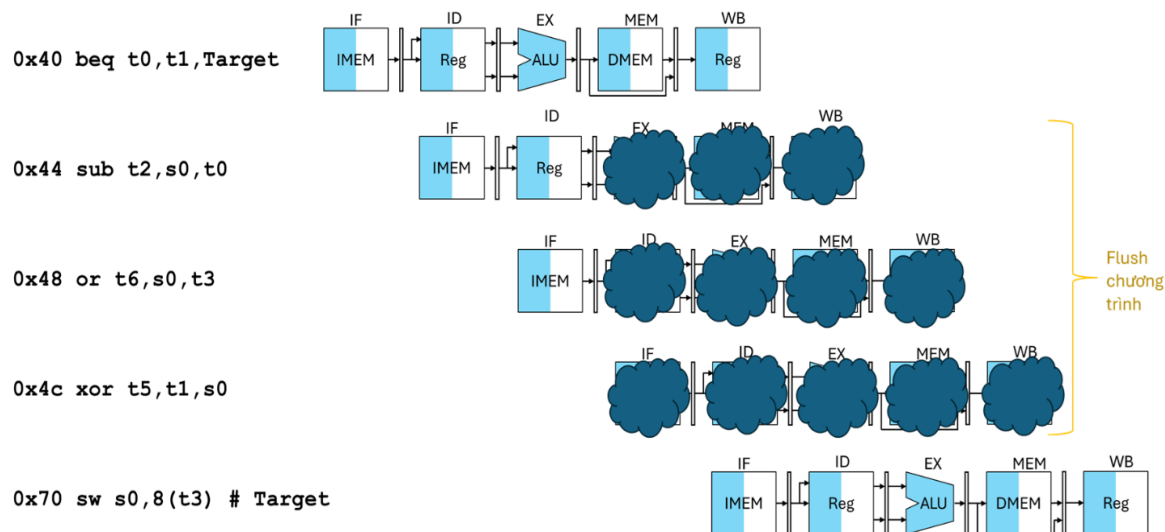
Giải pháp là dự đoán hướng đi của PC tức là Branch Prediction đơn giản trong pipeline 5 tầng cơ bản, hướng xử lý phổ biến là dự đoán không nhảy đến lệnh tiếp theo. Pipeline tiếp tục lấy các lệnh tuần tự cho đến khi kết quả nhánh được xác định. Nếu dự đoán đúng, pipeline tiếp tục hoạt động bình thường. Nếu dự đoán sai, các lệnh đã fetch sai đường sẽ bị flush và pipeline chuyển sang fetch lệnh đúng tại địa chỉ đích của nhánh.

Trường hợp dự đoán đúng thì pipeline tiếp tục xử lý bình thường, không có stall và không có flush. Hình 2.21 minh họa hoạt động bình thường của pipeline khi dự đoán hướng nhánh là đúng.



Hình 2.21: Pipeline hoạt động bình thường khi dự đoán đúng hướng nhánh

Trong trường hợp dự đoán sai hướng nhánh, khi kết quả nhánh được xác định thường tại giai đoạn EX, các lệnh đã được nạp theo hướng sai sẽ bị loại bỏ (flush) và pipeline tiếp tục nạp lệnh từ địa chỉ đích của nhánh. Hình 2.22 minh họa cơ chế flush pipeline khi dự đoán sai hướng nhánh trong pipeline 5 tầng.



Hình 2.22: Flush pipeline khi dự đoán sai hướng nhánh

Trong hình, lệnh beq t0, t1, Target tại địa chỉ 0x40 được xác định là nhánh được lấy, nhưng pipeline đã dự đoán không nhảy. Do đó, các lệnh sub t2, s0, t0, or t6, s0, t3 và xor t5, t1, s0 được nạp theo luồng sai và phải bị flush. Sau khi nhánh được xác nhận, pipeline tiếp tục nạp và thực thi lệnh đúng tại địa chỉ đích sw s0, 8 (t3).

2.3. KỸ THUẬT SIMD VÀ P-EXTENSION

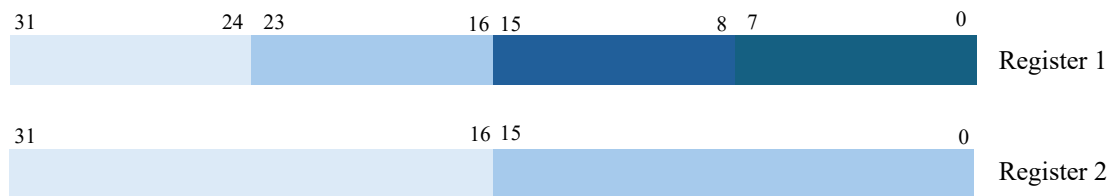
2.3.1. Giới thiệu về SIMD

SIMD (Single Instruction Multiple Data) là mô hình xử lý song song mức dữ liệu, trong đó một lệnh duy nhất được sử dụng để thực hiện cùng một phép toán trên nhiều phần tử dữ liệu độc lập. Nhờ khai thác tính song song mức dữ liệu, SIMD đặc biệt phù hợp với các bài toán có cấu trúc lặp trên nhiều phần tử như phép toán ma trận, xử lý ảnh, xử lý âm thanh và các thuật toán học máy [8]. Trong RISC-V, SIMD có thể được thể hiện thông qua hai phần mở rộng tiêu biểu là P-extension và V-extension.

2.3.2. Tổng quan về P-extension

P-extension là phần mở rộng tập lệnh được thiết kế để hỗ trợ xử lý song song mức dữ liệu theo kỹ thuật packed-SIMD [6]. Nó cho phép đóng gói nhiều phần tử dữ liệu có độ rộng nhỏ vào cùng một thanh ghi số nguyên và xử lý đồng thời bằng một lệnh. So với V-extension thì P-extension có cách tiếp cận đơn giản hơn do tận dụng trực tiếp các thanh ghi số nguyên hiện có thay vì sử dụng một tập thanh ghi vector riêng [9]. Vì vậy, P-extension phù hợp với các hệ thống nhúng cần cải thiện hiệu quả xử lý DSP nhưng vẫn phải giới hạn độ phức tạp và tài nguyên phần cứng.

Hình 2.23 mô tả các phương án đóng gói dữ liệu trong thanh ghi 32 bit của P-extension. Một thanh ghi có thể được chia thành bốn phần tử 8 bit hoặc hai phần tử 16 bit để thực hiện các phép toán packed-SIMD song song.



Hình 2.23: Các cách đóng gói dữ liệu trong thanh ghi 32 bit của P-extension

Tóm lại, P-extension được đề xuất nhằm cải thiện khả năng xử lý các thuật toán xử lý tín hiệu số hay xử lý ảnh trên bộ xử lý RISC-V thông qua các lệnh packed-SIMD và một số lệnh hỗ trợ phép toán số học chuyên dụng. Tuy nhiên, P-extension hiện vẫn chưa phải là một phần mở rộng đã được phê chuẩn chính thức mà nó chỉ mới là bản nháp. Theo tài liệu đặc tả vẫn đang được phát triển, do đó cấu trúc tập lệnh, cách đặt tên lệnh và một số nội dung kỹ thuật vẫn có thể tiếp tục thay đổi. Phiên bản tài liệu đặc tả được sử dụng trong đề tài là phiên bản 0.20-draft [10]. Vì phạm vi của P-extension tương đối lớn, đề tài chỉ lựa chọn một nhóm lệnh phù hợp để tích hợp và kiểm chứng trên bộ xử lý được thiết kế.

2.3.2. Các nhóm lệnh cơ bản của P-extension

P-extension mở rộng tập lệnh RISC-V bằng cách bổ sung nhiều lệnh xử lý dữ liệu song song trên các phần tử nhỏ trong cùng một thanh ghi. Thay vì chỉ thực

hiện phép toán trên toàn bộ một toán hạng 32 bit, các lệnh này cho phép chia dữ liệu thành nhiều lane 8 bit hoặc 16 bit và xử lý đồng thời trên từng lane.

Bảng 2.3 mô tả các nhóm lệnh cơ bản của P-extension. Các lệnh được chia làm 6 nhóm chính, mỗi nhóm bao gồm một số lệnh tiêu biểu kèm theo chức năng hoạt động của chúng.

Bảng 2.3: Các nhóm lệnh cơ bản của P-extension

Nhóm lệnh	Hoạt động	Ví dụ
Cộng/trừ packed-SIMD	Cộng/trừ độc lập theo từng lane byte hoặc halfword.	PADD.B, PSUB.B, PADD.H, PSUB.H,...
Trung bình và bão hòa	Tính trung bình hoặc giới hạn kết quả khi xảy ra tràn số.	PAADD.B, PASUB.H, PSADD.H, PSSUB.B,...
Xử lý chéo halfword	Cộng/trừ giữa các phần tử ở vị trí chéo nhau (nửa 16 bit).	PAS.HX, PSA.HX, PASA.HX, PSSA.HX,...
So sánh và min/max	So sánh độc lập trên từng lane, chọn phần tử nhỏ nhất/lớn nhất.	PMSEQ.H, PMSLT.H, PMIN.H, PMAXU.H,...
Trị tuyệt đối và clip	Lấy trị tuyệt đối và giới hạn miền giá trị biểu diễn (clip).	PSABS.H, PSATI.H, PUSATI.H
Nhân cộng dồn	Thực hiện phép nhân kết hợp cộng dồn nhiều cặp dữ liệu.	PM4ADDA.B, PM4ADDAU.B, PM4ADDASU.B

Trong các lệnh packed-SIMD cơ bản, tiền tố P cho biết phép toán được thực hiện song song trên nhiều phần tử dữ liệu. Hậu tố B và H lần lượt biểu thị phần tử byte và halfword. Ví dụ như PADD.B thực hiện phép cộng độc lập trên từng byte, trong khi PADD.H thực hiện phép cộng trên từng halfword.

Cộng/trừ packed-SIMD

Trong P-extension, các phép cộng và trừ không chỉ được xử lý theo cách thông thường mà còn có thể áp dụng nhiều cơ chế khác nhau tùy theo yêu cầu dữ liệu. Với kiểu quay vòng, các bit vượt quá độ rộng biểu diễn sẽ bị bỏ qua, vì vậy kết quả chỉ còn phần nằm trong số bit cho phép. Do phép trừ có thể được xây dựng dựa trên phép cộng với số bù hai, các lệnh trừ trong P-extension cũng được tổ chức theo các nhóm tương tự lệnh cộng, bao gồm dạng có dấu, không dấu và các biến thể làm việc với dữ liệu 8 bit hoặc 16 bit [11].

Trung bình và bão hòa

Để hạn chế việc mất mát dữ liệu do tràn số khi cộng/trừ, P-extension cung cấp thêm các biến thể xử lý trung bình và bão hòa. Với kiểu lấy trung bình, kết quả sau phép tính được dịch phải một bit để giảm một nửa giá trị, qua đó làm giảm nguy cơ vượt ngưỡng. Ngoài ra, cơ chế bão hòa cho phép chặn kết quả tại giới hạn trên hoặc giới hạn dưới của miền biểu diễn nếu xảy ra tràn.

Xử lý chéo halfword

Trong kỹ thuật packed-SIMD, hiệu quả xử lý phụ thuộc nhiều vào cách các phần tử dữ liệu được bố trí trong thanh ghi. Nếu dữ liệu chưa nằm đúng vị trí, bộ xử lý phải sử dụng thêm các lệnh sắp xếp hoặc hoán đổi trước khi thực hiện phép tính chính. Để hạn chế chi phí này, P-extension đưa vào các lệnh có khả năng thao tác chéo trên các nửa 16 bit của hai thanh ghi nguồn, giúp việc ghép và chọn dữ liệu diễn ra linh hoạt hơn.

So sánh và min/max

Nhóm lệnh so sánh trong P-extension thực hiện phép so sánh độc lập trên từng lane dữ liệu. Đối với dữ liệu 16 bit, hai phần tử trong thanh ghi nguồn Rs1 được lần lượt so sánh với các phần tử tương ứng trong thanh ghi Rs2. Tùy theo loại lệnh, điều kiện so sánh có thể là bằng nhau, nhỏ hơn hoặc nhỏ hơn hoặc bằng. Nếu điều kiện được thỏa mãn, lane kết quả được gán toàn bộ bit bằng 0xFFFF hoặc ngược lại thì được gán toàn bộ bit bằng 0 [11].

Bên cạnh đó, P-extension còn hỗ trợ các lệnh MIN và MAX. Các lệnh này cũng thực hiện so sánh theo từng lane, nhưng thay vì trả về giá trị logic thì kết quả là phần tử nhỏ hơn hoặc lớn hơn giữa hai toán hạng đầu vào.

Trị tuyệt đối và clip

Lệnh trị tuyệt đối được sử dụng để tính giá trị tuyệt đối của các phần tử có dấu trong thanh ghi nguồn. Đối với dữ liệu 16 bit, hai phần tử trong thanh ghi Rs1 được xử lý độc lập trên từng lane và kết quả được ghi vào vị trí tương ứng của thanh ghi Rd. Trường hợp đầu vào bằng 0x8000 tức là -32768 thì giá trị dương tương ứng vượt quá miền biểu diễn 16 bit có dấu, do đó kết quả được bão hòa về 0x7FFF tức là 32767 [11].

Lệnh clip hay còn gọi lệnh giới hạn dữ liệu được sử dụng để giới hạn giá trị của từng phần tử dữ liệu trong một khoảng xác định. Đối với lệnh clip có dấu, miền giá trị được thiết lập dựa trên giá trị tức thời imm với cận dưới là -2^{imm} và cận trên là $2^{imm} - 1$. Nếu giá trị đầu vào vượt ra ngoài khoảng này thì kết quả sẽ được đưa về giá trị biên tương ứng.

Nhân cộng dồn

Lệnh nhân cộng dồn (Multiply-Accumulate - MAC) được sử dụng phổ biến trong các thuật toán xử lý tín hiệu số, đặc biệt là các phép tính có dạng tổng của nhiều tích. Với kiểu dữ liệu 8 bit, mỗi thanh ghi nguồn được tách thành bốn phần tử nhỏ. Các phần tử tương ứng giữa hai thanh ghi được nhân đồng thời, sau đó kết quả của bốn phép nhân được cộng lại thành một giá trị tổng và tiếp tục cộng vào nội dung hiện có của thanh ghi đích.

2.3.3. Ưu điểm và hạn chế của P-extension

Ưu điểm chính của P-extension bao gồm:

- Tăng khả năng song song dữ liệu: Cho phép thực hiện cùng lúc nhiều phép toán trên nhiều phần tử dữ liệu như 8 bit hoặc 16 bit bằng cách đóng gói vào thanh ghi, từ đó giảm số lượng lệnh cần thực thi và tăng tốc độ xử lý.
- Tận dụng thanh ghi số nguyên sẵn có: P-extension không yêu cầu tập thanh ghi vector riêng mà khai thác trực tiếp các thanh ghi số nguyên

hiện có, nhờ đó giúp đơn giản hóa thiết kế phần cứng và giảm chi phí tài nguyên so với các kiến trúc vector phức tạp hơn.

Các hạn chế của P-extension bao gồm:

- Giới hạn song song: Số phần tử dữ liệu có thể xử lý đồng thời phụ thuộc vào độ rộng thanh ghi, không giống như V-extension thì nó sẽ hỗ trợ xử lý vector với độ dài linh hoạt hơn [6].
- Yêu cầu dữ liệu liên tiếp: Hiệu quả cao nhất khi các phần tử dữ liệu được sắp xếp liên tiếp và nằm chính xác trong bộ nhớ. Trong thực tế, dữ liệu thường không nằm đúng vị trí mong muốn, do đó chương trình cần thêm bước ghép bit để chuẩn bị dữ liệu trước khi thực hiện SIMD. Điều này làm giảm tốc độ xử lý của P-extension.

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

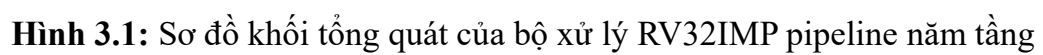
3.1. TỔNG QUAN KIẾN TRÚC LỖI RV32IMP

Chương này trình bày thiết kế core RISC-V theo cấu hình RV32IMP. Hệ thống được chia làm hai phần chính: lõi xử lý nền tảng RV32IM pipeline 5 tầng, và khối P-extension SIMD Pext chuyên trách thực thi tập lệnh packed-SIMD.

Thay vì thiết kế khối SIMD như một bộ đồng xử lý độc lập, hệ thống RV32IMP đặt trực tiếp SIMD Pext tại tầng EX. Sự lựa chọn này loại bỏ hoàn toàn độ trễ giao tiếp và chi phí truyền dữ liệu giữa lõi chính và khối SIMD, cho phép các lệnh packed-SIMD thực thi và hoàn thành trong cùng một chu kỳ với các lệnh thông thường. Hệ thống này hướng tới các ứng dụng xử lý tín hiệu số và xử lý ảnh trên các nền tảng nhúng có tài nguyên hạn chế, nơi dữ liệu thường là các vector 8 bit hoặc 16 bit.

3.1.1. Sơ đồ khối

Hình 3.1 trình bày sơ đồ khối tổng quát của hệ thống RV32IMP. Tầng EX chứa hai khối thực thi song song: ALU xử lý nhóm lệnh RV32IM và SIMD Pext xử lý nhóm lệnh packed-SIMD. Kết quả từ hai khối này được bộ đa hợp chọn thông qua tín hiệu điều khiển rdSel trước khi truyền sang các tầng sau.



Các khối chính của hệ thống gồm:

- Program Counter và Instruction Memory: giữ địa chỉ lệnh hiện tại, tính $PC + 4$ và nạp lệnh từ bộ nhớ chương trình.
- Decoder và Control Logic: phân tách các trường lệnh và sinh tín hiệu điều khiển cho pipeline.
- Immediate Generator: tạo hằng số (immediate) cho các định dạng lệnh RV32I/M và nhóm P-extension.
- Register File: gồm 32 thanh ghi 32 bit, được bổ sung cổng đọc thứ ba cho thanh ghi đích (rd) phục vụ riêng nhóm lệnh nhân cộng dồn (MAC).
- ALU: xử lý các phép toán RV32I/M, tính địa chỉ load/store và địa chỉ nhảy branch/jump.
- SIMD Pext: xử lý các phép toán packed-SIMD trên dữ liệu 8 bit và 16 bit.
- Branch Comparator và Branch Control: quyết định hướng đi của PC đối với branch/jump.
- Hazard Unit: xử lý các loại xung đột trong pipeline bao gồm xung đột dữ liệu hay còn gọi là Data Hazard, xung đột điều khiển hay còn gọi là Control Hazard và load-use hazard.
- Data Memory và Write Back MUX: truy cập dữ liệu bộ nhớ và chọn dữ liệu ghi về Register File.

3.1.2. Hoạt động của toàn hệ thống

Tầng IF (Instruction Fetch): PC cấp địa chỉ đọc lệnh từ Instruction Memory và bộ cộng tự cập nhật địa chỉ $PC+4$. Nếu không xuất hiện stall do xung đột dữ liệu, PC được cập nhật tuần tự mỗi chu kỳ. Khi một lệnh rẽ nhánh hoặc nhảy (branch/jump) được lấy tại tầng EX, tín hiệu PCSel sẽ định hướng lấy địa chỉ mới từ ALU, đồng thời các lệnh đã đi vào pipeline bị xóa bỏ (flush).

Tầng ID (Instruction Decode): Lệnh được giải mã để phân tách các trường tín hiệu. Control Logic sinh tín hiệu điều khiển cho toàn bộ datapath. Immediate Generator mở rộng số tức thời. Với các lệnh P-extension, Control Logic tạo tín hiệu ALUSel cho SIMD Pext và cấu hình cờ rdSel để đón kết quả từ SIMD Pext

thay vì ALU. Đặc biệt, nếu lệnh giải mã là nhóm MAC có sử dụng thêm cổng đọc dataR3 của Register File để truy xuất giá trị hiện tại của thanh ghi đích.

Tầng EX (Execute): ALU và SIMD Pext hoạt động song song. ALU thực hiện các tính toán số nguyên cơ bản, tính địa chỉ load/store và branch, các phép toán của M extension. Trong khi đó, SIMD Pext xử lý các phép toán packed-SIMD. Các toán hạng đi vào tầng EX được chọn bởi bộ Forwarding Unit để lấy kết quả mới nhất. Đối với lệnh MAC, đường forwardACC chuyên biệt sẽ đảm bảo SIMD Pext nhận đúng giá trị tích lũy mới nhất mà không gây hazard RAW.

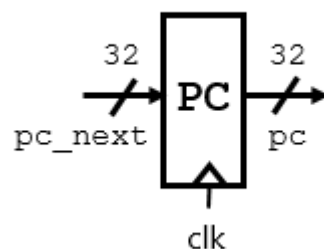
Tầng MEM: Data Memory chỉ được kích hoạt bởi lệnh Load/Store. Các lệnh không truy cập bộ nhớ, bao gồm toàn bộ lệnh P-extension, chỉ truyền kết quả thực thi đi qua tầng này.

Tại tầng WB, dữ liệu ghi ngược về Register File được bộ MUX quyết định lấy từ kết quả ALU, dữ liệu đọc từ bộ nhớ hoặc PC+4.

3.2. THIẾT KẾ LỖI CƠ SỞ RV32IM

3.2.1. Program Counter

Hình 3.2 minh họa cấu trúc khối Program Counter (PC), bộ thanh ghi trung tâm chịu trách nhiệm xác định địa chỉ lệnh kế tiếp.



Hình 3.2: Program Counter

Program Counter (PC) là một trong những thành phần nền tảng của mọi kiến trúc xử lý lệnh tuần tự, có nhiệm vụ xác định vị trí của lệnh kế tiếp trong bộ nhớ. Trong thiết kế RV32IM, PC được triển khai dưới dạng một thanh ghi 32 bit, phản ánh không gian địa chỉ theo đơn vị byte của bộ nhớ lệnh.

PC hoạt động hoàn toàn theo cơ chế đồng bộ với xung nhịp hệ thống. Tại mỗi cạnh lên của tín hiệu clk, PC cập nhật giá trị mới từ tín hiệu pc_next. Giá trị

này được tính toán theo từng chu kỳ dựa trên loại lệnh đang thực thi. Nó có thể là $PC+4$ trong các lệnh tuần tự, địa chỉ đích của nhánh có điều kiện hoặc địa chỉ đích của lệnh nhảy.

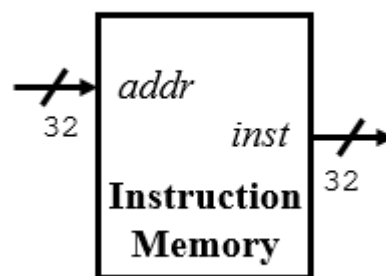
Bảng 3.1 mô tả các tín hiệu của Program Counter bao gồm các tín hiệu như là clk , pc_next và pc .

Bảng 3.1: Mô tả các tín hiệu của Program Counter

Tín hiệu	Mô tả
clk	Tín hiệu clock cạnh lên
pc_next	Giá trị pc tiếp theo được nạp vào, thường là $pc+4$
pc	Giá trị pc hiện tại

3.2.2. Instruction Memory

Hình 3.3 minh họa cấu trúc khối Instruction Memory, bộ nhớ dùng để truy xuất lệnh dựa trên địa chỉ do PC cung cấp.



Hình 3.3: Instruction Memory

Instruction Memory là mô-đun lưu trữ mã lệnh của chương trình, thường được xây dựng dưới dạng bộ nhớ chỉ đọc hoặc mảng memory được khởi tạo trước. Trong kiến trúc RV32IM, nó được tổ chức theo từ 32 bit, tương ứng với độ rộng chuẩn của các lệnh trong tập lệnh RV32IM.

Dựa trên địa chỉ đầu vào là giá trị pc do Program Counter cung cấp, Instruction Memory truy xuất và xuất ra nội dung lệnh là $inst$ tương ứng, đảm bảo bộ xử lý luôn nhận được lệnh kế tiếp để đưa vào giai đoạn giải mã và thực thi.

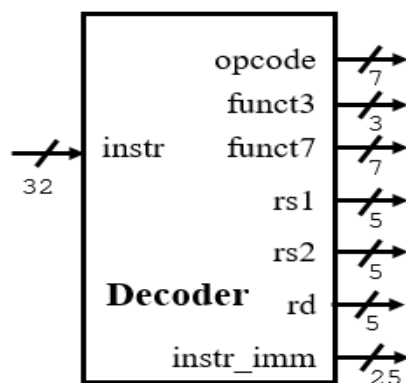
Bảng 3.2 mô tả các tín hiệu của Instruction Memory bao gồm các tín hiệu như là addr và inst.

Bảng 3.2: Mô tả các tín hiệu của Instruction Memory

Tín hiệu	Mô tả
addr	Địa chỉ lệnh hay còn gọi là giá trị PC hiện tại
inst	Mã lệnh được truy xuất dựa trên giá trị PC tương ứng

3.2.3. Decoder

Hình 3.4 minh họa cấu trúc khối Decoder trong bộ xử lý. Khối này nhận lệnh 32 bit từ ngõ vào instr và tách ra các trường cần thiết cho quá trình điều khiển và thực thi lệnh.



Hình 3.4: Decoder

Decoder đảm nhiệm chức năng giải mã lệnh trong kiến trúc RV32IM. Từ lệnh đầu vào 32 bit, khối này tách các trường opcode, funct3, funct7, rs1, rs2, rd và instr_imm. Các tín hiệu sau giải mã được đưa đến khối khác để xác định loại lệnh, toán hạng nguồn, thanh ghi đích và phép toán cần thực hiện. Do đó, Decoder đóng vai trò trung gian quan trọng giữa giai đoạn lấy lệnh và các giai đoạn xử lý tiếp theo trong datapath.

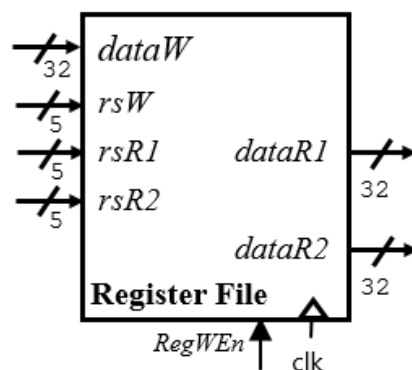
Bảng 3.3 mô tả các tín hiệu của Decoder bao gồm các tín hiệu như là instr, opcode, funct3, funct7, rs1, rs2, rd và instr_imm.

Bảng 3.3: Mô tả các tín hiệu của Decoder

Tín hiệu	Mô tả
instr	Mã lệnh được nhận từ bộ nhớ lệnh
opcode	Trường mã lệnh chính dùng để xác định loại lệnh cần thực hiện
funct3	Trường chức năng 3 bit dùng để phân biệt chi tiết các biến thể của lệnh
funct7	Trường chức năng 7 bit hỗ trợ xác định phép toán trong một số nhóm lệnh
rs1	Địa chỉ thanh ghi nguồn thứ nhất
rs2	Địa chỉ thanh ghi nguồn thứ hai
rd	Địa chỉ thanh ghi đích, nơi lưu kết quả sau khi lệnh được thực thi
instr_imm	Mã lệnh đã được giải mã cho Immediate Generator

3.2.4. Register File

Hình 3.5 minh họa cấu trúc khối Register File với hai cổng đọc và một cổng ghi, cho phép truy xuất song song các thanh ghi.



Hình 3.5: Register File

Register File trong kiến trúc RV32IM được tổ chức dưới dạng một mảng gồm 32 thanh ghi, mỗi thanh ghi rộng 32 bit. Trong đó, thanh ghi x0 được cố định ở giá trị 0 và không thể bị ghi đè. Mô-đun này cung cấp hai cổng đọc và một cổng ghi, cho phép truy xuất song song hai toán hạng nguồn và ghi kết quả vào một thanh ghi đích.

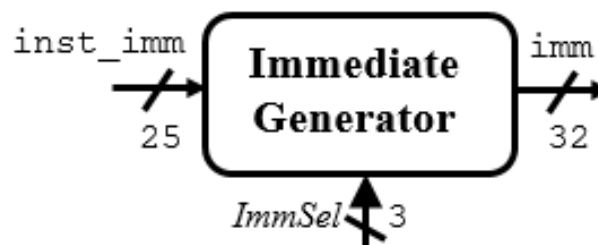
Bảng 3.4 mô tả các tín hiệu của Register File bao gồm các tín hiệu như là clk, rsW, rsR1, rsR2, dataW, dataR1, dataR2 và RegWEn.

Bảng 3.4: Mô tả các tín hiệu của Register File

Tín hiệu	Mô tả
clk	Tín hiệu clock cạnh lên
rsW	Địa chỉ thanh ghi đích cần ghi
rsR1	Địa chỉ thanh ghi nguồn 1
rsR2	Địa chỉ thanh ghi nguồn 2
dataW	Dữ liệu 32 bit cần ghi vào thanh ghi đích
dataR1	Dữ liệu đọc ra từ thanh ghi được chỉ định bởi rsR1
dataR2	Dữ liệu đọc ra từ thanh ghi được chỉ định bởi rsR2
RegWEn	Tín hiệu cho phép ghi, khi mức ‘1’ thì dữ liệu dataW được ghi vào thanh ghi rsW tại cạnh lên của xung clock

3.2.5. Immediate Generator

Hình 3.6 minh họa mô-đun Immediate Generator, nơi nhận mã lệnh 25 bit đã được giải mã và trích xuất các trường immediate theo từng định dạng lệnh để tạo ra giá trị hằng số 32 bit đưa vào các khối xử lý tiếp theo.



Hình 3.6: Immediate Generator

Immediate Generator là mô-đun chịu trách nhiệm phân tích trường mã lệnh và tạo ra giá trị hằng số tức thời được sử dụng bởi nhiều loại lệnh trong kiến trúc RV32IM. Mỗi định dạng lệnh của RISC-V tổ chức phần immediate theo một cách khác nhau. Vì vậy mô-đun này sẽ nhận mã lệnh đã được tách ra từ Decoder và sắp xếp lại các bit tương ứng để tạo giá trị immediate đã được mở rộng dấu lên 32 bit.

Ở đầu vào, Immediate Generator nhận tín hiệu `inst_imm` là mã lệnh đang được giải mã từ Decoder. Dựa trên opcode và định dạng của lệnh, mô-đun sẽ lựa chọn các bit cần thiết để ghép thành một giá trị hằng. Giá trị immediate đầu ra `imm` được truyền đến các khối khác trong datapath.

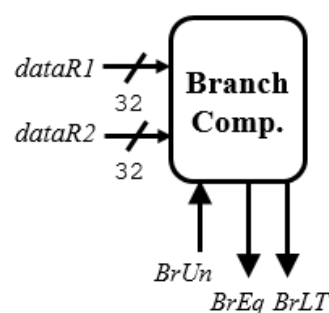
Bảng 3.5 mô tả các tín hiệu của Immediate Generator bao gồm các tín hiệu như là `instr`, `imm` và `ImmSel`.

Bảng 3.5: Mô tả các tín hiệu của Immediate Generator

Tín hiệu	Mô tả
<code>Instr_imm</code>	Mã lệnh 25 bit được lấy từ Decoder
<code>imm</code>	Giá trị immediate 32 bit đã được trích xuất và mở rộng dấu từ mã lệnh
<code>ImmSel</code>	Tín hiệu dùng để chọn kiểu immediate cần giải mã từ trường lệnh 32 bit

3.2.6. Branch Comparator

Hình 3.7 minh họa mô-đun Branch Comparator, nơi thực hiện so sánh hai toán hạng đọc từ Register File và tạo ra các tín hiệu điều kiện dùng cho các lệnh nhảy có điều kiện.



Hình 3.7: Branch Comparator

Branch Comparator thực hiện so sánh hai toán hạng đọc từ Register File, tín hiệu điều khiển `BrUn` quyết định so sánh có dấu hay không dấu. Có ba chế độ so sánh: bằng, nhỏ hơn có dấu và nhỏ hơn không dấu. Kết quả được xuất qua hai tín hiệu `BrEq` và `BrLT` để khối Branch Control quyết định rẽ nhánh hay tiếp tục tuần tự.

Bảng 3.6 mô tả các tín hiệu của Branch Comparator bao gồm các tín hiệu như là dataR1, dataR2, BrUn, BrEq và BrLT.

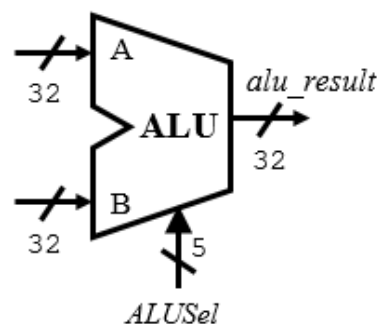
Bảng 3.6: Mô tả các tín hiệu của Branch Comparator

Tín hiệu	Mô tả
dataR1	Giá trị 32 bit của toán hạng thứ nhất được đọc từ Register File
dataR2	Giá trị 32 bit của toán hạng thứ hai được đọc từ Register File
BrUn	Tín hiệu điều khiển lựa chọn kiểu so sánh có dấu hoặc không dấu
BrEq	Tín hiệu kết quả cho biết hai toán hạng bằng nhau
BrLT	Tín hiệu kết quả cho biết toán hạng thứ nhất nhỏ hơn toán hạng thứ hai

Mặc dù chỉ có hai tín hiệu kết quả, BrEq và BrLT kết hợp với BrUn đủ để biểu diễn toàn bộ sáu lệnh nhánh của RISC-V. Các lệnh BGE/BGEU không cần mạch so sánh riêng mà được suy ra bằng cách đảo điều kiện BrLT ngay tại khối Branch Control.

3.2.7. Arithmetic Logic Unit (ALU)

Hình 3.8 minh họa mô-đun ALU, nơi thực hiện các phép toán số học và logic dựa trên hai toán hạng đầu vào và tín hiệu điều khiển ALUSel.



Hình 3.8: Arithmetic Logic Unit (ALU)

ALU xử lý các phép toán số học và logic trong datapath, nhận hai toán hạng A và B mỗi giá trị 32 bit cùng tín hiệu điều khiển ALUSel 5 bit để chọn phép toán tương ứng. Kết quả 32 bit trên ngõ ra alu_result được sử dụng cho các mục đích khác nhau tùy loại lệnh: tính địa chỉ bộ nhớ, ghi vào Register File hoặc tính địa chỉ cho logic rẽ nhánh.

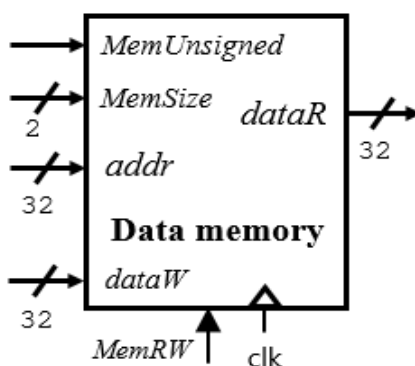
Bảng 3.7 mô tả các tín hiệu của ALU bao gồm các tín hiệu như là A, B, ALUSel và alu_result.

Bảng 3.7: Mô tả các tín hiệu của Arithmetic Logic Unit (ALU)

Tín hiệu	Mô tả
A	Giá trị 32 bit của toán hạng thứ nhất
B	Giá trị 32 bit của toán hạng thứ hai
ALUSel	Tín hiệu điều khiển lựa chọn kiểu tính toán
alu_result	Kết quả sau khi tính toán

3.2.8. Data Memory

Hình 3.9 minh họa mô-đun Data Memory, dùng để đọc và ghi dữ liệu dựa trên địa chỉ đầu vào và tín hiệu điều khiển MemRW.



Hình 3.9: Data Memory

Data Memory lưu trữ và truy xuất dữ liệu phục vụ các lệnh load/store. Địa chỉ 32 bit được cung cấp qua addr, dữ liệu 32 bit cần ghi qua dataW, chế độ đọc/ghi được chọn bởi MemRW. Tín hiệu MemSize xác định kích thước truy cập có thể là byte, half-word hoặc word và MemUnsigned quyết định cách mở rộng dữ liệu khi đọc là sign-extend hoặc zero-extend. Khi đọc, kết quả trả về qua dataR còn khi ghi, dữ liệu được cập nhật tại cạnh lên của xung clock.

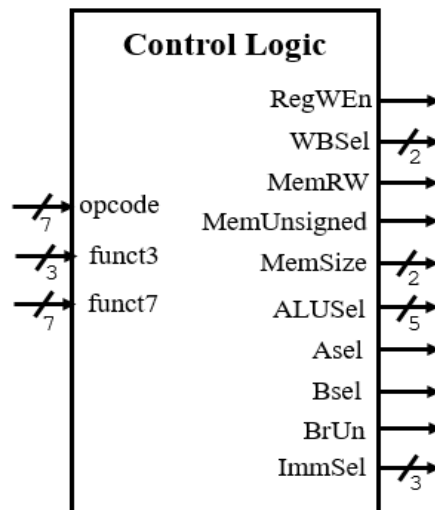
Bảng 3.8 mô tả các tín hiệu của Data Memory bao gồm các tín hiệu như là clk, addr, MemRW, Memsize, MemUnsigned, dataW và dataR.

Bảng 3.8: Mô tả các tín hiệu của Data Memory

Tín hiệu	Mô tả
clk	Tín hiệu clock cạnh lên
addr	Địa chỉ 32 bit xác định ô nhớ cần truy cập
MemRW	Tín hiệu điều khiển đọc/ghi của bộ nhớ
MemSize	Tín hiệu xác định kích thước dữ liệu truy cập bộ nhớ
MemUnsigned	Tín hiệu xác định kiểu mở rộng dữ liệu khi load
dataW	Dữ liệu 32 bit ghi vào bộ nhớ khi thực hiện lệnh Store
dataR	Dữ liệu 32 bit đọc ra từ bộ nhớ khi thực hiện lệnh Load

3.2.9. Control Logic

Hình 3.10 minh họa mô-đun Control Logic, thành phần chịu trách nhiệm sinh ra các tín hiệu điều khiển như RegWEn, BrUn, BSel, MemSize, MemUnsigned, ASel, ALUSel, MemRW, WBSel và ImmSel nhằm điều phối hoạt động của các khối trong datapath.



Hình 3.10: Control Logic

Control Logic giải mã các trường opcode 7 bit, funct3 3 bit và funct7 7 bit từ Decoder để sinh ra các tín hiệu điều khiển điều phối hoạt động của toàn bộ datapath. Bảng 3.9 mô tả chi tiết các tín hiệu của Control Logic.

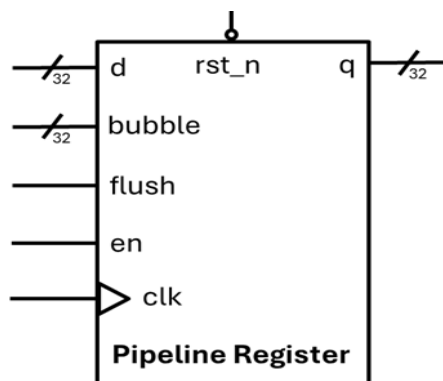
Bảng 3.9: Mô tả các tín hiệu của Control Logic

Tín hiệu	Mô tả
opcode	Trường 7 bit xác định loại lệnh
funct3	Trường 3 bit phân biệt biến thể lệnh
funct7	Trường 7 bit hỗ trợ xác định phép toán
BrUn	Chọn kiểu so sánh: có dấu hoặc không dấu
ImmSel	Chọn định dạng immediate (I/S/B/U/J/P)
RegWEn	Cho phép ghi kết quả vào Register File
ASel	Chọn toán hạng A của ALU: “0”- rs1 hoặc “1”- PC hiện tại
BSel	Chọn toán hạng B của ALU: “0”- rs2 hoặc “1”- immediate
ALUSel	Xác định phép toán ALU cần thực thi
MemRW	Chọn chế độ truy cập Data Memory: đọc hoặc ghi
MemSize	Kích thước dữ liệu truy cập: byte, half-word hoặc word
MemUnsigned	Kiểu mở rộng khi load: sign-extend hoặc zero-extend
WBSel	Chọn nguồn ghi về Register File: ‘0’- dữ liệu data memory, ‘1’- kết quả ALU, hoặc ‘2’- PC+4

3.3.10. Pipeline Register

Pipeline Register lưu trữ và đồng bộ dữ liệu giữa các giai đoạn pipeline, đảm bảo các lệnh được tách biệt và xử lý song song an toàn.

Hình 3.11 minh họa cấu trúc cơ bản của một Pipeline Register cùng các tín hiệu điều khiển liên quan.



Hình 3.11: Pipeline Register

Mô-đun được điều khiển bởi ba tín hiệu:

- rst_n: reset bất đồng bộ về trạng thái bubble.
- flush: được tạo bởi Hazard Unit để xoá lệnh sai và nạp bubble.
- en (enable): cho phép cập nhật dữ liệu, khi en = 0 pipeline giữ nguyên giá trị cũ để thực hiện stall.

Bảng 3.10 trình bày trạng thái đầu ra của Pipeline Register tương ứng với các tổ hợp tín hiệu điều khiển khác nhau.

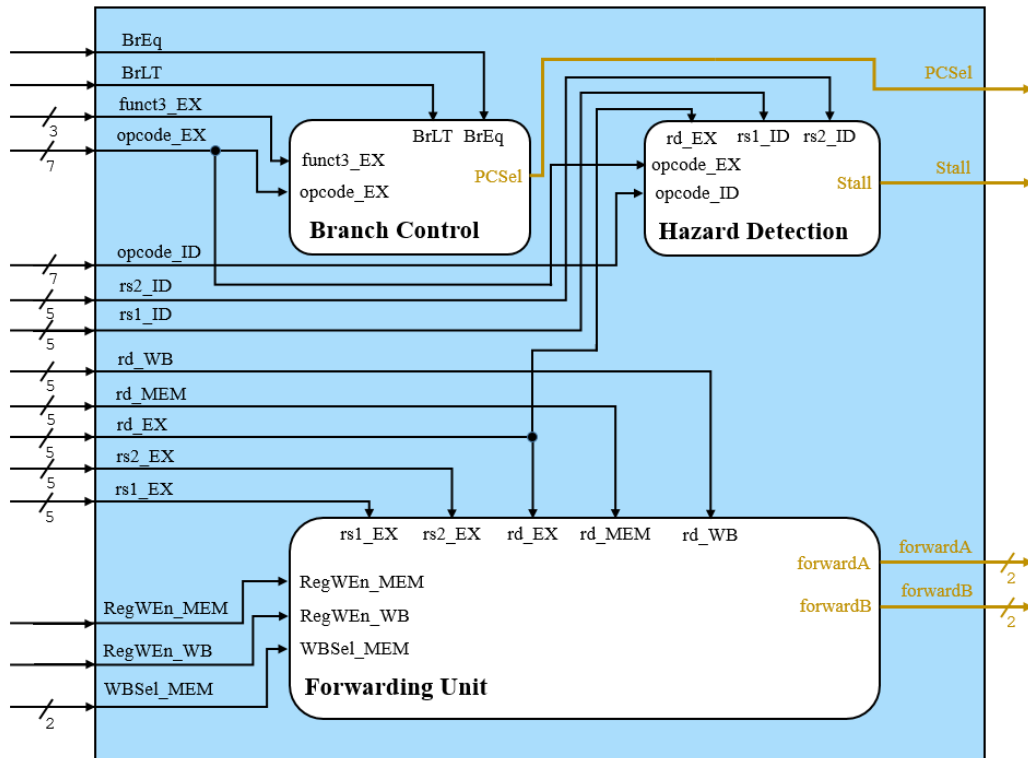
Bảng 3.10: Bảng trạng thái Pipeline Register

Ngõ vào			Ngõ ra	Trạng thái
rst_n	flush	en	q	
0	X	X	bubble	Reset
1	1	X	bubble	Flush
1	0	1	d	Enable (update)
1	0	0	q	stall

Ngõ vào bubble ở pipeline register dùng để đưa vào một lệnh “rỗng” hay còn gọi là NOP (32'h00000013) nhằm stall.

3.3.11. Hazard Unit

Hình 3.12 minh họa cấu trúc mô-đun Hazard Unit, thành phần chịu trách nhiệm phát hiện và xử lý các xung đột dữ liệu, xung đột điều khiển và tình huống load-use trong kiến trúc pipeline.

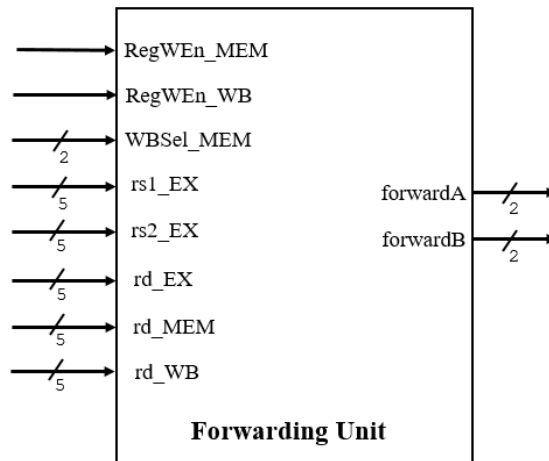


Hình 3.12: Hazard Unit

Hazard Unit chịu trách nhiệm phát hiện và xử lý các xung đột trong pipeline, bao gồm data hazard, control hazard và tình huống load-use không thể giải quyết bằng forwarding. Tùy loại xung đột, mô-đun kích hoạt các cơ chế tương ứng: stall để dừng pipeline, flush để xóa lệnh sai, hoặc forwarding để chuyển tiếp dữ liệu sớm. Hazard Unit được cấu thành từ ba khối con: Branch Control xử lý rẽ nhánh, Hazard Detection phát hiện xung đột load-use, và Forwarding Unit giải quyết xung đột dữ liệu.

❖ *Forwarding Unit*

Trong pipeline, xung đột dữ liệu RAW xảy ra khi lệnh ở giai đoạn EX cần kết quả chưa kịp ghi vào Register File từ các giai đoạn MEM hoặc WB. Forwarding Unit giải quyết bằng cách chuyển trực tiếp dữ liệu từ Register Pipeline EX/MEM hoặc MEM/WB về đầu vào ALU, loại bỏ phần lớn stall không cần thiết ngoại trừ tình huống load-use. Hình 3.13 minh họa cấu trúc và các tín hiệu của Forwarding Unit.



Hình 3.13: Forwarding Unit

Forwarding Unit so sánh thanh ghi nguồn rs1, rs2 của lệnh tại EX với thanh ghi đích rd và tín hiệu điều khiển của các lệnh tại MEM và WB, từ đó sinh ra forwardA và forwardB để chọn nguồn toán hạng cho ALU. Bảng 3.11 trình bày mã hóa của hai tín hiệu này.

Bảng 3.11: Mã hóa tín hiệu Forwarding (forwardA, forwardB)

Mã	Nguồn dữ liệu	Ý nghĩa
2'b10	EX/MEM	Kết quả ALU của lệnh ngay trước
2'b01	MEM/WB	Giá trị chuẩn bị write back trở về register
2'b00	ID/EX	Không forward, lấy từ Register File

Forwarding từ EX/MEM chỉ được thực hiện khi lệnh tại MEM ghi kết quả ALU về thanh ghi. Cơ chế forwarding từ giai đoạn EX/MEM và MEM/WB được điều khiển bởi các tín hiệu logic nhằm xác định nguồn dữ liệu phù hợp cho ALU.

Bảng 3.12, 3.13 và 3.14 mô tả các điều kiện cho phép forwarding và trạng thái điều khiển các tín hiệu forwardA và forwardB.

Bảng 3.12: Điều kiện allow_exmem_fwd

Ngõ vào		Ngõ ra
RegWEn_MEM	WBSel_MEM	allow_exmem_fwd
0	WB_ALU	0
0	≠ WB_ALU	0
1	≠ WB_ALU	0
1	WB_ALU	1

Bảng 3.13: Trạng thái điều khiển forwardA

Ngõ vào				Ngõ ra
allow_exmem_fwd	rd_MEM	RegWEn_WB	rd_WB	forwardA
1	rs1_EX	X	x	10
X	X	1	rs1_EX	01
X	X	X	X	00

Bảng 3.14: Trạng thái điều khiển forwardB

Ngõ vào				Ngõ ra
allow_exmem_fwd	rd_MEM	RegWEn_WB	rd_WB	forwardB
1	rs2_EX	X	x	10
X	X	1	rs2_EX	01
X	X	X	X	00

Trường hợp 1 - Forward từ EX/MEM (2'b10): Xảy ra khi lệnh liền trước có kết quả tính toán từ ALU cần sử dụng ngay. Dữ liệu được chuyển tiếp trực tiếp từ Register Pipeline EX/MEM.

Điều kiện: $\text{RegWEn_MEM} = 1$, $\text{WBSel_MEM} = \text{WB_ALU}$ (kết quả từ ALU), $\text{rd_MEM} \neq x0$ và trùng địa chỉ thanh ghi ($\text{rd_MEM} == \text{rs_EX}$).

Trường hợp 2 - Forward từ MEM/WB (2'b01): Xảy ra khi dữ liệu cần thiết thuộc về lệnh cách đó 2 chu kỳ tức đang ở giai đoạn WB. Dữ liệu hoàn thiện được lấy từ Register Pipeline MEM/WB.

Điều kiện: $\text{RegWEn_WB} = 1$, $\text{rd_WB} \neq x0$ và trùng địa chỉ thanh ghi ($\text{rd_WB} == \text{rs_EX}$). Lưu ý: Chỉ kích hoạt khi không xảy ra trường hợp 1.

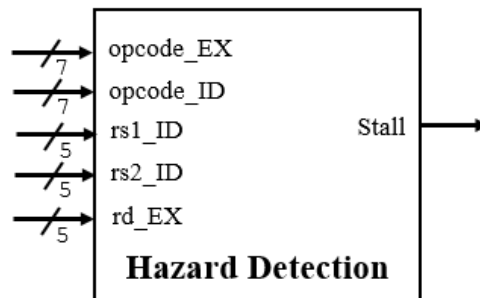
Trường hợp 3 – Mặc định (2'b00): Không có phụ thuộc dữ liệu hoặc thanh ghi đích là x0. ALU nhận toán hạng trực tiếp từ Register File.

❖ *Hazard Detection*

Hoạt động tại giai đoạn ID, Hazard Detection Unit có nhiệm vụ phát hiện xung đột dữ liệu load-use - trường hợp mà kỹ thuật forwarding không thể xử lý. Khi lệnh đang ở giai đoạn ID cần sử dụng dữ liệu được nạp từ bộ nhớ bởi lệnh

LOAD ngay trước đó tức là ở giai đoạn EX, pipeline sẽ phải tạm dừng để chờ dữ liệu sẵn sàng.

Hình 3.14 minh họa khối Hazard Detection cùng các tín hiệu đầu vào phục vụ quá trình phát hiện xung đột.



Hình 3.14: Hazard Detection

Tín hiệu stall chỉ được kích hoạt khi đồng thời thỏa mãn các điều kiện sau:

- Lệnh tại giai đoạn EX là lệnh LOAD (opcode_EX = OC_I_LOAD).
- Thanh ghi đích của lệnh LOAD hợp lệ (rd_EX $\neq$ x0).
- Lệnh tại giai đoạn ID có sử dụng ít nhất một thanh ghi nguồn (rs1 hoặc rs2) và địa chỉ thanh ghi này trùng với rd_EX.

Việc xác định lệnh tại giai đoạn ID có sử dụng các thanh ghi nguồn hay không phụ thuộc vào opcode của lệnh. Bảng 3.15 trình bày kết quả phân loại các lệnh theo nhu cầu sử dụng rs1 và rs2, giúp tránh phát hiện sai phụ thuộc dữ liệu.

Bảng 3.15: Xác định nhu cầu sử dụng thanh ghi nguồn của lệnh ở ID

Ngõ vào	Ngõ ra	
	use_rs1_ID	use_rs2_ID
OC_R (R-type)	1	1
OC_S (STORE)	1	1
OC_B (BRANCH)	1	1
OC_I_ALU	1	0
OC_I_LOAD (LOAD)	1	0
OC_I_JALR (JALR)	1	0
OC_U_LUI (LUI)	0	0
OC_U_AUIPC (AUIPC)	0	0
OC_J (JAL)	0	0

Dựa trên loại lệnh tại EX và nhu cầu sử dụng thanh ghi của lệnh tại ID, Hazard Detection Unit quyết định có chèn stall hay không.

Bảng 3.16 mô tả quan hệ vào - ra của khối Hazard Detection, cho thấy stall chỉ được kích hoạt trong trường hợp load-use thực sự xảy ra.

Bảng 3.16: Quan hệ ngõ vào – ngõ ra của Hazard Detection

Ngõ vào				Ngõ ra
opcode_EX	rd_EX	use_rs1_ID	use_rs2_ID	stall
OC_I_LOAD	X	0	0	0
OC_I_LOAD	rs2_ID	0	1	1
OC_I_LOAD	rs1_ID	1	0	1
OC_I_LOAD	rs1_ID rs2_ID	1	1	1
≠ OC_I_LOAD	X	X	X	0

Khi EX là LOAD: LOAD có dữ liệu sẵn ở cuối giai đoạn MEM, nhưng lệnh kế tiếp ở ID sẽ cần giá trị đó ngay ở đầu EX (chu kỳ kế tiếp). Forwarding không thể cung cấp dữ liệu vì giá trị từ LOAD chưa được tạo ra ở EX/MEM và giá trị chỉ xuất hiện ở MEM/WB → Lệnh sau không thể nhận dữ liệu kịp → phải stall.

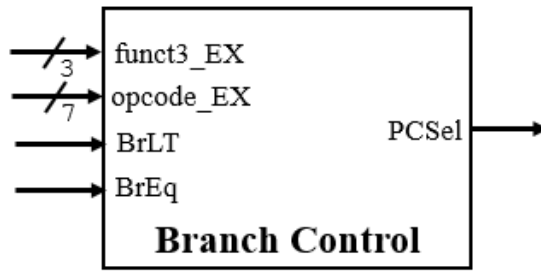
Khi EX không phải LOAD: R-type, I-type ALU, STORE, BRANCH đều có kết quả tại EX hoặc trong EX/MEM. Forwarding cung cấp dữ liệu kịp thời → Không cần stall.

Khi stall = 1, pipeline sẽ làm cho PC giữ nguyên, IF/ID giữ nguyên lệnh cũ, ID/EX được chèn bubble để xóa tín hiệu điều khiển → LOAD có thêm 1 chu kỳ để hoàn tất truy cập bộ nhớ trước khi lệnh sau dùng dữ liệu.

❖ **Branch Control**

Hoạt động tại giai đoạn EX, Branch Control Unit chịu trách nhiệm điều khiển luồng thực thi của chương trình thông qua tín hiệu PCSel. Dựa trên việc giải mã lệnh (opcode, funct3) kết hợp với kết quả so sánh từ Branch Comparator (BrEq, BrLT), khối này quyết định cập nhật PC theo luồng tuần tự ($PC + 4$) hay chuyển sang địa chỉ đích của lệnh nhánh.

Hình 3.15 minh họa cấu trúc và các tín hiệu vào/ra của Branch Control.



Hình 3.15: Branch Control

Dựa trên loại lệnh, Branch Control được chia thành hai nhóm chính:

- Lệnh nhảy không điều kiện (JAL, JALR): luôn yêu cầu chuyển hướng luồng thực thi, do đó PCSel = 1.
- Lệnh nhánh có điều kiện (B-type): quyết định nhảy hay không phụ thuộc vào funct3 và kết quả so sánh BrEq, BrLT từ Branch Comparator.

Quan hệ giữa các tín hiệu đầu vào và quyết định điều khiển PC được thể hiện trong bảng 3.17, cho thấy PCSel chỉ được kích hoạt khi lệnh yêu cầu thay đổi luồng thực thi hoặc điều kiện nhánh được thỏa mãn.

Bảng 3.17: Bảng quan hệ ngõ vào – ngõ ra của Branch Control

Ngõ vào				Ngõ ra
opcode_EX	funct3_EX	BrEq	BrLT	PCSel
OC_J (JAL)	X	X	X	1
OC_I_JALR	X	X	X	1
OC_B	F3_BEQ	1	X	1
OC_B	F3_BNE	0	X	1
OC_B	F3_BLT	X	1	1
OC_B	F3_BLTU	X	1	1
OC_B	F3_BGE	1	X	1
		X	0	1
OC_B	F3_BGEU	1	X	1
		X	0	1
X	X	X	X	0

Đối với lệnh nhảy không điều kiện (JAL, JALR), Branch Control không xét các cờ so sánh và luôn đặt PCSel = 1, đảm bảo PC được cập nhật sang địa chỉ đích theo đặc tả RV32IM.

Với lệnh nhánh có điều kiện (BEQ, BNE, BLT, BGE, BLTU, BGEU), Branch Control xác định quyết định nhảy dựa trên funct3_EX và các cờ so sánh BrEq và BrLT từ Branch Comparator:

- BEQ: Nhảy khi BrEq = 1 (hai toán hạng bằng nhau).
- BNE: Nhảy khi BrEq = 0 (hai toán hạng khác nhau).
- BLT/BLTU: Nhảy khi BrLT = 1 ($rs1 < rs2$). Khối Branch Comparator đã tính sẵn BrLT theo signed hoặc unsigned tùy funct3.
- BGE/BGEU: Nhảy khi $rs1 \geq rs2$.
 - Thực hiện bằng biểu thức logic: $(\sim BrLT) \mid BrEq$.
 - Nếu rs1 không nhỏ hơn hoặc bằng nhau thì điều kiện nhảy thỏa.

Khi PCSel = 1 thì giá trị PC sẽ nhảy đến địa chỉ mới do đó các lệnh đã được nạp đang ở IF và ID sẽ bị xóa bằng lệnh “NOP” để đảm bảo chính xác hoạt động của pipeline. Đối với các opcode còn lại, Branch Control giữ PCSel = 0 để pipeline tiếp tục thực thi tuần tự.

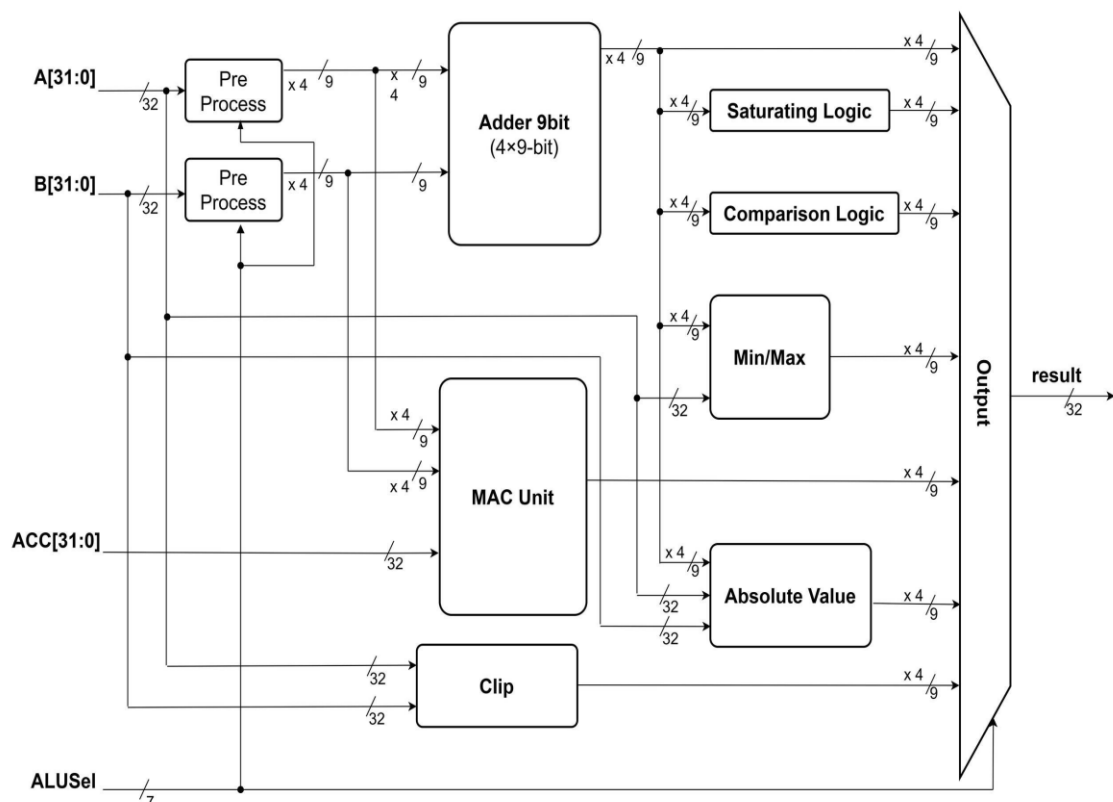
3.3. THIẾT KẾ KHỐI THỰC THI SIMD PEXT

3.3.1. Cấu trúc tổng thể của khối SIMD Pext

Mục tiêu lớn nhất khi thiết kế khối SIMD Pext là tăng khả năng tính toán nhưng tối thiểu hóa việc sử dụng tài nguyên, đặc biệt là logic cell trên FPGA. Để đạt được điều đó, thiết kế áp dụng mạnh mẽ triết lý "Tái sử dụng phần cứng".

Thay vì xây dựng nhiều bộ cộng riêng rẽ cho từng nhóm lệnh cộng/trừ, lệnh so sánh, hay lệnh trị tuyệt đối, SIMD Pext gom toàn bộ chúng chạy qua một cụm tính toán duy nhất: Shared Adder 4x9 bit. Thiết kế này tiết kiệm đáng kể tài nguyên khi tổng hợp so với các phương pháp cài đặt rời rạc thông thường. Khối SIMD Pext được chia làm ba cụm xử lý độc lập về logic bao gồm: Shared Adder, khối nhân cộng dồn MAC Unit và logic giới hạn giá trị Clip.

Hình 3.16 mô tả sơ đồ tổng quát của SIMD Pext với các tín hiệu đầu vào A, B và ACC còn tín hiệu đầu ra là kết quả tính toán của P-extension.



Hình 3.16: Sơ đồ tổng quan khối SIMD Pext [11]

Một thanh ghi 32 bit được chia thành bốn lane 8 bit hoặc hai lane 16 bit. Các lane byte được mở rộng thành 9 bit trước khi đưa vào bộ cộng để giữ thông tin dấu/carry trung gian. Với phép toán halfword, hai lane 9 bit liền kề được ghép bằng carry để tạo thành phép cộng/trừ 16 bit. Cách tái sử dụng bộ cộng 4x9 bit này phù hợp với hướng thiết kế SIMD Pext.

Bảng 3.18 liệt kê danh sách các nhóm lệnh P-extension được hỗ trợ trên thiết kế:

Bảng 3.18: Nhóm lệnh P-extension được hỗ trợ trong SIMD Pext

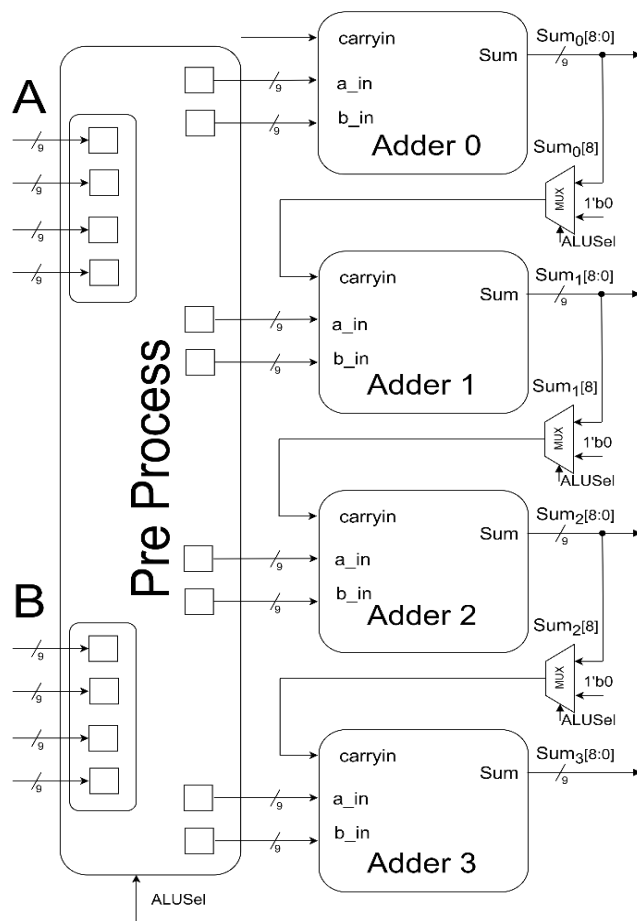
Nhóm lệnh	Lệnh tiêu biểu	Chức năng
Cộng/trừ packed-SIMD	PADD.B, PSUB.B, PADD.H, PSUB.H,...	Cộng/trừ độc lập theo byte hoặc halfword
Trung bình và bão hòa	PAADD.B, PASUB.H, PSADD.H, PSSUB.B,...	Tính trung bình hoặc giới hạn kết quả khi tràn
Xử lý chéo halfword	PAS.HX, PSA.HX, PASA.HX, PSSA.HX,...	Cộng/trừ giữa các halfword ở vị trí chéo
So sánh và min/max	PMSEQ.H, PMSLT.H, PMIN.H, PMAXU.H,...	So sánh, chọn phần tử nhỏ nhất/lớn nhất
Trị tuyệt đối và clip	PSABS.H, PSATL.H, PUSATL.H	Lấy trị tuyệt đối và giới hạn miền giá trị
Nhân cộng dồn	PM4ADDA.B, PM4ADDASU.B, PM4ADDAU.B	Nhân bốn cặp byte và cộng dồn vào rd

Để đảm bảo tính chính xác khi triển khai tập lệnh P-extension, đề tài đã tham khảo mô hình tham chiếu P-extension được viết bằng Chisel từ nguồn [12], sau đó sẽ thiết kế lại bằng ngôn ngữ SystemVerilog nhằm tích hợp vào thiết kế tổng thể.

3.3.2. Bộ cộng SIMD 4x9 bit

Bộ cộng SIMD là thành phần trung tâm của khối SIMD Pext. Nó bao gồm bốn bộ cộng 9 bit hoạt động song song. Việc mở rộng thiết kế lên 9 bit đóng vai trò tạo ra một bit dự phòng để lưu giữ cờ nhớ carry-out và cờ dấu, đảm bảo không thất thoát dữ liệu trung gian. Khi chạy ở chế độ byte mode, bốn lane hoàn toàn ngắt kết nối với nhau carry in của cả bốn khối bằng không để tính toán song song. Ở chế độ halfword mode, carry-out của lane 0 được nối vào lane 1, carry-out của

lane 2 được nối vào lane 3, carry lane 2 bằng không tạo thành hai bộ cộng 16 bit độc lập như được trình bày trong sơ đồ khối hình 3.17.



Hình 3.17: Cấu trúc bộ cộng SIMD 4x9 bit cho chế độ byte và halfword [11]

Thay vì thiết kế các khối phần cứng rời rạc, lõi bộ cộng này được tái sử dụng hardware reuse cho toàn bộ các nhóm lệnh sau.

❖ **Nhóm phép Cộng/Trừ (ADD/SUB)**

Cả hai phép toán đều tái sử dụng bộ cộng 9 bit. Phép cộng đặt carry-in = 0 và truyền thẳng toán hạng A, B vào bộ cộng. Phép trừ thực hiện theo bù 2 bằng cách đảo toàn bộ bit của toán hạng B (kể cả bit dấu) và đặt carry-in = 1 cho tất cả lane, không cần mạch trừ riêng. Kết quả lấy 8 bit thấp Sum[7:0] của mỗi lane.

❖ **Nhóm tính Bão hòa (Saturation - PSADD, PSSUB, PSADDU, PSSUBU)**

Sau khi bộ cộng 9 bit tính xong, phần cứng kiểm tra đồng thời Sum[8] và Sum[7] để phát hiện tràn số học. Bảng 3.19 mô tả logic bão hòa cho lệnh có dấu.

Bảng 3.19: Phát hiện tràn số có dấu (Signed - PSADD, PSSUB)

Sum[8]	Sum[7]	Trường hợp	Kết quả
0	0	Không tràn, kết quả dương	Sum[7:0]
0	1	Tràn dương	+127
1	0	Tràn âm	-128
1	1	Không tràn, kết quả âm	Sum[7:0]

Với lệnh không dấu, chỉ cần kiểm tra Sum[8] nếu bằng 1 thì tràn, kết quả được bão hòa về giới hạn tương ứng. Bảng 3.20 dưới mô tả logic bão hòa cho lệnh không dấu.

Bảng 3.20: Phát hiện tràn số không dấu (Unsigned - PSADDU, PSSUBU)

Sum[8]	Trường hợp	Kết quả cộng (PSADDU)	Kết quả trừ (PSSUBU)
0	Không tràn	Sum[7:0]	Sum[7:0]
1	Tràn	255	0

❖ *Nhóm tính Trung bình (Averaging - PAADD, PASUB)*

Toán hạng A và B được mở rộng dấu lên 9 bit trước khi đưa vào bộ cộng, đảm bảo carry-out không bị mất. Kết quả 9 bit sau đó được dịch phải 1 bit để thu được trung bình cộng, loại bỏ hoàn toàn nhu cầu dùng bộ chia.

❖ *Nhóm tính toán Chéo (Cross - PAS.HX, PSA.HX)*

Bộ cộng 9 bit sử dụng cách ghép chéo dữ liệu đầu vào theo hai cặp là A[15:0] với B[31:16] và A[31:16] với B[15:0]. Khi đó, một lane thực hiện phép cộng, lane còn lại thực hiện phép trừ, như thể hiện trong Bảng 3.21.

Bảng 3.21: Cấu hình trao đổi dữ liệu cho nhóm Cross

Lệnh	Lane thấp rd[15:0]	Lane cao rd[31:16]
PAS.HX	A[15:0] – B[31:16]	A[31:16] + B[15:0]
PSA.HX	A[15:0] + B[31:16]	A[31:16] – B[15:0]

❖ **Nhóm lệnh So sánh (PMSEQ, PMSLT, PMSLTU)**

Thay vì dùng khối so sánh riêng, phần cứng tái sử dụng bộ cộng để tính A - B theo bù 2, sau đó trích xuất cờ trạng thái từ kết quả.

So sánh bằng (PMSEQ): Dựa trên $A = B \Leftrightarrow A - B = 0$. Nếu tất cả bit bằng 0 thì xuất 0xFFFF (True), ngược lại xuất 0x0000 (False).

So sánh có dấu (PMSLT, PMIN, PMAX): Quan hệ A nhỏ hơn B được xác định theo công thức:

$$Sum_{MSB} \oplus ((A_{MSB} \oplus B_{MSB}) \& (A_{MSB} \oplus Sum_{MSB})) \quad (3.1)$$

- Khi A và B cùng dấu $A_{MSB} \oplus B_{MSB} = 0$: phép trừ không gây ra tràn bit. Hàm rút gọn về $Sum_{MSB} \oplus 0$. Do đó $Sum_{MSB} = 1 \rightarrow A < B$
- Khi A và B khác dấu $A_{MSB} \oplus B_{MSB} = 1$. Phép trừ có thể gây ra tràn bit. Hàm logic rút gọn về $Sum_{MSB} \oplus (A_{MSB} \oplus Sum_{MSB})$:
 - A âm, B dương: $Sum_{MSB} \oplus (A_{MSB} \oplus Sum_{MSB}) = 1 \rightarrow A < B$.
 - A dương, B âm: $Sum_{MSB} \oplus (A_{MSB} \oplus Sum_{MSB}) = 0 \rightarrow A > B$.

So sánh không dấu (PMSLTU, PMINU, PMAXU): Quan hệ A nhỏ hơn B được xác định theo công thức:

$$(A_{MSB} = 0) \& (B_{MSB} = 1) \mid ((A_{MSB} = B_{MSB}) \& (Sum_{MSB} = 1)) \quad (3.2)$$

- Về đầu xét trực tiếp MSB nếu $A_{MSB}=0$ và $B_{MSB}=1$ thì chắc chắn A nhỏ hơn B.
- Nếu A_{MSB} và B_{MSB} cùng dấu. Kết quả phép trừ không gây ra tràn bit. $Sum_{MSB} = 1 \rightarrow A < B$.

Kết quả của từng lệnh được mô tả chi tiết trong bảng 3.22 là kết quả khi A nhỏ hơn B.

Bảng 3.22: Kết quả khi A nhỏ hơn B

Lệnh	Điều kiện	Kết quả
PMSLT	$A < B$	Đúng xuất 0xFFFF, sai xuất 0x0000
PMSLTU	$A < B$	Đúng xuất 0xFFFF, sai xuất 0x0000
PMIN / PMINU	$A < B$	Đúng chọn A, sai chọn B
PMAX / PMAXU	$A < B$	Đúng chọn B, sai chọn A

❖ **Nhóm lệnh Trị tuyệt đối (PSABS)**

Phần cứng kiểm tra bit dấu của từng halfword. Nếu dataR1 dương, dữ liệu truyền thẳng ra đầu ra. Nếu dataR1 âm, bộ cộng tính $0 - \text{dataR1}$ bằng cách đặt $A = 0$ và $B = \sim \text{dataR1}$ với carry-in = 1 theo chế độ halfword. Trường hợp đặc biệt $\text{dataR1} = -32768$ tức là 0x8000 thì được bão hòa về +32767 do không có giá trị đối xứng trong 16 bit. Kết quả khi thỏa mãn điều kiện của lệnh được trình bày trong bảng 3.23.

Bảng 3.23: Logic xử lý PSABS theo từng trường hợp

Điều kiện	Hành động	Kết quả
$\text{dataR1}[15] = 0$ (A dương)	Truyền thẳng	$A[15:0]$
$\text{dataR1}[15] = 1$ và $A \neq 0x8000$	Tính $0 - \text{dataR1}$ qua bộ cộng	Sum[15:0]
$\text{dataR1}[15:0] = 0x8000$ (-32768)	Bão hòa	+32767 (0x7FFF)

Nhờ triết lý thiết kế "Data-path dùng chung" xung quanh lõi bộ cộng 4x9 bit, kiến trúc đã loại bỏ hoàn toàn các khối so sánh hay tính trung bình dư thừa. Điều này giúp tối giản tối đa diện tích silicon và tài nguyên logic, đồng thời vẫn duy trì được thông lượng xử lý song song cao trong một chu kỳ xung nhịp.

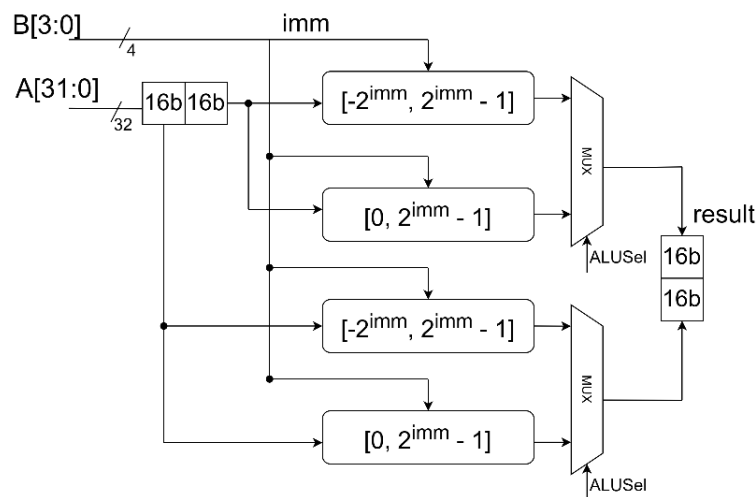
3.3.3. Nhóm lệnh giới hạn dữ liệu

Trong thiết kế, lệnh Clip hay còn gọi là lệnh giới hạn dữ liệu được thiết kế nhằm giới hạn giá trị của phần tử dữ liệu về một miền xác định, với biên độ linh hoạt được lập trình thông qua trường imm của tập lệnh. Nhằm hỗ trợ đa dạng kiểu dữ liệu, phần cứng cung cấp hai chế độ bão hòa là có dấu giới hạn từ $[-2^{imm}, 2^{imm} - 1]$ và không dấu giới hạn $[0, 2^{imm} - 1]$.

Dữ liệu đầu vào Rs1 được tách thành các phần tử độc lập và đối chiếu song song với ngưỡng của imm. Về mặt thiết kế RTL, nhằm tối ưu hóa tài nguyên phần cứng và giảm thiểu độ trễ, khối Clip không sử dụng các mạch so sánh truyền thống. Thay vào đó, thiết kế áp dụng kỹ thuật mặt nạ bit. Cụ thể, phần tử đầu vào sẽ được AND với giá trị ngưỡng được tạo ra từ imm để xem có vượt ngưỡng hay không:

- Nếu phát hiện tràn dương, kết quả bị xén về ngưỡng trần $2^{imm} - 1$
- Nếu phát hiện tràn âm, kết quả bị xén về ngưỡng sàn -2^{imm} đối với có dấu và 0 đối với không dấu.
- Nếu không tràn, giá trị gốc được giữ nguyên.

Hình 3.18 mô tả sơ đồ luồng dữ liệu của lệnh Clip trên hai phần tử 16-bit. Dữ liệu được cấp song song vào cả hai khối xử lý bão hòa có dấu và không dấu. Tại tầng cuối, tín hiệu điều khiển ALUSel sẽ điều khiển bộ ghép kênh để chọn ra kết quả xử lý chính xác theo yêu cầu của lệnh, trước khi ghi ngược lại vào thanh ghi đích.



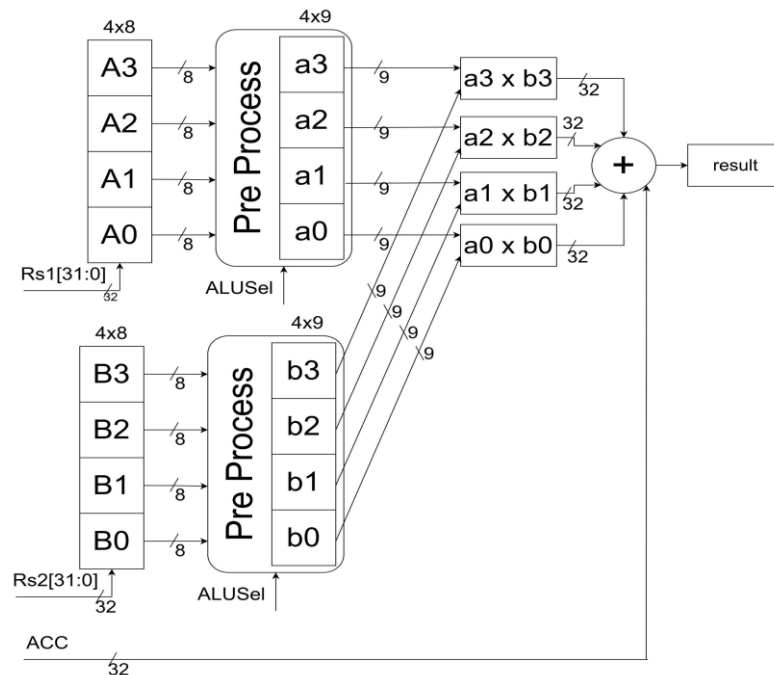
Hình 3.18: Sơ đồ thiết kế khối giới hạn dữ liệu Clip

3.3.4. Nhóm lệnh nhân cộng dồn

Nhóm lệnh nhân cộng dồn hay còn gọi là MAC trong tập lệnh P-extension được thiết kế dựa trên cơ chế xử lý song song dữ liệu mức phần tử. Quá trình thực thi lệnh được chia thành các giai đoạn như sau:

- Đầu tiên, hai thanh ghi Rs1 và Rs2 được đọc từ Register File rồi tách thành bốn cặp dữ liệu 8 bit. Các phần tử này được mở rộng lên 9 bit theo kiểu sign-extend hoặc zero-extend tùy loại lệnh.
- Giai đoạn nhân song song: Bốn cặp phần tử 9 bit này được cấp song song vào bốn bộ nhân, tạo ra bốn kết quả tích số trung gian đồng thời.
- Cuối cùng, các tích trung gian được cộng lại và tiếp tục cộng dồn với giá trị ACC, tương ứng với giá trị tích lũy hiện tại của lệnh MAC. Kết quả 32 bit cuối cùng sau đó được ghi trở lại thanh ghi đích trong Register File.

Nhờ kiến trúc phần cứng chuyên dụng này, hệ thống có thể thực hiện phép nhân của bốn cặp dữ liệu và cộng dồn chỉ trong một chu kỳ lệnh duy nhất, thay vì phải sử dụng một chuỗi nhiều lệnh nhân và cộng rời rạc. Hình 3.19 minh họa sơ đồ thiết kế của khối MAC trong kiến trúc P-extension.



Hình 3.19: Sơ đồ thiết kế khối nhân cộng dồn MAC trong P-extension

3.4. TÍCH HỢP SIMD PEXT VÀO PIPELINE RV32IM

Để tích hợp khối SIMD Pext vào pipeline RV32IM, thiết kế cần mở rộng một số phân hệ chính bao gồm tầng giải mã, tầng thực thi, Register File, Forwarding Unit và Hazard Detection. Các điểm mở rộng chính được tổng hợp trong Bảng 3.24.

Bảng 3.24: Các điểm mở rộng khi tích hợp SIMD Pext vào pipeline RV32IM

Phân hệ	Mở rộng trong RV32IM	Mục đích
Decoder/Control Logic	Nhận diện các opcode của P-extension tạo các tín hiệu ALUSel và rdSel	Giải mã và điều khiển lệnh P-extension
Tầng EX	Đặt SIMD Pext song song với ALU	Thực thi SIMD trong cùng chu kỳ EX
Register File	Bổ sung cổng đọc thứ 3 (rsR3)	Thực hiện cung cấp giá trị accumulator cũ cho MAC
Forwarding Unit	Bổ sung thêm đường bypass forwardACC.	Chuyển tiếp giá trị tích lũy liên tiếp
Hazard Detection	Mở rộng giám sát toán hạng thứ 3 (rd)	Đảm bảo stall an toàn với RAW Hazard

3.4.1. Mở rộng tầng giải mã

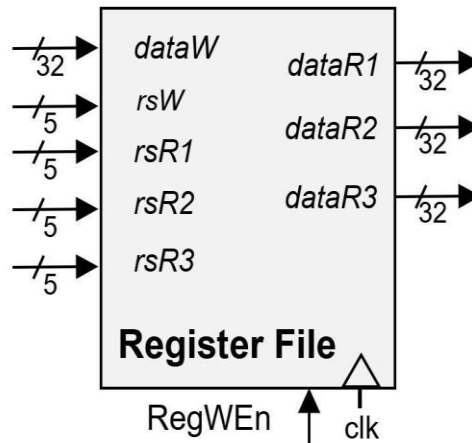
Ở tầng ID, Control Logic được cấu hình lại để nhận diện mã lệnh thuộc hai opcode đặc biệt là OC_PEXT (Register-to-Register) và OC_PEXT_IMM (Immediate). Khi xác định được lệnh thuộc P-extension, bộ điều khiển kích hoạt các tác vụ:

- Giải mã trường funct3 và funct7 để ấn định tín hiệu ALUSel cho P-extension.
- Nâng cờ bật bộ đa hợp bằng rdSel = ALUP_RESULT, ra lệnh cho toàn hệ thống ghi nhận kết quả đầu ra của SIMD Pext tại tầng EX.
- Nếu thuộc nhóm lệnh nhân cộng dồn, tín hiệu nhận diện is_mac được nâng lên mức 1.

3.4.2. Mở rộng Register File

Thách thức khó khăn nhất của việc tích hợp lệnh MAC vào pipeline chuẩn của RISC-V là lệnh MAC yêu cầu thao tác đọc cùng lúc 3 toán hạng: rs1, rs2 và thanh ghi đích rd đóng vai trò Accumulator.

Trong khi đó, Register File truyền thống chỉ cung cấp 2 cổng đọc. Giải pháp hiện thực RTL là chèn thêm một cổng đọc thứ ba là rsR3 vào Register File. Thiết kế đã nối giá trị dataR3 này qua Register Pipeline ID/EX, trực tiếp trở thành ngõ vào ACC của SIMD Pext tại tầng EX. Hình 3.20 mô tả Register File được mở rộng thêm cổng đọc rsR3, phục vụ việc đọc giá trị rd làm toán hạng tích lũy cho lệnh MAC.



Hình 3.20: Register File sau khi bổ sung cổng đọc rsR3

3.4.3. Xử lý xung đột dữ liệu

❖ Forwarding Unit

Đối với P-extension, các lệnh nhân cộng dồn MAC yêu cầu đọc giá trị từ thanh ghi đích rd để làm toán hạng thứ ba. Điều này sinh ra một loại RAW Hazard mới ví dụ hai lệnh MAC thực thi liên tiếp trên cùng một rd, trong khi hệ thống Forwarding cơ bản chỉ hỗ trợ kiểm tra và chuyển tiếp cho rs1 và rs2.

Để giải quyết, kiến trúc RV32IMP mở rộng Forwarding Unit bằng cách bổ sung tín hiệu điều khiển forwardACC. Kênh này quét địa chỉ thanh ghi đích rd đang đứng ở giai đoạn MEM và WB hoạt động tương tự như forwardA và

forwardB. Dựa trên các điều kiện và mức độ ưu tiên giống hệt bảng 3.11, forwardACC sẽ quyết định định tuyến kết quả mới:

- Chuyển tiếp từ EX/MEM: Lấy ngõ ra ALU/SIMD Pext tại tầng MEM.
- Chuyển tiếp từ MEM/WB: Lấy dữ liệu chuẩn bị ghi Write Back tại tầng WB.

❖ *Hazard Detection*

Việc lệnh MAC sử dụng thanh ghi đích rd làm toán hạng thứ ba cũng đòi hỏi khối Hazard Detection Unit phải được mở rộng để giám sát thêm nguy cơ Load-Use trên toán hạng này.

Để giải quyết, khối Hazard Detection Unit được mở rộng bằng cách sử dụng cờ nhận diện lệnh MAC is_mac_ID tại tầng ID để bổ sung nhánh kiểm tra xung đột. Dựa trên cơ chế cơ bản giống hệt bảng 3.16, tín hiệu stall sẽ được kích hoạt để stall pipeline 1 chu kỳ khi thỏa mãn đồng thời:

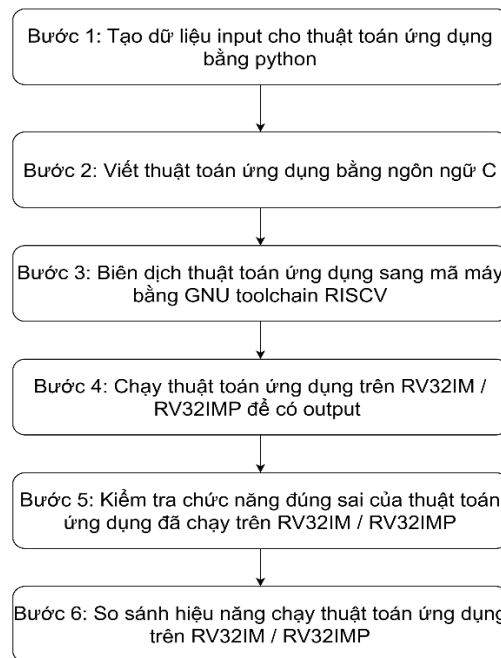
- Lệnh đi trước ở tầng EX là lệnh LOAD.
- Đích ghi của lệnh LOAD (rd_EX) trùng khớp với thanh ghi rd_ID của lệnh MAC.

Việc đóng băng này đảm bảo lệnh LOAD có đủ thời gian đưa dữ liệu về đến chốt MEM/WB, từ đó chuyển tiếp an toàn vào ngõ nhận ACC thông qua kênh forwardACC.

3.5. QUY TRÌNH ĐÁNH GIÁ VÀ KIỂM THỬ CÁC ỨNG DỤNG

Quy trình đánh giá và kiểm thử các ứng dụng được xây dựng nhằm đánh giá khả năng thực thi chương trình thực tế của bộ xử lý RV32IM tích hợp P-extension. Ba ứng dụng được lựa chọn để kiểm thử gồm bộ lọc bộ lọc FIR, thuật toán phát hiện biên Sobel và nhân ma trận.

Hình 3.21 mô tả tổng quát quy trình đánh giá và kiểm thử các ứng dụng trên hai kiến trúc RV32IM và RV32IMP, từ bước tạo dữ liệu đầu vào, xây dựng và biên dịch chương trình, đến kiểm tra kết quả và so sánh hiệu năng thực thi.



Hình 3.21: Quy trình đánh giá và kiểm thử các ứng dụng trên RV32IM và RV32IMP

Các liệu đầu vào được tạo trước bằng Python. Cụ thể, Python tạo tín hiệu và hệ số lọc cho FIR, chuyển ảnh đầu vào của Sobel thành mảng dữ liệu và tạo hai ma trận đầu vào cho nhân ma trận. Các dữ liệu này được xuất ra file hex để nạp vào vùng Data Memory của hệ thống mô phỏng.

Chương trình xử lý của từng ứng dụng được viết bằng ngôn ngữ C và được biên dịch bằng RISC-V GNU Toolchain, cụ thể là riscv32-unknown-elf-gcc. Sau khi biên dịch, mã chương trình được chuyển sang định file hex và nạp vào Instruction Memory. Khi mô phỏng bắt đầu, bộ xử lý đọc lệnh từ Instruction Memory, truy xuất dữ liệu từ Data Memory, thực hiện chương trình và ghi kết quả trở lại vùng nhớ đầu ra.

Sau khi chương trình kết thúc, testbench sẽ xuất nội dung vùng nhớ kết quả ra file hex. Đối với chương trình nhân ma trận, kết quả này được so sánh với dữ liệu golden do Python tính toán để kiểm tra độ chính xác. Đối với FIR và Sobel, dữ liệu đầu ra được Python đọc lại để vẽ tín hiệu hoặc tái tạo ảnh đầu ra, từ đó đánh giá trực quan kết quả xử lý của bộ xử lý. Sau khi xác nhận kết quả thực thi đúng, các thông số như số chu kỳ thực thi và số lệnh hoàn thành được thu thập để so sánh hiệu năng giữa hai kiến trúc RV32IM và RV32IMP.

CHƯƠNG 4

KẾT QUẢ VÀ ĐÁNH GIÁ

4.1. KẾT QUẢ MÔ PHỎNG

4.1.1. Mô phỏng thiết kế RISC-V RV32IM

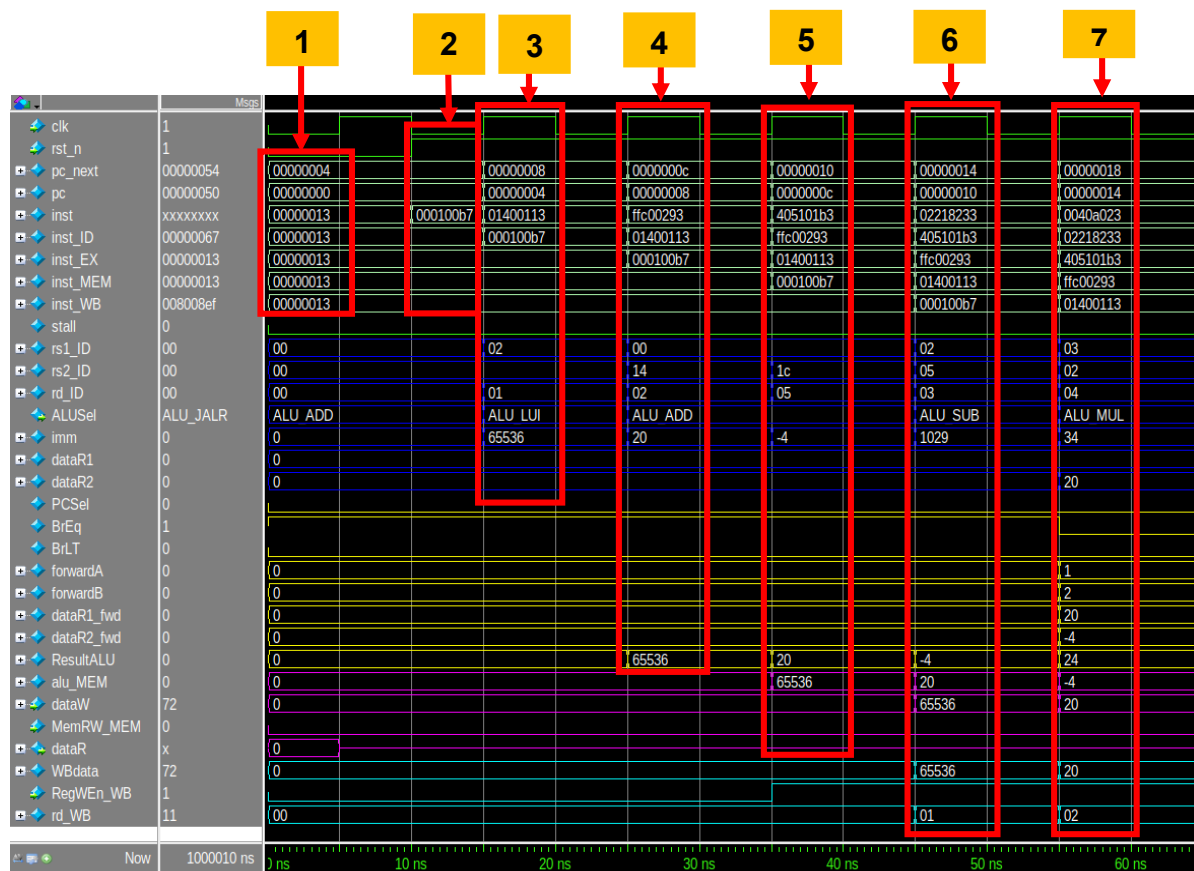
Bảng 4.1 trình bày mã Assembly dùng để mô phỏng và kiểm tra hoạt động của các nhóm lệnh chính trong kiến trúc RV32IM. Chương trình bao gồm các lệnh tính toán số học, logic, truy cập bộ nhớ, rẽ nhánh và nhảy. Các lệnh được sắp xếp có chủ đích nhằm tạo ra các tình huống hazard như RAW, load-use và control hazard để đánh giá khả năng xử lý của bộ xử lý pipeline.

Bảng 4.1: Mã Assembly dùng mô phỏng các lệnh của thiết kế RV32IM

PC	Mã Assembly	Mã hex	Chú thích
0x00	lui x1, 0x00010	0x000100B7	Gán giá trị cho x1= 0x10000 (65536)
0x04	addi x2, x0, 20	0x01400113	Thực hiện phép cộng x0 với 20, nhưng vì x0 luôn bằng 0 nên tương đương gán x2 = 20
0x08	addi x5, x0, -4	0xFFC00293	Tương tự khởi tạo giá trị x5 = -4
0x0C	sub x3, x2, x5	0x405101B3	Thực hiện phép trừ: $x3 = x2 - x5 = 20 - (-4) = 24$
0x10	mul x4, x3, x2	0x02218233	Thực hiện phép nhân: $x4 = x3 \times x2 = 24 \times 20 = 480$
0x14	sw x4, 0(x1)	0x0040A023	Lưu giá trị x4 vào Data Memory với địa chỉ $x1 + 0 = 65536$
0x18	lw x6, 0(x1)	0x0000A303	Load giá trị từ địa chỉ $x1 + 0 = 65536$ ra thanh ghi x6
0x1C	add x7, x6, x3	0x003303B3	Thực hiện cộng hai thanh ghi: $x7 = x6 + x3 = 480 + 24 = 504$

0x20	and x8, x7, x4	0x0043F433	Thực hiện phép AND: $x8 = x7 \& x4 = 504 \& 480 = 480$
0x24	or x9, x2, x3	0x003164B3	Thực hiện phép OR bit: $x9 = x2 \mid x3 = 20 \mid 24 = 28$
0x28	xor x10, x7, x4	0x0043C533	Thực hiện phép XOR bit: $x10 = x7 \wedge x4 = 504 \wedge 480 = 24$
0x2C	slli x11, x3, 1	0x00119593	Thực hiện phép dịch trái logic: $x11 = x3 \ll 1 = 24 \ll 1 = 48$
0x30	srli x12, x4, 4	0x00425613	Thực hiện phép dịch phải logic: $x12 = x4 \gg 4 = 480 \gg 4 = 30$
0x34	srai x13, x5, 1	0x4012D693	Thực hiện dịch phải số học: $x13 = x5 \ggg 1 = -4 \ggg 1 = -2$
0x38	slt x14, x5, x2	0x0022A733	Thực hiện so sánh có dấu: $x14 = (x5 < x2)$. Vì $-4 < 20$, nên $x14 = 1$
0x3C	bne x7, x4, 8	0x00439463	So sánh không bằng giữa $x7 = 504$ và $x4 = 480$, điều kiện đúng nên nhảy tới $PC = 0x00000044$
0x40	addi x16, x0, 123	0x07B00813	Gán giá trị $x16 = 123$, nhưng vì lệnh nhánh trước đó được thực thi nên lệnh này sẽ bị xóa
0x44	jal x17, 8	0x008008EF	Lệnh nhảy không điều kiện, lưu địa chỉ quay lại vào $x17 = PC + 4 = 0x00000048$, sau đó nhảy tới $PC = 0x0000004C$
0x48	addi x18, x0, 222	0x0DE00913	Gán giá trị $x18 = 222$, nhưng lệnh nhảy được thực thi nên lệnh này sẽ bị xóa
0x4C	jalr x0, 0(x0)	0x00000067	Nhảy gián tiếp về địa chỉ $x0 + 0 = 0x00000000$. Vì $x0 = 0$, không lưu địa chỉ quay lại. Lệnh này làm chương trình quay lại từ đầu

Hình 4.1 mô tả dạng sóng mô phỏng của thiết kế RV32IM từ 0 – 65ns, với các tín hiệu màu xanh lá là của giai đoạn nạp lệnh, các tín hiệu màu xanh dương là của giai đoạn giải mã lệnh, các tín hiệu màu vàng là của giai đoạn thực thi, các tín hiệu màu hồng là của giai đoạn truy cập bộ nhớ và các tín hiệu màu xanh ngọc là của giai đoạn ghi kết quả.



Hình 4.1: Dạng sóng mô phỏng của thiết kế RV32IM từ 0 – 65ns

❖ Tại điểm số 1

- Giá trị $rst_n = 0$ để khởi tạo giá PC cho địa chỉ đầu tiên là $32'h00000000$, đồng thời ta có thể thấy các inst ở các giai đoạn IF, ID, EX, MEM, WB đều được đặt với giá trị $32'h00000013$.

❖ Tại điểm số 2

- Ngay khi rst_n được kéo lên mức 1 thì inst ngay lập tức được cập nhật vào theo giá trị PC hiện tại là $0x00$, ứng với giá trị PC là lệnh lui x1, $0x00010$.

❖ Tại điểm số 3

- IF: Lệnh addi x2, x0, 20 được nạp tại địa chỉ PC = 0x04.
- ID: Lệnh lui x1, 0x00010 được giải mã với giá trị thanh ghi đích rd_ID = x1, ALUSel = ALU_LUI và giá trị tạm thời imm = 65536 ứng với 0x00010.

❖ Tại điểm số 4

- IF: Lệnh addi x5, x0, -4 được nạp tại địa chỉ PC = 0x08.
- ID: Lệnh addi x2, x0, 20 được giải mã với giá trị thanh ghi đích rd_ID = x2, thanh ghi nguồn rs1_ID = x0, ALUSel = ALU_ADD và giá trị tạm thời imm = 20.
- EX: Lệnh lui x1, 0x00010 được thực thi do đó giá trị ResultALU = 65536.

❖ Tại điểm số 5

- IF: Lệnh sub x3, x2, x5 được nạp tại địa chỉ PC = 0x0C.
- ID: Lệnh addi x5, x0, -4 được giải mã với giá trị thanh ghi đích rd_ID = x5, thanh ghi nguồn rs1_ID = x0, ALUSel = ALU_ADD và giá trị tạm thời imm = -4.
- EX: Lệnh addi x2, x0, 20 được thực thi và giá trị ResultALU = 20.
- MEM: giá trị ngõ ra của lệnh lui x1, 0x00010 lúc này chính alu_MEM, giá trị này là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 65536.

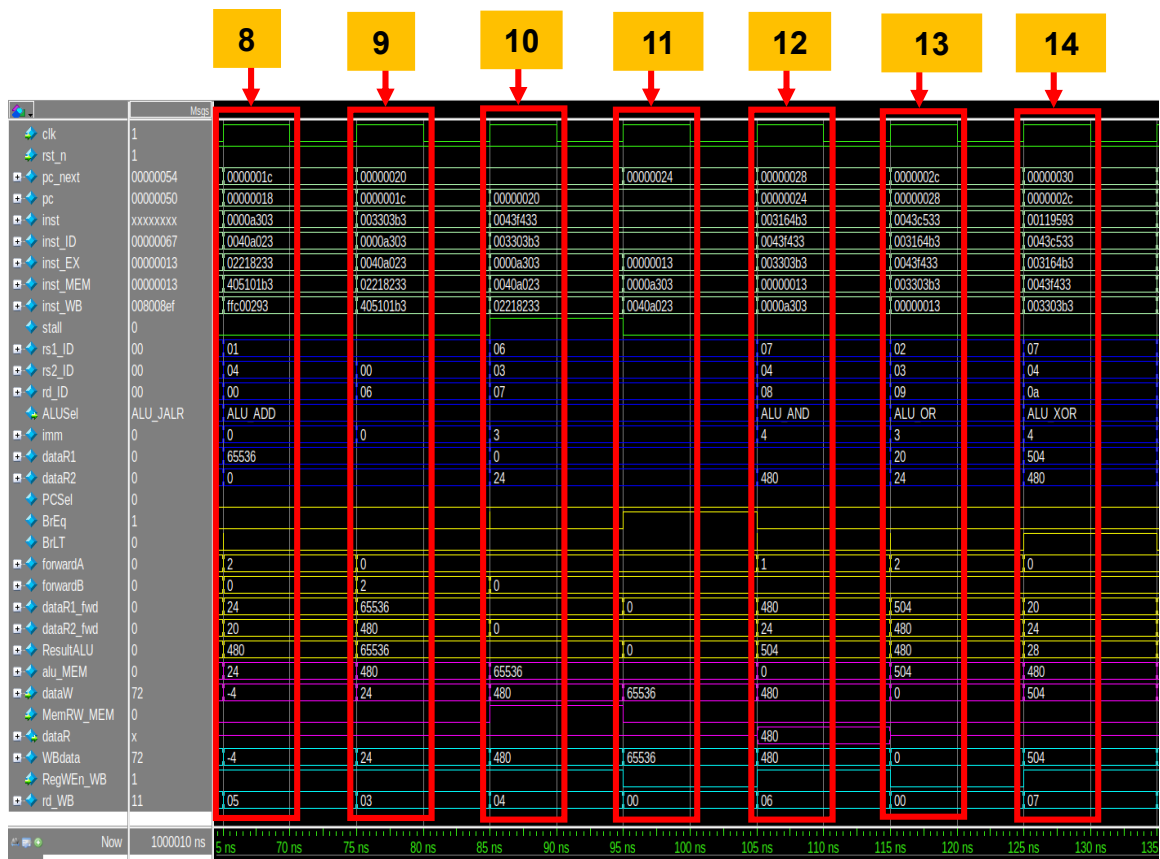
❖ Tại điểm số 6

- IF: Lệnh mul x4, x3, x2 được nạp tại địa chỉ PC = 0x10.
- ID: Lệnh sub x3, x2, x5 được giải mã với giá trị thanh ghi đích rd_ID = x3, thanh ghi nguồn rs1_ID = x2, rs2_ID = x5, ALUSel = ALU_SUB.
- EX: Lệnh addi x5, x0, -4 được thực thi và giá trị ResultALU = -4.
- MEM: Lệnh addi x2, x0, 20 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 20.
- WB: Lệnh lui x1, 0x00010 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x1 và giá trị WBdata = 65536.

❖ Tại điểm số 7

- IF: Lệnh sw x4, 0(x1) được nạp tại địa chỉ PC = 0x14.
- ID: Lệnh mul x4, x3, x2 được giải mã với giá trị thanh ghi đích rd_ID = x4, thanh ghi nguồn rs1_ID = x3, rs2_ID = x2, ALUSel = ALU_MUL.
- EX: Lệnh sub x3, x2, x5 được thực thi, tuy nhiên hai giá trị data của x2 và x5 chưa có nên ta sẽ forwarding do đó ta có thấy forwardA = 1 còn forwardB = 2 ứng với giá trị x2 = dataR1_fwd = 20 và x5 = dataR2_fwd = -4 do đó giá trị ResultALU = 24.
- MEM: Lệnh addi x5, x0, -4 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = -4.
- WB: Lệnh addi x2, x0, 20 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x2 và giá trị WBdata = 20.

Hình 4.2 mô tả dạng sóng mô phỏng của thiết kế RV32IM từ 65 – 135ns.



Hình 4.2: Dạng sóng mô phỏng của thiết kế RV32IM từ 65 – 135ns

❖ Tại điểm số 8

- IF: Lệnh lw x6, 0(x1) được nạp tại địa chỉ PC = 0x18.
- ID: Lệnh sw x4, 0(x1) được giải mã với giá trị thanh ghi nguồn rs1_ID = x1, rs2_ID = x4, ALUSel = ALU_ADD và giá trị imm = 0.
- EX: Lệnh mul x4, x3, x2 được thực thi tuy nhiên giá trị data của x3 chưa có nên ta sẽ forwarding do đó ta có thấy forwardA = 2 ứng với giá trị x3 = dataR1_fwd = 24 và giá trị ResultALU = 480.
- MEM: Lệnh sub x3, x2, x5 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 24.
- WB: Lệnh addi x5, x0, -4 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x5 và giá trị WBdata = -4.

❖ Tại điểm số 9

- IF: Lệnh add x7, x6, x3 được nạp tại địa chỉ PC = 0x1C.
- ID: Lệnh lw x6, 0(x1) được giải mã với giá trị thanh ghi đích rd_ID = x6, thanh ghi nguồn rs1_ID = x1, ALUSel = ALU_ADD và giá trị imm = 0.
- EX: Lệnh sw x4, 0(x1) được thực thi tuy nhiên giá trị data của x4 chưa có nên ta sẽ forwarding do đó ta có thấy forwardB = 2 ứng với giá trị x4 = dataR2_fwd = 480 và giá trị ResultALU = 65536.
- MEM: Lệnh mul x4, x3, x2 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 480.
- WB: Lệnh sub x3, x2, x5 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x3 và giá trị WBdata = 24.

❖ Tại điểm số 10

- IF: Lệnh and x8, x7, x4 được nạp tại địa chỉ PC = 0x20, ta có thể thấy stall = 1 do lw ở EX chưa xong x6 nhưng ID cần x6 do đó giá trị pc tiếp theo được nạp sẽ giữ nguyên.
- ID: Lệnh add x7, x6, x3 được giải mã với giá trị thanh ghi đích rd_ID = x7, thanh ghi nguồn rs1_ID = x6, rs2_ID = x3, ALUSel = ALU_ADD.

- EX: Lệnh lw x6, 0(x1) được thực thi với giá trị $x1 + 0$ tính được $Result_ALU = 65536$.
- MEM: Lệnh sw x4, 0(x1) có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó $alu_MEM = 65536$ và $MemRW_MEM = 1$ cho phép ghi vào Data Memory.
- WB: Lệnh mul x4, x3, x2 sẽ được ghi trở lại vào Register File, do đó $RegWEn_WB = 1$, $rd_WB = x4$ và giá trị $WBdata = 480$.

❖ Tại điểm số 11

- IF: Lệnh and x8, x7, x4 được giữ nguyên tại địa chỉ $PC = 0x20$.
- ID: Lệnh add x7, x6, x3 được giữ nguyên.
- EX: Lúc này nó được chèn bằng lệnh “NOP” tức là $32'h00000013$
- MEM: Lệnh lw x6, 0(x1) tiếp tục thực thi với địa chỉ là $alu_MEM = 65536$.
- WB: Lệnh sw x4, 0(x1) sẽ được hoàn thành và lệnh này ko ghi vào Register File, do đó $RegWEn_WB = 0$.

❖ Tại điểm số 12

- IF: Lệnh or x9, x2, x3 được nạp tại địa chỉ $PC = 0x24$.
- ID: Lệnh add x8, x7, x4 được giải mã với giá trị thanh ghi đích $rd_ID = x8$, thanh ghi nguồn $rs1_ID = x7$, $rs2_ID = x4$, $ALUSel = ALU_AND$.
- EX: Lệnh add x7, x6, x3 được thực thi tuy nhiên giá trị data của x6 chưa có nên ta sẽ forwarding do đó ta có thấy $forwardA = 1$ ứng với giá trị $x6 = dataR1_fwd = 480$ và giá trị $ResultALU = 504$.
- MEM: Mã lệnh ở MEM lúc này là $32'h00000013$.
- WB: Lệnh lw x6, 0(x1) sẽ được ghi trở lại vào Register File, do đó $RegWEn_WB = 1$, $rd_WB = x6$ và giá trị $WBdata = 480$.

❖ Tại điểm số 13

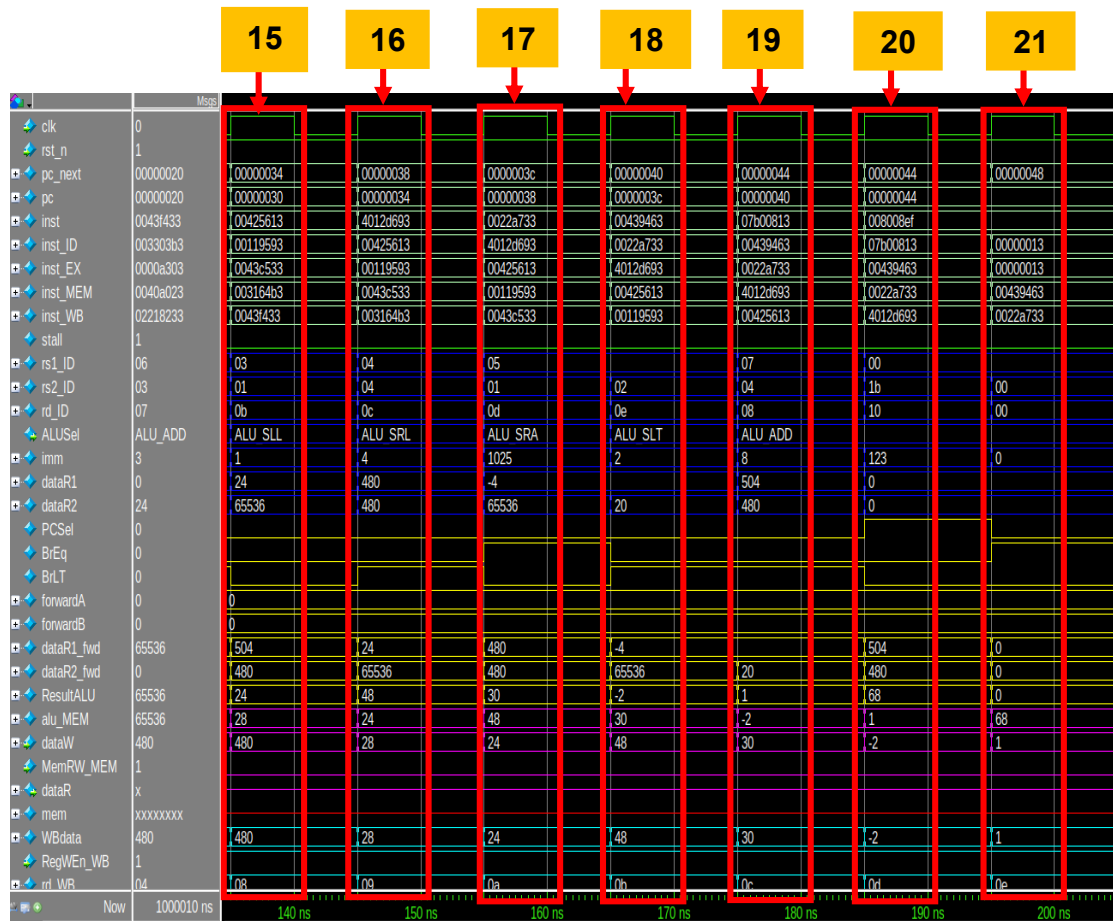
- IF: Lệnh xor x10, x7, x4 được nạp tại địa chỉ $PC = 0x28$.

- ID: Lệnh or x9, x2, x3 được giải mã với giá trị thanh ghi đích rd_ID = x9, thanh ghi nguồn rs1_ID = x2, rs2_ID = x3, ALUSel = ALU_OR.
- EX: Lệnh add x8, x7, x4 được thực thi tuy nhiên giá trị data của x7 chưa có nên ta sẽ forwarding do đó ta có thấy forwardA = 2 ứng với giá trị x7 = dataR1_fwd = 504 và giá trị ResultALU = 480.
- MEM: Lệnh add x7, x6, x3 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 504.
- WB: Mã lệnh ở WB lúc này là 32'h00000013.

❖ Tại điểm số 14

- IF: Lệnh slli x11, x3, 1 được nạp tại địa chỉ PC = 0x2C.
- ID: Lệnh xor x10, x7, x4 được giải mã với giá trị thanh ghi đích rd_ID = x10, thanh ghi nguồn rs1_ID = x7, rs2_ID = x4, ALUSel = ALU_XOR.
- EX: Lệnh or x9, x2, x3 được thực thi do đó giá trị ResultALU = 28.
- MEM: Lệnh add x8, x7, x4 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 480.
- WB: Lệnh add x7, x6, x3 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x7 và giá trị WBdata = 504.

Hình 4.3 mô tả dạng sóng mô phỏng của thiết kế RV32IM từ 135 – 205ns.



Hình 4.3: Dạng sóng mô phỏng của thiết kế RV32IM từ 135 – 205ns

❖ Tại điểm số 15

- IF: Lệnh srli x12, x4, 4 được nạp tại địa chỉ PC = 0x30.
- ID: Lệnh slli x11, x3, 1 được giải mã với giá trị thanh ghi đích rd_ID = x11 ứng với giá trị 0b, thanh ghi nguồn rs1_ID = x3, ALUSel = ALU_SLL và giá trị imm = 1.
- EX: Lệnh xor x10, x7, x4 được thực thi do đó giá trị ResultALU = 24.
- MEM: Lệnh or x9, x2, x3 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 28.
- WB: Lệnh add x8, x7, x4 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x8 và giá trị WBdata = 480.

❖ Tại điểm số 16

- IF: Lệnh srli x13, x5, 1 được nạp tại địa chỉ PC = 0x34.

- ID: Lệnh srli x12, x4, 4 được giải mã với giá trị thanh ghi đích rd_ID = x12 ứng với giá trị 0c, thanh ghi nguồn rs1_ID = x4, ALUSel = ALU_SRL và giá trị imm = 4.
- EX: Lệnh slli x11, x3, 1 được thực thi do đó giá trị ResultALU = 48.
- MEM: Lệnh xor x10, x7, x4 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 24.
- WB: Lệnh or x9, x2, x3 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x9 và giá trị WBdata = 28.

❖ Tại điểm số 17

- IF: Lệnh slt x14, x5, x2 được nạp tại địa chỉ PC = 0x38.
- ID: Lệnh srai x13, x5, 1 được giải mã với giá trị thanh ghi đích rd_ID = x13 ứng với giá trị 0d, thanh ghi nguồn rs1_ID = x5, ALUSel = ALU_SRA.
- EX: Lệnh srli x12, x4, 4 được thực thi do đó giá trị ResultALU = 30.
- MEM: Lệnh slli x11, x3, 1 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 48.
- WB: Lệnh xor x10, x7, x4 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x10 ứng với 0a và giá trị WBdata = 24.

❖ Tại điểm số 18

- IF: Lệnh bne x7, x4, 8 được nạp tại địa chỉ PC = 0x3C.
- ID: Lệnh slt x14, x5, x2 được giải mã với giá trị thanh ghi đích rd_ID = x14 ứng với giá trị 0e, thanh ghi nguồn rs1_ID = x5, rs2_ID = x2 và ALUSel = ALU_SLT.
- EX: Lệnh srai x13, x5, 1 được thực thi do đó giá trị ResultALU = -2.
- MEM: Lệnh srli x12, x4, 4 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 30.
- WB: Lệnh slli x11, x3, 1 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x11 ứng với 0b và giá trị WBdata = 48.

❖ Tại điểm số 19

- IF: Lệnh `addi x16, x0, 123` được nạp tại địa chỉ `PC = 0x40`.
- ID: Lệnh `bne x7, x4, 8` được giải mã với giá trị thanh ghi nguồn `rs1_ID = x7`, `rs2_ID = x4` và giá trị offset để nhảy còn gọi là `imm = 8`.
- EX: Lệnh `slt x14, x5, x2` được thực thi do đó giá trị `ResultALU = 1`.
- MEM: Lệnh `srai x13, x5, 1` có giá trị ngõ ra `alu_MEM` là giá trị ngõ ra ALU đã được tính toán ở EX do đó `alu_MEM = -2`.
- WB: Lệnh `srli x12, x4, 4` sẽ được ghi trở lại vào Register File, do đó `RegWEn_WB = 1`, `rd_WB = x12` ứng với `0c` và giá trị `WBdata = 30`.

❖ Tại điểm số 20

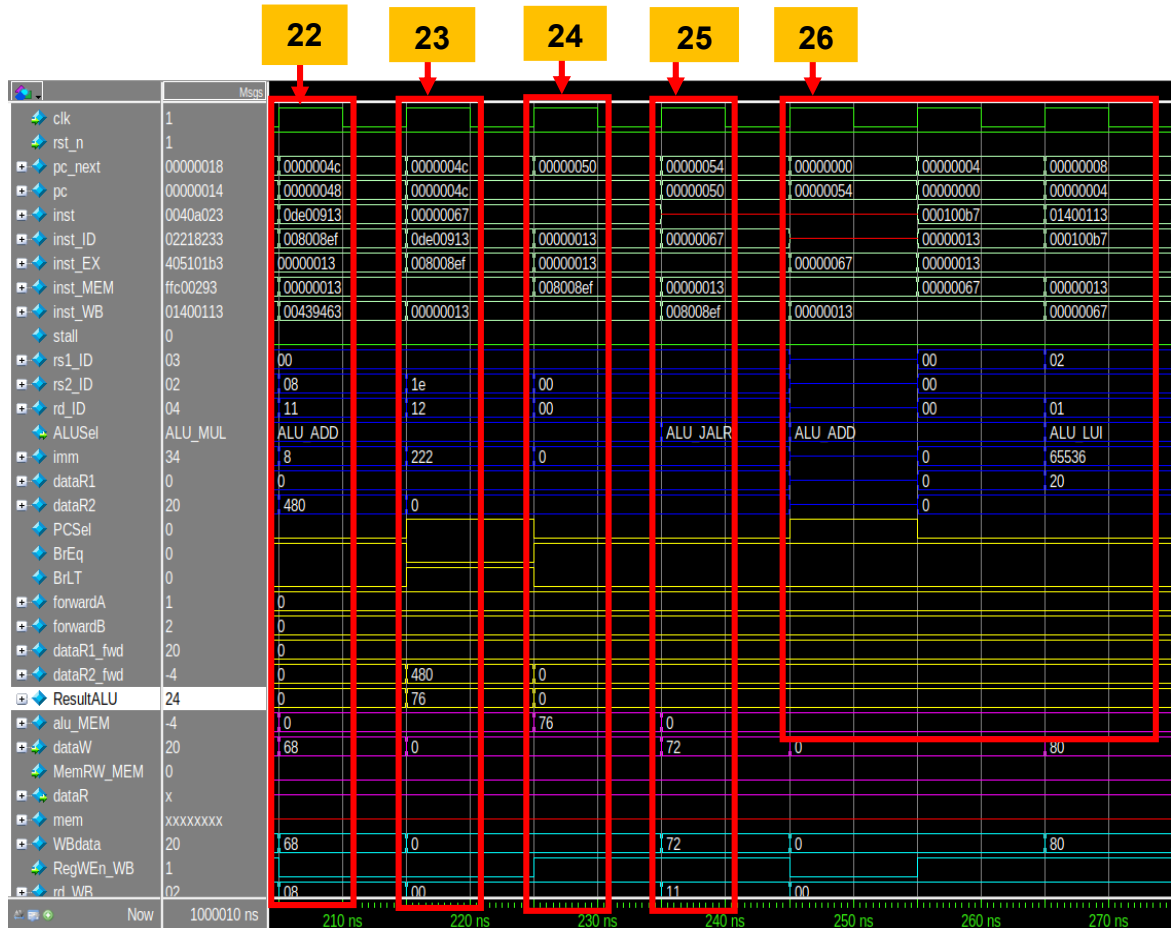
- IF: Lệnh `jal x17, 8` được nạp tại địa chỉ `PC = 0x44`.
- ID: Lệnh `addi x16, x0, 123` được giải mã với giá trị thanh ghi đích `rd_ID = x16` ứng với `10`, thanh ghi nguồn `rs1_ID = x0` và giá trị `imm = 123`.
- EX: Lệnh `bne x7, x4, 8` được thực thi vì `x7` không bằng `x4` thỏa điều kiện so sánh không bằng, do đó giá trị `PCSel = 1` và `Result_ALU = 68` tức là `0x44`.
- MEM: Lệnh `slt x14, x5, x2` có giá trị ngõ ra `alu_MEM` là giá trị ngõ ra ALU đã được tính toán ở EX do đó `alu_MEM = 1`.
- WB: Lệnh `x13, x5, 1` sẽ được ghi trở lại vào Register File, do đó `RegWEn_WB = 1`, `rd_WB = x13` ứng với `0d` và giá trị `WBdata = -2`.

❖ Tại điểm số 21

- IF: Lệnh `jal x17, 8` được nạp lại địa chỉ `PC = 0x44` do thỏa điều kiện thực hiện lệnh nhánh.
- ID, EX: Lệnh `jal x17, 8`, `addi x16, x0, 123` bị xóa bằng lệnh “NOP” tức là `32'h00000013` do đã nhảy tới `PC` là `0x44` nên hai lệnh đã Fetch và Decode sẽ không thực thi.
- MEM: Lệnh `bne x7, x4, 8` có giá trị ngõ ra `alu_MEM` là giá trị ngõ ra ALU đã được tính toán ở EX do đó `alu_MEM = 68`.

- WB: Lệnh slt x14, x5, x2 sẽ được ghi trở lại vào Register File, do đó $RegWEn_WB = 1$, $rd_WB = x14$ ứng với 0e và giá trị WBdata = 1.

Hình 4.4 mô tả dạng sóng mô phỏng của thiết kế RV32IM từ 205 – 275ns.



Hình 4.4: Dạng sóng mô phỏng của thiết kế RV32IM từ 205 – 275ns

❖ Tại điểm số 22

- IF: Lệnh addi x18, x0, 222 được nạp lại địa chỉ PC = 0x48.
- ID: Lệnh jal x17, 8 được giải mã với giá trị thanh ghi đích rd_ID = x17 ứng với 11 và giá trị offset là imm = 8.
- EX, MEM: Lệnh ở EX và MEM là 32'h00000013.
- WB: Lệnh bne x7, x4, 8 sẽ được hoàn tất và không ghi trở lại vào Register File, do đó $RegWEn_WB = 0$.

❖ Tại điểm số 23

- IF: Lệnh jalr x0, 0(x0) được nạp tại địa chỉ PC = 0x4C.

- ID: Lệnh `addi x18, x0, 222` được giải mã với giá trị thanh ghi đích `rd_ID = x18` ứng với 12, thanh ghi nguồn `rs1_ID = x0` và giá trị `imm = 222`.
- EX: Lệnh `jal x17, 8` được thực thi và nhảy tới địa chỉ `PC = 76` tức là `0x4c` và `PCSel = 1`.
- MEM, WB: Lệnh ở MEM và WB là `32'h00000013`.

❖ Tại điểm số 24

- IF: Lệnh `jalr x0, 0(x0)` được nạp lại địa chỉ `PC = 0x4C` do đã thực hiện lệnh nhảy.
- ID, EX: Lệnh `jalr x0, 0(x0)`, `addi x18, x0, 222` bị xóa bằng lệnh “NOP” tức là `32'h00000013` do đã nhảy tới PC là `0x4C` nên hai lệnh đã Fetch và Decode sẽ không thực thi.
- MEM: Lệnh `jal x17, 8` có giá trị ngõ ra `alu_MEM` là giá trị ngõ ra ALU đã được tính toán ở EX do đó `alu_MEM = 76`.
- WB: Lệnh ở WB là `32'h00000013`.

❖ Tại điểm số 25

- IF: Do hết chương trình nên không có thêm lệnh nạp vào
- ID: Lệnh `jalr x0, 0(x0)` được giải mã với giá trị thanh ghi đích `rd_ID = x0`, thanh ghi nguồn `rs1_ID = x0`, giá trị `offset` là `imm = 0` và `ALUSel = ALU_JARL`.
- EX, MEM: Lệnh ở EX và MEM là `32'h00000013`.
- WB: Lệnh `jal x17, 8` sẽ được hoàn tất và ghi trở lại vào Register File, do đó `RegWEn_WB = 1`, `WB_data = PC + 4 = 0x48 = 72`.

❖ Tại điểm số 26

- IF, ID: Do hết chương trình nên không có thêm lệnh nạp vào.
- EX: Lệnh `jalr x0, 0(x0)` được thực thi do đó `PC_next = 32'h00000000` và `PCSel = 1`.
- MEM, WB: Lệnh ở MEM và WB là `32'h00000013`.
- Ở clock kế tiếp PC nạp vào là `32'h00000000`, do đó sẽ chạy lại từ đầu theo đã dự định.

4.1.2. Mô phỏng thiết kế RISC-V RV32IMP

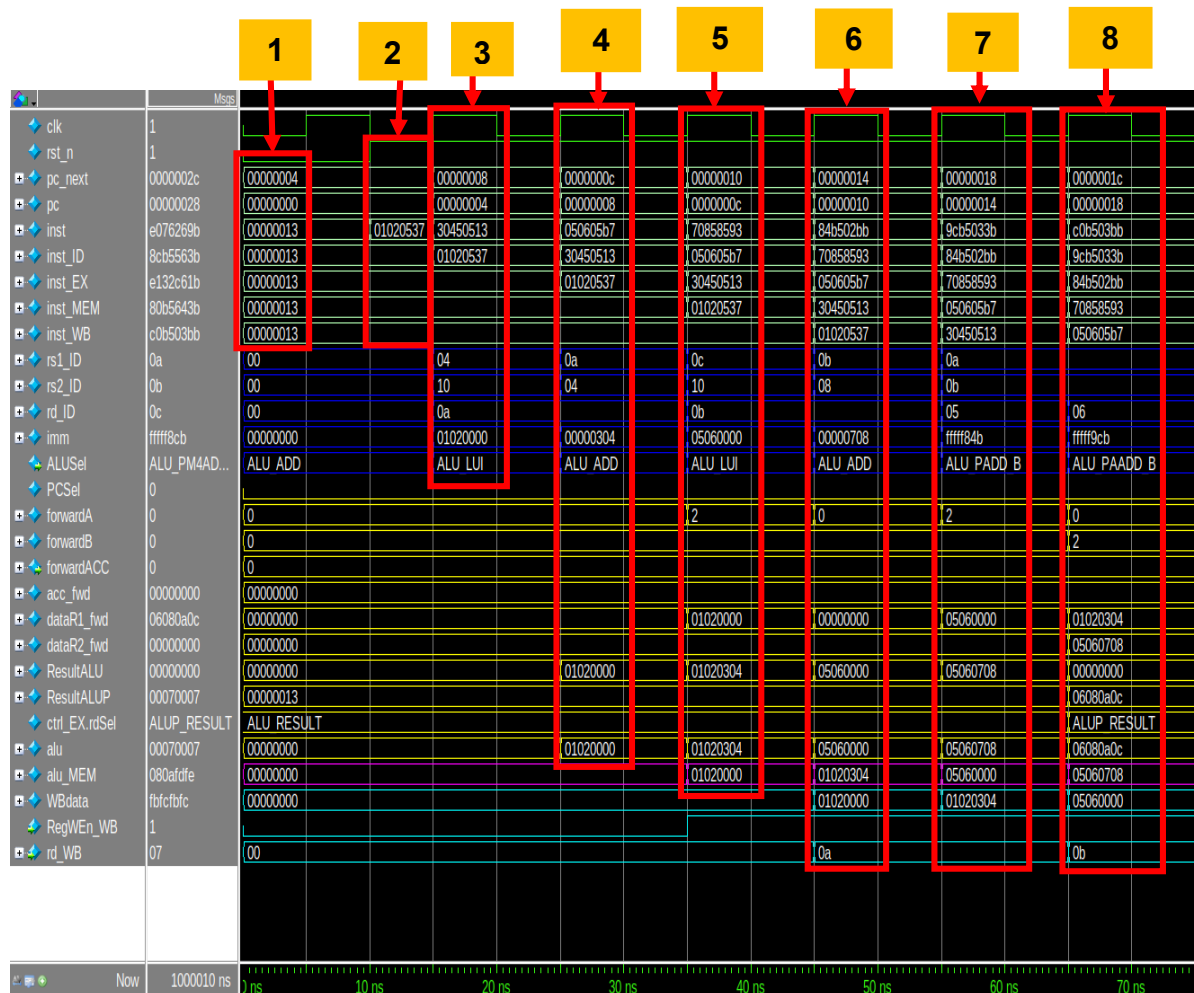
Bảng 4.2 trình bày mã Assembly dùng để mô phỏng và kiểm tra hoạt động của một số nhóm lệnh chính trong kiến trúc RV32IMP. Chương trình gồm các lệnh khởi tạo dữ liệu và các lệnh P-extension như cộng SIMD theo byte, cộng trung bình, trừ halfword, cộng trừ chéo theo halfword, giới hạn dữ liệu, MAC, trị tuyệt đối và max theo halfword. Các lệnh được sắp xếp liên tiếp nhằm kiểm tra kết quả tính toán của từng nhóm lệnh. Đồng thời, chuỗi lệnh psati.h, pm4adda.b và psabs.h tạo phụ thuộc dữ liệu RAW rõ ràng giữa các thanh ghi. Qua đó, chương trình hỗ trợ đánh giá khả năng forwarding của pipeline khi xử lý lệnh MAC.

Bảng 4.2: Mã Assembly dùng mô phỏng các lệnh của thiết kế RV32IMP

PC	Mã Assembly	Mã hex	Chú thích
0x00	lui x10, 0x01020	0x01020537	Gán giá trị x10 = 0x01020000
0x04	addi x10, x10, 0x304	0x30450513	Thực hiện cộng x10 = x10 + 0x304 = 0x01020304
0x08	lui x11, 0x05060	0x050605B7	Gán giá trị x11 = 0x05060000
0x0C	addi x11, x11, 0x708	0x70858593	Thực hiện cộng x11 = x11 + 0x708 = 0x05060708
0x10	padd.b x5, x10, x11	0x84b502bb	Cộng SIMD theo từng byte: 01+05, 02+06, 03+07, 04+08, kết quả x5 = 0x06080A0C
0x14	paadd.b x6, x10, x11	0x9cb5033b	Cộng trung bình có dấu theo từng byte, kết quả x6 = 0x03040506
0x18	psub.h x7, x10, x11	0xc0b503bb	Trừ SIMD theo từng halfword 16 bit. Kết quả lưu vào x7 = 0xFBFCFBFC
0x1C	pas.hx x8, x10, x11	0x80b5643b	Cộng/trừ chéo theo halfword: 0x0102 + 0x0708 = 0x080A, 0x0304 - 0x0506 = 0xFDFE, kết quả x8 = 0x080AFDFE

0x20	psati.h x12, x5, 3	0xe132c61b	Giới hạn dữ liệu có dấu halfword của x5 trong khoảng [-8, 7], do hai halfword của x5 đều lớn hơn 7 nên kết quả x12 = 0x00070007
0x24	pm4adda.b x12, x10, x11	0x8cb5563b	Thực hiện MAC: $x12 = x12 + (1 \times 5 + 2 \times 6 + 3 \times 7 + 4 \times 8)$, tức $x12 = 0x00070007 + 0x46 = 0x0007004D$
0x28	psabs.h x13, x12	0xe076269b	Lấy trị tuyệt đối theo từng halfword của x12, vì cả hai halfword đều dương nên x13 = 0x0007004D
0x2C	pmax.h x14, x13, x11	0xf0b6e73b	Lấy giá trị lớn hơn có dấu theo từng halfword giữa x13 và x11, kết quả ghi vào x14 = 0x05060708
0x30	j stop	0x0000006f	Lệnh nhảy về chính nó để giữ chương trình dừng tại vòng lặp

Hình 4.5 mô tả dạng sóng mô phỏng của thiết kế RV32IMP từ 0 – 75ns, với các tín hiệu màu xanh lá là của giai đoạn nạp lệnh, các tín hiệu màu xanh dương là của giai đoạn giải mã lệnh, các tín hiệu màu vàng là của giai đoạn thực thi, các tín hiệu màu hồng là của giai đoạn truy cập bộ nhớ và các tín hiệu màu xanh ngọc là của giai đoạn ghi kết quả.



Hình 4.5: Dạng sóng mô phỏng của thiết kế RV32IMP từ 0 – 75ns

❖ Tại điểm số 1

- Giá trị $rst_n = 0$ để khởi tạo giá PC cho địa chỉ đầu tiên là $32'h00000000$, đồng thời ta có thể thấy các inst ở các giai đoạn IF, ID, EX, MEM, WB đều được đặt với giá trị $32'h00000013$.

❖ Tại điểm số 2

- Ngay khi rst_n được kéo lên mức 1 thì inst ngay lập tức được cập nhật vào theo giá trị PC hiện tại là $0x00$, ứng với giá trị PC là lệnh lui $x10, 0x01020$.

❖ Tại điểm số 3

- IF: Lệnh $addi\ x10, x10, 0x304$ được nạp tại địa chỉ $PC = 0x04$.

- ID: Lệnh lui x10, 0x01020 được giải mã với giá trị thanh ghi đích rd_ID = x10 ứng với 0a, ALUSel = ALU_LUI và giá trị tạm thời imm = 0x01020000.

❖ Tại điểm số 4

- IF: Lệnh lui x11, 0x05060 được nạp tại địa chỉ PC = 0x08.
- ID: Lệnh addi x10, x10, 0x304 được giải mã với giá trị thanh ghi đích rd_ID = x10, thanh ghi nguồn rs1_ID = x10, ALUSel = ALU_ADD và giá trị tạm thời imm = 0x00000304.
- EX: Lệnh lui x10, 0x01020 được thực thi, rdSel chọn ALU_RESULT, do đó alu = 0x01020000 vì lệnh lui một lệnh của RV32IM.

❖ Tại điểm số 5

- IF: Lệnh addi x11, x11, 0x708 được nạp tại địa chỉ PC = 0x0C.
- ID: Lệnh lui x11, 0x05060 được giải mã với giá trị thanh ghi đích rd_ID = x11 ứng với 0b, thanh ghi nguồn rs1_ID = x11 ứng với 0b, ALUSel = ALU_LUI và giá trị tạm thời imm = 0x05060000.
- EX: Lệnh addi x10, x10, 0x304 được thực thi và giá trị x10 hiện tại chưa có nên forwardA = 2 ứng với x10 = dataR1_fwd = 0x01020000, rdSel chọn ALU_RESULT, do đó giá trị alu = 0x01020304.
- MEM: giá trị ngõ ra của lui x10, 0x01020 lúc này chính alu_MEM, giá trị này là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 0x01020000.

❖ Tại điểm số 6

- IF: Lệnh padd.b x5, x10, x11 được nạp tại địa chỉ PC = 0x10.
- ID: Lệnh addi x11, x11, 0x708 được giải mã với giá trị thanh ghi đích rd_ID = x11 ứng với 0b, thanh ghi nguồn rs1_ID = x11 ứng với 0b, ALUSel = ALU_ADD và giá trị imm = 0x00000708.
- EX: Lệnh lui x11, 0x05060 được thực thi, rdSel chọn ALU_RESULT, do đó giá trị alu = 0x05060000.

- MEM: Lệnh `addi x10, x10, 0x304` có giá trị ngõ ra `alu_MEM` là giá trị ngõ ra ALU đã được tính toán ở EX do đó `alu_MEM = 0x01020304`.
- WB: Lệnh lui `x10, 0x01020` sẽ được ghi trở lại vào Register File, do đó `RegWEn_WB = 1`, `rd_WB = x10` ứng với 0a và giá trị `WBdata = 0x01020000`.

❖ Tại điểm số 7

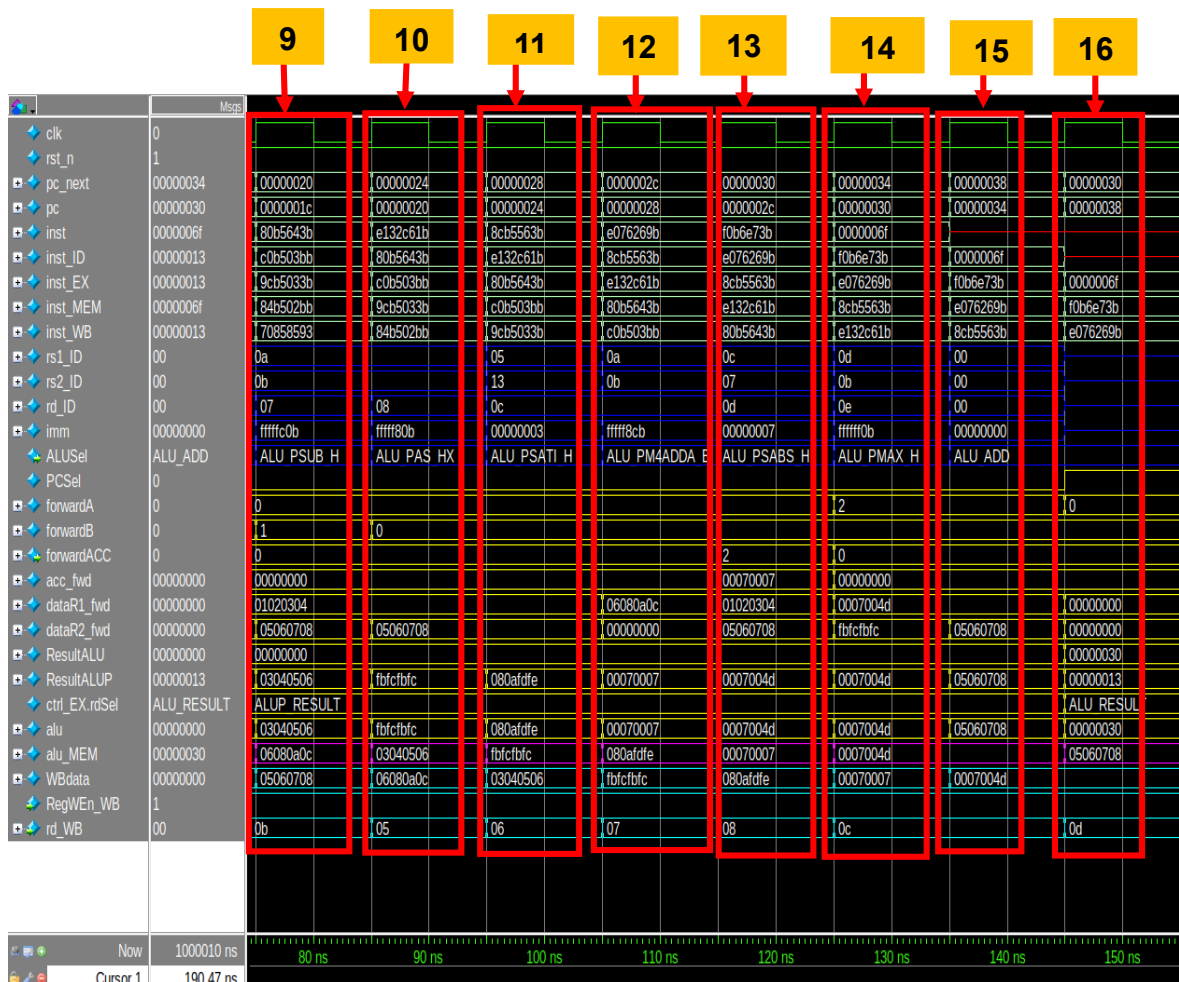
- IF: Lệnh `paadd.b x6, x10, x11` được nạp tại địa chỉ `PC = 0x14`.
- ID: Lệnh `padd.b x5, x10, x11` được giải mã với giá trị thanh ghi đích `rd_ID = x5`, thanh ghi nguồn `rs1_ID = x10` ứng với 0a, `rs2_ID = x11` ứng với 0b, `ALUSel = ALU_PADD_B`.
- EX: Lệnh `addi x11, x11, 0x708` được thực thi và giá trị `x11` hiện tại chưa có nên `forwardA = 2` ứng với `x11 = dataR1_fwd = 0x05060000`, `rdSel` chọn `ALU_RESULT`, do đó giá trị `alu = 0x05060708`.
- MEM: Lệnh lui `x11, 0x05060` có giá trị ngõ ra `alu_MEM` là giá trị ngõ ra ALU đã được tính toán ở EX do đó `alu_MEM = 0x05060000`.
- WB: Lệnh `addi x10, x10, 0x304` sẽ được ghi trở lại vào Register File, do đó `RegWEn_WB = 1`, `rd_WB = x10` ứng với 0a và giá trị `WBdata = 0x01020304`.

❖ Tại điểm số 8

- IF: Lệnh `psub.h x7, x10, x11` được nạp tại địa chỉ `PC = 0x18`.
- ID: Lệnh `paadd.b x6, x10, x11` được giải mã với giá trị thanh ghi đích `rd_ID = x6`, thanh ghi nguồn `rs1_ID = x10` ứng với 0a, `rs2_ID = x11` ứng với 0b, `ALUSel = ALU_PAADD_B`.
- EX: Lệnh `padd.b x5, x10, x11` được thực thi và giá trị `x11` hiện tại chưa có nên `forwardB = 2` ứng với `x11 = dataR2_fwd = 0x05060708`, `rdSel` chọn `ALUP_RESULT` vì đây lệnh của P-extension, do đó giá trị `alu = 0x06080a0c`.

- MEM: Lệnh `addi x11, x11, 0x708` có giá trị ngõ ra `alu_MEM` là giá trị ngõ ra ALU đã được tính toán ở EX do đó `alu_MEM = 0x05060708`.
- WB: Lệnh lui `x11, 0x05060` sẽ được ghi trở lại vào Register File, do đó `RegWEn_WB = 1`, `rd_WB = x11` ứng với `0b` và giá trị `WBdata = 0x05060000`.

Hình 4.6 mô tả dạng sóng mô phỏng của thiết kế RV32IMP từ 75 – 155ns.



Hình 4.6: Dạng sóng mô phỏng của thiết kế RV32IMP từ 75 – 155ns

❖ Tại điểm số 9

- IF: Lệnh `pas.hx x8, x10, x11` được nạp tại địa chỉ `PC = 0x1C`.
- ID: Lệnh `psub.h x7, x10, x11` được giải mã với giá trị thanh ghi đích `rd_ID = x7`, thanh ghi nguồn `rs1_ID = x10` ứng với `0a`, `rs2_ID = x11` ứng với `0b`, `ALUSel = ALU_PSUB_H`.

- EX: Lệnh paadd.b x6, x10, x11 được thực thi và giá trị x11 hiện tại chưa có nên forwardB = 1 ứng với x11 = dataR2_fwd = 0x05060708, rdSel chọn ALUP_RESULT vì đây lệnh của P-extension, do đó giá trị alu = 0x03040506.
- MEM: Lệnh padd.b x5, x10, x11 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 0x06080a0c.
- WB: Lệnh addi x11, x11, 0x708 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x11 ứng với 0b và giá trị WBdata = 0x05060708.

❖ Tại điểm số 10

- IF: Lệnh psati.h x12, x5, 3 được nạp tại địa chỉ PC = 0x20.
- ID: Lệnh pas.hx x8, x10, x11 được giải mã với giá trị thanh ghi đích rd_ID = x8, thanh ghi nguồn rs1_ID = x10 ứng với 0a, rs2_ID = x11 ứng với 0b, ALUSel = ALU_PAS_HX.
- EX: Lệnh psub.h x7, x10, x11 được thực thi, rdSel chọn ALUP_RESULT vì đây lệnh của P-extension, do đó giá trị alu = 0xfbfcfbfc.
- MEM: Lệnh paadd.b x6, x10, x11 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 0x03040506.
- WB: Lệnh padd.b x5, x10, x11 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x5 và giá trị WBdata = 0x06080a0c.

❖ Tại điểm số 11

- IF: Lệnh pm4adda.b x12, x10, x11 được nạp tại địa chỉ PC = 0x24.
- ID: Lệnh psati.h x12, x5, 3 được giải mã với giá trị thanh ghi đích rd_ID = x12 ứng với 0c, thanh ghi nguồn rs1_ID = x5, ALUSel = ALU_PSATI_H và giá trị imm = 3.

- EX: Lệnh pas.hx x8, x10, x11 được thực thi, rdSel chọn ALUP_RESULT vì đây lệnh của P-extension, do đó giá trị alu = 0x080afdfe.
- MEM: Lệnh psub.h x7, x10, x11 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 0xfbfcfbfc.
- WB: Lệnh paadd.b x6, x10, x11 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x6 và giá trị WBdata = 0x03040506.

❖ Tại điểm số 12

- IF: Lệnh psabs.h x13, x12 được nạp tại địa chỉ PC = 0x28.
- ID: Lệnh pm4adda.b x12, x10, x11 được giải mã với giá trị thanh ghi đích rd_ID = x12 ứng với 0c, thanh ghi nguồn rs1_ID = x10 ứng với 0a, rs2_ID = x11 ứng với 0b, ALUSel = ALU_PM4ADDA_B.
- EX: Lệnh psati.h x12, x5, 3 được thực thi, rdSel chọn ALUP_RESULT vì đây lệnh của P-extension, do đó giá trị alu = 0x00070007.
- MEM: Lệnh pas.hx x8, x10, x11 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 0x080afdfe.
- WB: Lệnh psub.h x7, x10, x11 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x7 và giá trị WBdata = 0xfbfcfbfc.

❖ Tại điểm số 13

- IF: Lệnh pmax.h x14, x13, x11 được nạp tại địa chỉ PC = 0x2C.
- ID: Lệnh psabs.h x13, x12 được giải mã với giá trị thanh ghi đích rd_ID = x13 ứng với 0d, thanh ghi nguồn rs1_ID = x12 ứng với 0c, ALUSel = ALU_PSABS_H.
- EX: Lệnh pm4adda.b x12, x10, x11 được thực thi tuy nhiên giá trị thanh ghi x12 hiện tại chưa có mà lệnh này thì cần giá trị thanh ghi đích hiện tại nên forwardACC = 2 ứng với x12 = acc_fwd =

0x00070007, rdSel chọn ALUP_RESULT vì đây lệnh của P-extension, do đó giá trị alu = 0x0007004d.

- MEM: Lệnh psati.h x12, x5, 3 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 0x00070007.
- WB: Lệnh pas.hx x8, x10, x11 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x8 và giá trị WBdata = 0x080afdfe.

❖ Tại điểm số 14

- IF: Lệnh j stop được nạp tại địa chỉ PC = 0x30.
- ID: Lệnh pmax.h x14, x13, x11 được giải mã với giá trị thanh ghi đích rd_ID = x14 ứng với 0e, thanh ghi nguồn rs1_ID = x13 ứng với 0d, rs2_ID = x11 ứng với 0b, ALUSel = ALU_PMAX_H.
- EX: Lệnh psabs.h x13, x12 được thực thi tuy nhiên giá trị x12 chưa có nên forwardA = 2 ứng với x12 = dataR1_fwd = 0x0007004d, rdSel chọn ALUP_RESULT vì đây lệnh của P-extension, do đó giá trị alu = 0x0007004d.
- MEM: Lệnh pm4adda.b x12, x10, x11 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 0x0007004d.
- WB: Lệnh psati.h x12, x5, 3 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x12 ứng với 0c và giá trị WBdata = 0x00070007.

❖ Tại điểm số 15

- IF: Không còn lệnh để nạp.
- ID: Lệnh j stop được giải mã với giá trị thanh ghi đích rd_ID = x0, giá trị imm = 0 và ALUSel = ALU_ADD.
- EX: Lệnh pmax.h x14, x13, x11 được thực thi tuy nhiên giá trị x13 chưa có nên forwardA = 2 ứng với x13 = dataR1_fwd = 0x0007004d, rdSel chọn ALUP_RESULT vì đây lệnh của P-extension, do đó giá trị alu = 0x05060708.

- MEM: Lệnh psabs.h x13, x12 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 0x0007004d.
- WB: Lệnh pm4adda.b x12, x10, x11 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x12 ứng với 0c và giá trị WBdata = 0x0007004d.

❖ Tại điểm số 16

- IF: Không có lệnh được nạp.
- ID: Không có lệnh được nạp
- EX: Lệnh j stop được thực thi, rdSel chọn ALU_RESULT vì đây lệnh của RV32IM, do đó giá trị alu = PC + imm = PC = 0x30 và PCSel = 1 nên quay lại PC là 0x30 để tạo vòng lặp vô hạn.
- MEM: Lệnh pmax.h x14, x13, x11 có giá trị ngõ ra alu_MEM là giá trị ngõ ra ALU đã được tính toán ở EX do đó alu_MEM = 0x05060708.
- WB: Lệnh psabs.h x13, x12 sẽ được ghi trở lại vào Register File, do đó RegWEn_WB = 1, rd_WB = x13 ứng với 0d và giá trị WBdata = 0x0007004d.

4.2. ĐÁNH GIÁ HIỆU NĂNG QUA CÁC ỨNG DỤNG

Sau khi hoàn thành việc mô phỏng tất cả các lệnh của RV32IM và RV32IMP và tất cả đều hoạt động chính xác. Tiếp theo để đánh giá hiệu năng của hai thiết kế thì chúng ta sẽ tiến hành chạy thông qua ba ứng dụng bao gồm: Nhân ma trận, bộ lọc FIR và thuật toán phát hiện biên Sobel. Đặc điểm chung của các ứng dụng nêu trên là đều thực hiện phép toán là nhân và cộng lại với nhau và nó thực hiện lặp đi lặp lại cùng phép toán này trên nhiều dữ liệu khác nhau. Đây là dạng tính toán rất phù hợp để đánh giá hiệu quả của các lệnh mở rộng P-extension, đặc biệt là các lệnh xử lý dữ liệu song song và lệnh nhân cộng dồn MAC. Thông qua ba ứng dụng này, có thể so sánh rõ hơn sự khác biệt giữa RV32IM là phiên bản baseline và RV32IMP là phiên bản SIMD khi xử lý các bài toán có tính lặp cao. Kết quả sẽ là cơ sở để đánh giá mức độ cải thiện về số chu kỳ thực thi, thời gian xử lý và hiệu quả khai thác tài nguyên của kiến trúc được đề xuất.

4.2.1. Nhân ma trận

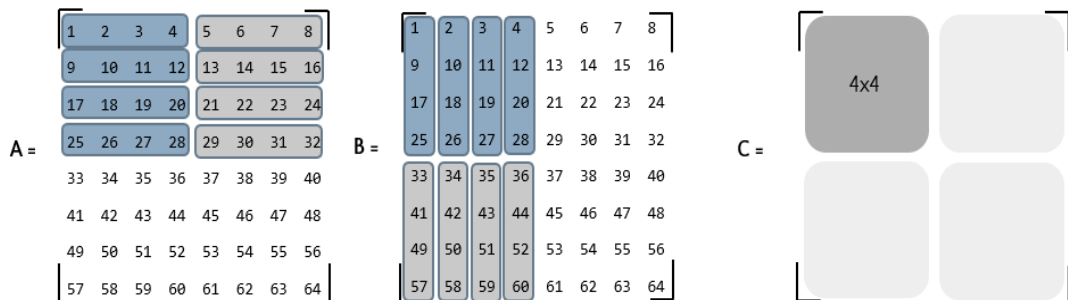
Nhân ma trận là một trong những phép toán cơ bản và thường gặp trong xử lý số và các ứng dụng khoa học tính toán. Kết quả của phép nhân hai ma trận A và B là một ma trận C, trong đó mỗi phần tử $C[i][j]$ được tính bằng tổng các tích của các phần tử tương ứng từ hàng i của ma trận A và cột j của ma trận B. Công thức tổng quát của nhân ma trận được biểu diễn như sau:

$$C[i][j] = \sum_{k=0}^{w_A-1} A[i][k].B[k][j] \quad (4.1)$$

Trong đó w_A là số cột của ma trận A và cũng là số hàng của ma trận B.

SIMD được áp dụng nhằm tăng tốc độ nhân ma trận so với phiên bản baseline. Với phiên bản baseline, các phần tử trong từng hàng của ma trận A và cột tương ứng của ma trận B vẫn được xử lý tuần tự, tức mỗi lần lặp chỉ thực hiện một phép nhân và cộng cho một phần tử. Còn với P-extension chúng ta sẽ sử dụng lệnh PM4ADDA.B sử dụng các thanh ghi 32 bit với bốn phần tử 8 bit liên tiếp của hàng A, cột B, thực hiện phép nhân cộng dồn trên bốn cặp phần tử và chương trình được tổ chức theo hướng chia khối 4×4 , tham khảo từ tài liệu [11]. Cả hai phiên bản đều được tổ chức theo dạng khối 4×4 , đồng thời thì mười sáu accumulator được sử dụng để lưu trữ tạm thời kết quả của toàn bộ khối 4×4 trong quá trình tính toán. Điểm khác biệt là phiên bản baseline tích lũy kết quả theo từng phần tử, còn phiên bản SIMD xử lý theo từng nhóm bốn phần tử 8 bit, giúp giảm số vòng lặp, số lệnh và tăng hiệu quả tính toán.

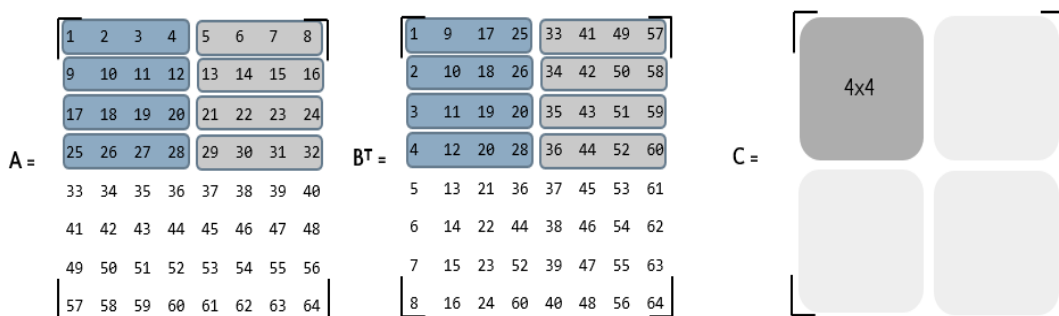
Hình 4.7 mô tả hoạt động nhân ma trận khi sử dụng SIMD, xử lý bằng các lệnh MAC song song cho phép tính toán bốn phần tử 8 bit cùng lúc.



Hình 4.7: Nhân ma trận sử dụng SIMD [11]

Tuy nhiên khi ta áp dụng SIMD trực tiếp trên các cột của ma trận, các giá trị nằm rải rác trong các byte khác nhau của bộ nhớ. Vì trong bộ nhớ ta sẽ chỉ lưu được dữ liệu theo hàng. Do đó, nếu muốn xử lý theo SIMD thì ta sẽ đọc bốn word thuộc bốn hàng liên tiếp của B, sau đó sử dụng các phép dịch bit, mask và OR để ghép các byte cùng cột thành word mong muốn. Điều này sẽ làm ảnh hưởng đến tốc độ của SIMD. Để cải thiện, ma trận B được chuyển vị trước sao cho các cột trở thành hàng liên tiếp trong bộ nhớ. Nhờ vậy, các giá trị cần thiết có thể được load trực tiếp vào thanh ghi và giảm được thao tác dịch bit, mask và ghép byte lặp lại trong vòng lặp chính và tối ưu hiệu quả thực thi SIMD.

Hình 4.8 mô tả quá trình nhân ma trận sử dụng SIMD, trong đó ma trận B được chuyển vị B^T để các cột của B trở thành hàng liên tiếp trong bộ nhớ. Các khối 4×4 của ma trận C được tính song song, tận dụng tối đa hiệu năng các lệnh MAC của SIMD giúp tăng hiệu suất tính toán.

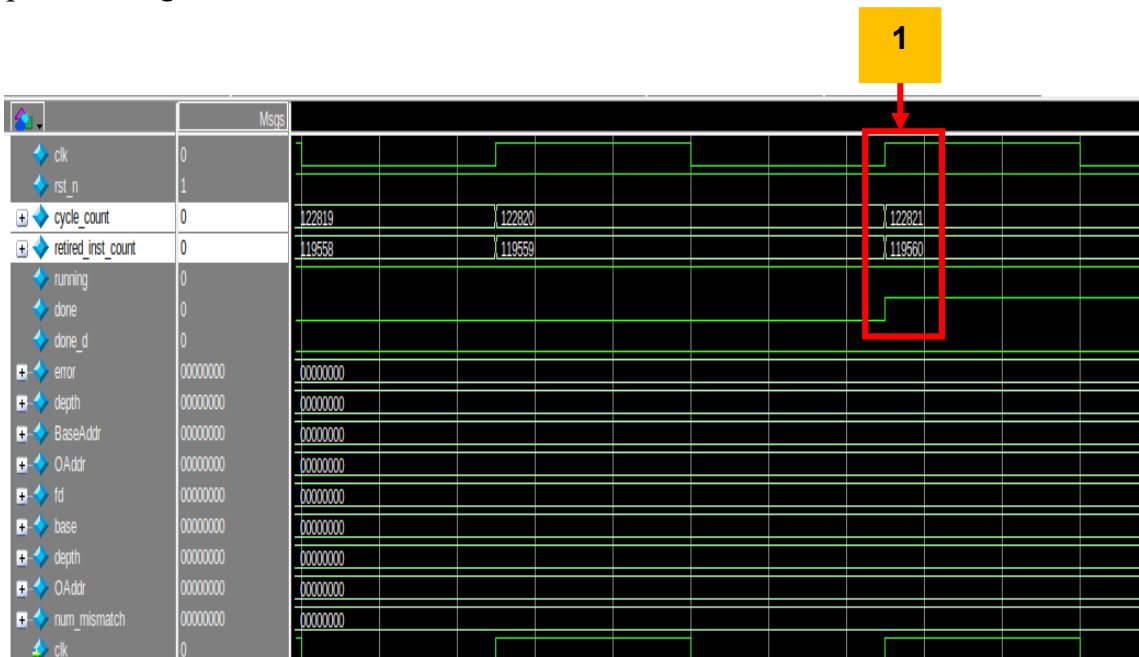


Hình 4.8: Nhân ma trận sử dụng SIMD cùng với ma trận chuyển vị B^T [11]

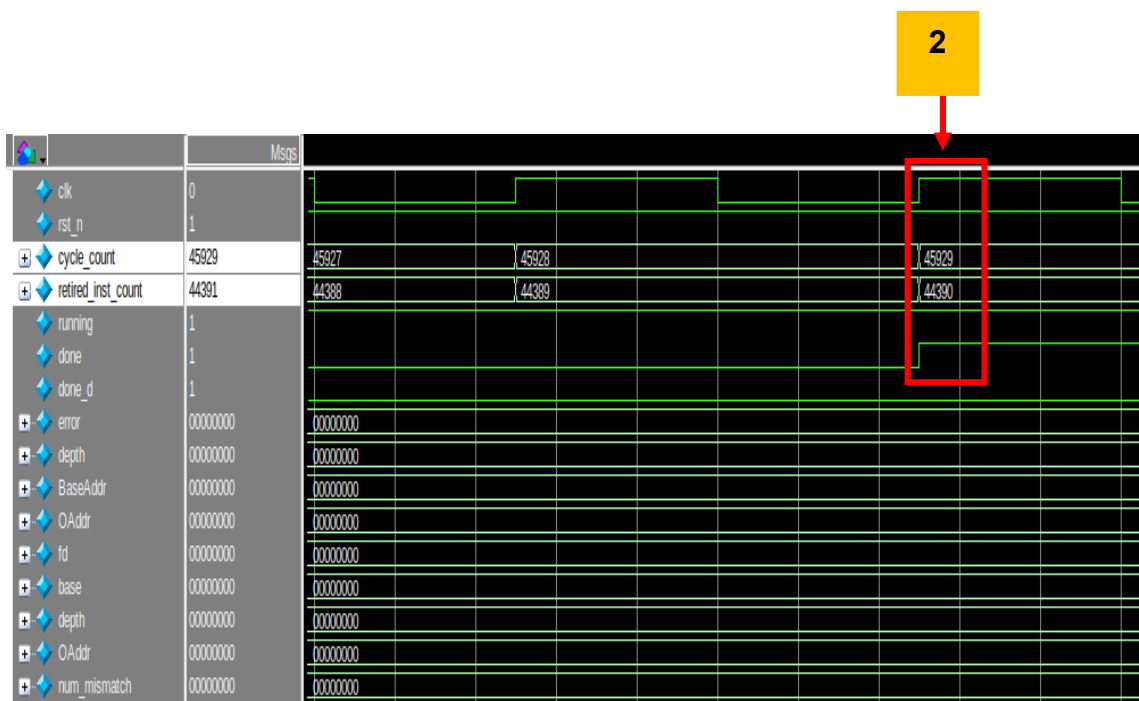
Trong bài toán nhân ma trận, hai ma trận đầu vào sẽ được tạo bằng Python. Các phần tử của hai ma trận được giới hạn trong khoảng 8 bit có dấu, nhằm phù hợp với định dạng dữ liệu đầu vào của lệnh PM4ADDA.B.

Sau khi hoàn tất quá trình nhân ma trận, chương trình ghi giá trị 1 vào biến `done_flag` tại địa chỉ 0x8001FFFC. Đây là địa chỉ được testbench sử dụng để theo dõi trạng thái kết thúc chương trình. Trong quá trình mô phỏng, khi testbench phát hiện tín hiệu `done = 1`, quá trình thực thi được xem là hoàn tất. Tại thời điểm này, testbench sẽ ghi nhận số chu kỳ thực thi và số lệnh đã hoàn tất, sau đó dừng chương trình mô phỏng.

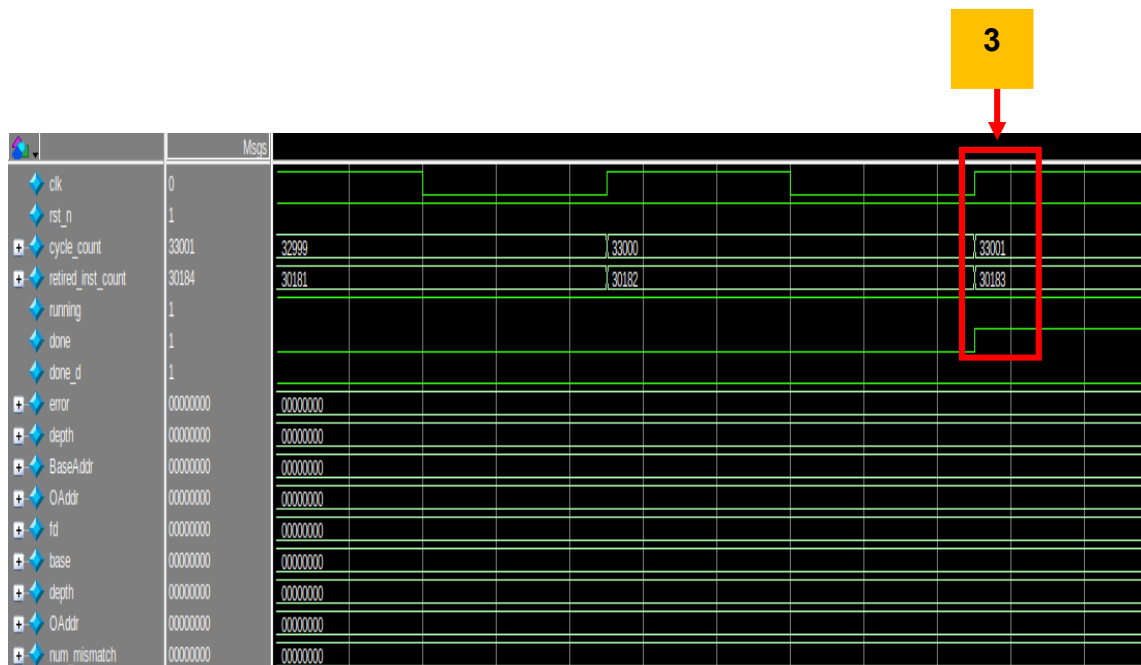
Hình 4.9, 4.10 và 4.11 mô tả kết quả mô phỏng của nhân ma trận trên ba phiên bản, ghi nhận số chu kì thực thi và số lệnh hoàn tất.



Hình 4.9: Dạng sóng mô phỏng của RV32IM trên ứng dụng nhân ma trận



Hình 4.10: Dạng sóng mô phỏng của RV32IMP trên ứng dụng nhân ma trận



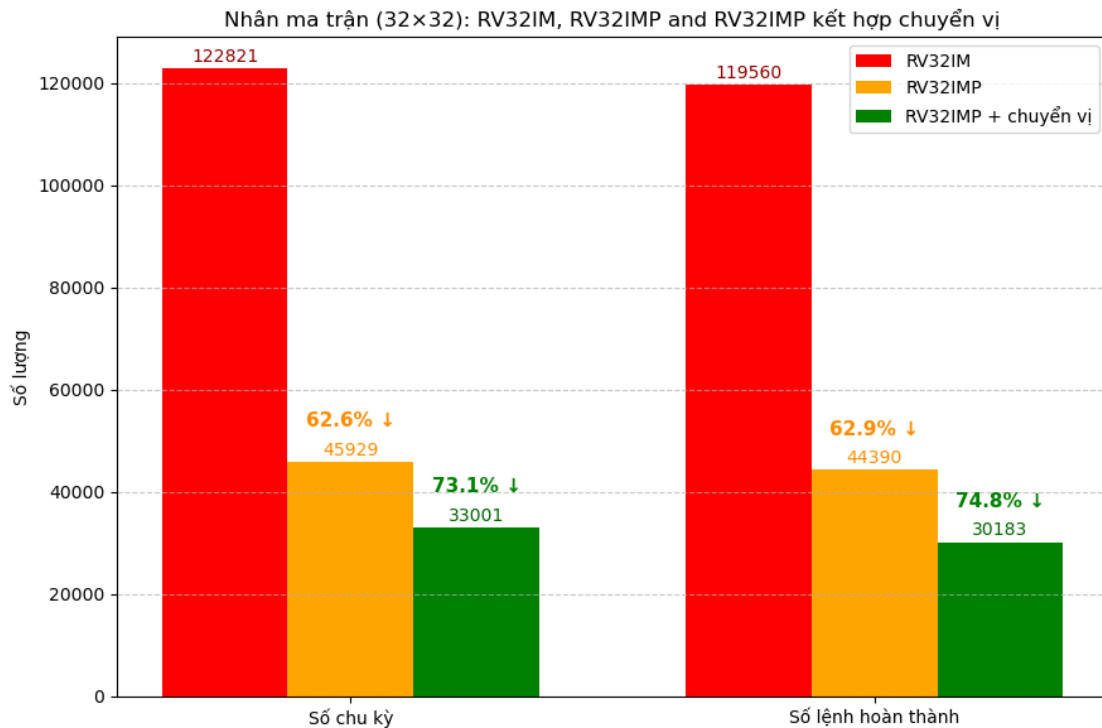
Hình 4.11: Dạng sóng mô phỏng của RV32IMP kết hợp chuyển vị ma trận trên ứng dụng nhân ma trận

- ❖ Tại điểm số 1: Khi tín hiệu done = 1 thì chương trình sẽ được kết thúc với chu kỳ thực thi và số lệnh hoàn tất trên RV32IM lần lượt là 122821 và 119560.
- ❖ Tại điểm số 2: Tương tự thì khi tín hiệu done = 1 thì chương trình cũng sẽ được kết thúc với chu kỳ thực thi và số lệnh hoàn tất trên RV32IMP lần lượt là 45929 và 44390.
- ❖ Tại điểm số 3: Tương tự như trên thì khi tín hiệu done = 1 thì sẽ ghi nhận được chu kỳ thực thi và số lệnh hoàn tất trên RV32IMP kết hợp chuyển vị ma trận lần lượt là 33001 và 30183.

Sau khi chạy trên cả ba phiên bản, kết quả output từ mỗi phiên bản được so sánh với golden output do Python tạo ra. Việc so sánh này giúp xác nhận rằng cả hai phiên bản đều tính toán chính xác và không có sai lệch, đồng thời làm cơ sở để đánh giá hiệu năng thực sự của SIMD so với phiên bản baseline.

Hình 4.12 so sánh hiệu năng nhân ma trận 32×32 giữa phiên bản baseline và SIMD. Các cột màu đỏ đại diện cho phiên bản baseline là RV32IM, cột màu cam đại diện cho phiên bản SIMD là RV32IMP còn màu xanh đại diện cho phiên

bản SIMD kết hợp chuyển vị, thể hiện sự giảm đáng kể số chu kỳ và số lệnh hoàn tất khi sử dụng SIMD.



Hình 4.12: So sánh hiệu năng phép nhân ma trận giữa RV32IM, RV32IMP và RV32IMP kết hợp chuyển vị ma trận

Kết quả đánh giá cho thấy cả hai phương án có SIMD đều cải thiện rõ rệt hiệu năng so với phiên bản baseline. Với phương án SIMD ghép byte trong quá trình tính toán, tổng số chu kỳ thực thi giảm 62,6% và số lệnh hoàn tất giảm 62,9%. Còn phương án SIMD kết hợp chuyển vị ma trận B đạt hiệu quả cao hơn với tổng số chu kỳ thực thi giảm 73,1% và số lệnh hoàn tất giảm 74,8%.

Nguyên nhân là sau khi chuyển vị, các phần tử thuộc cùng một cột của ma trận B được sắp xếp liên tiếp trong bộ nhớ. Nhờ đó, chương trình có thể đọc trực tiếp dữ liệu theo từng word 32 bit và đưa vào lệnh SIMD mà không cần lặp lại các phép dịch bit, mask và OR để ghép byte trong vòng lặp tính toán. Mặc dù bước chuyển vị làm phát sinh thêm chi phí xử lý ban đầu, thao tác này chỉ được thực hiện một lần nên hiệu năng tổng thể vẫn tốt hơn.

4.2.2. Bộ lọc FIR

Bộ lọc FIR được sử dụng trong đề tài như một bài toán đại diện cho nhóm ứng dụng xử lý tín hiệu số có tính lặp cao. Về mặt toán học, FIR có thể được xem như một cửa sổ trượt áp dụng trên tín hiệu đầu vào, trong đó mỗi mẫu đầu ra $y(n)$ được tính bằng tổng các tích của các mẫu đầu vào với các hệ số FIR tương ứng.

$$y(n) = \sum_{k=0}^{N-1} h(k) \cdot x(n-k) \quad (4.2)$$

Trong đó $x(n)$ là tín hiệu đầu vào, $h(k)$ là hệ số của bộ lọc, N là số tap và $y(n)$ là mẫu tín hiệu đầu ra.

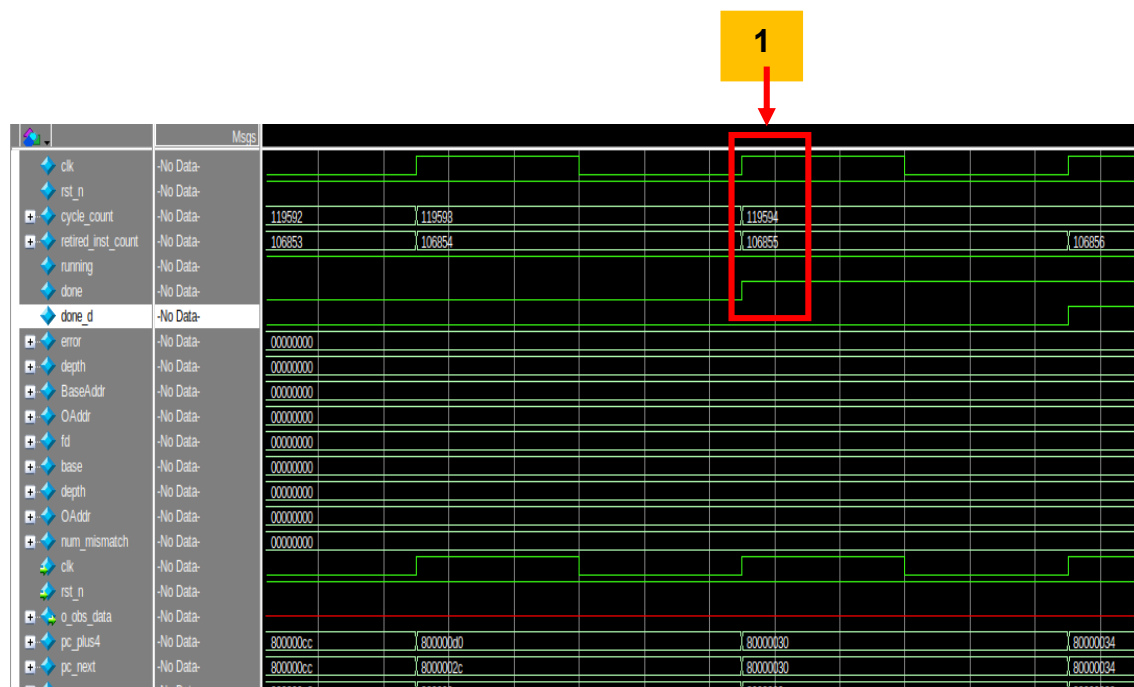
Ở phiên bản baseline, bộ lọc FIR được thực hiện theo phương pháp tuần tự. Với mỗi mẫu đầu ra, chương trình duyệt qua từng hệ số lọc và từng mẫu đầu vào tương ứng, sau đó thực hiện từng phép nhân và cộng với từng phần tử. Đối với các mẫu đầu chưa đủ số lượng taps, chương trình áp dụng zero-padding, tức các mẫu có chỉ số âm được xem bằng 0.

Còn với phiên bản SIMD, phần zero-padding ở các mẫu đầu vẫn được xử lý bằng phiên bản baseline do tại giai đoạn này chưa đủ số mẫu hợp lệ để khai thác hiệu quả lệnh SIMD. Khi đã đủ số mẫu hợp lệ, chương trình sử dụng lệnh PM4ADDA.B để xử lý đồng thời bốn cặp mẫu đầu vào và hệ số FIR trong một lệnh [11]. Do đó, phiên bản SIMD giúp giảm số vòng lặp và số lệnh so với baseline, từ đó cải thiện hiệu năng thực thi.

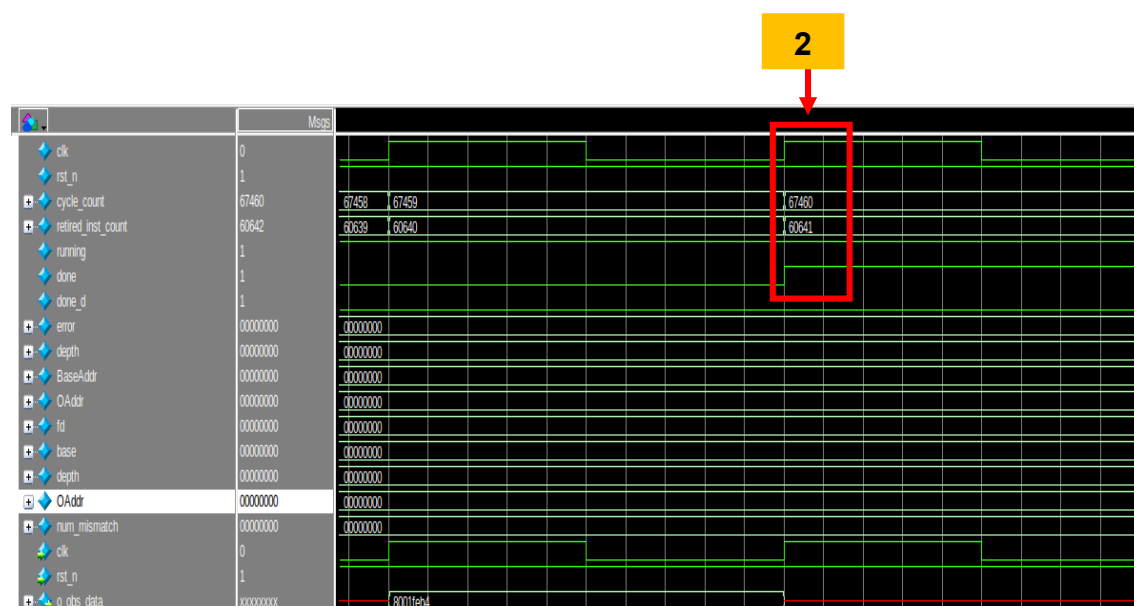
Tín hiệu ngõ vào được tạo ra bằng Python bao gồm thành phần tần số thấp $f_{\text{tín hiệu}} = 10 \text{ Hz}$, nhiều tần số cao $f_{\text{nhiều}} = 100 \text{ Hz}$ và một lượng nhiễu trắng ngẫu nhiên. Hệ số của bộ lọc sẽ được tạo ra bằng hàm firwin() của Python với các thông số cụ thể của bộ lọc bao gồm độ dài bộ lọc là 32, tần số cắt là 15 Hz, tần số lấy mẫu là 400 Hz và cửa sổ là Hamming. Tín hiệu đầu vào và hệ số bộ lọc đều được chuẩn hóa trong khoảng 8 bit từ $[-128, 127]$ để đảm bảo phù hợp với yêu cầu của phần cứng và lệnh PM4ADDA.B. Tần số lấy mẫu là 400 Hz tuân theo định lý lấy mẫu của Nyquist tức là $f_s \geq 2f_{\text{max}} = 200 \text{ Hz}$.

Tương tự như chương trình nhân ma trận, sau khi hoàn tất quá trình xử lý, chương trình cũng sẽ ghi giá trị 1 vào biến done_flag tại địa chỉ 0x8001FFFC. Khi testbench phát hiện done = 1, quá trình mô phỏng được xem là hoàn tất và đồng thời số chu kỳ thực thi và số lệnh hoàn tất sẽ được ghi nhận.

Hình 4.13 và 4.14 mô tả kết quả mô phỏng của bộ lọc FIR trên hai phiên bản, ghi nhận số chu kỳ thực thi và số lệnh hoàn tất.



Hình 4.13: Dạng sóng mô phỏng của RV32IM trên ứng dụng bộ lọc FIR

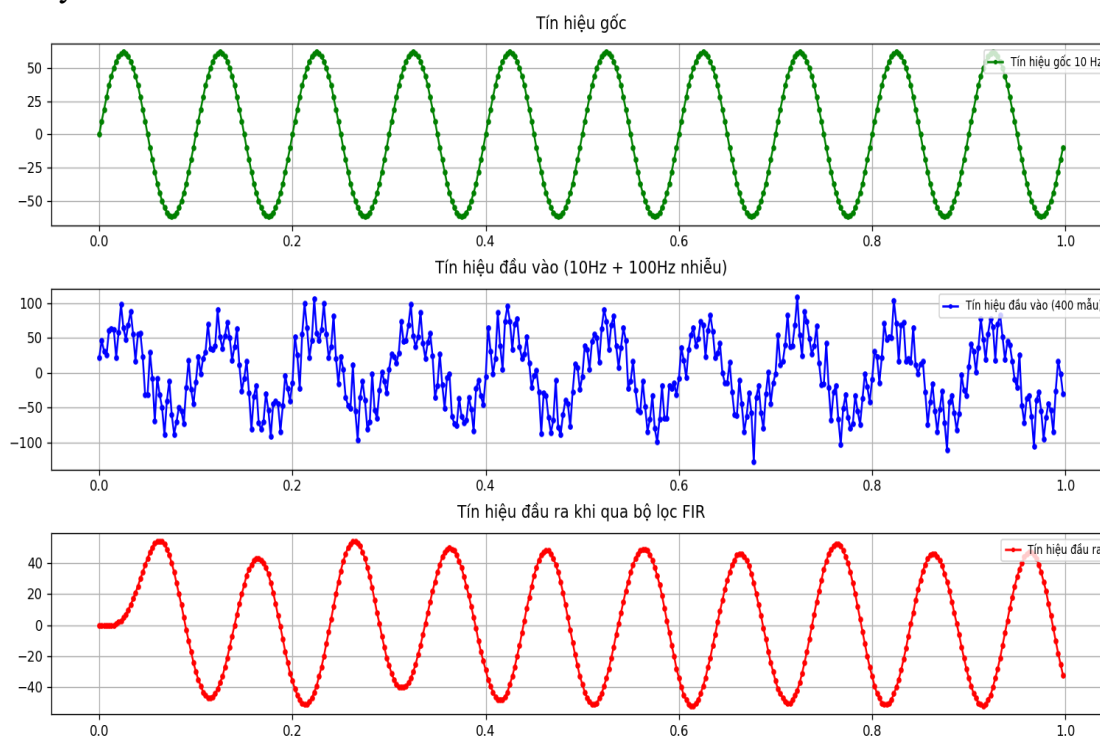


Hình 4.14: Dạng sóng mô phỏng của RV32IMP trên ứng dụng bộ lọc FIR

- ❖ Tại điểm số 1: Khi tín hiệu done = 1 thì chương trình sẽ được hoàn thành với chu kì thực thi và số lệnh hoàn tất trên RV32IM lần lượt là 119594 và 106855.
- ❖ Tại điểm số 2: Tương tự thì khi tín hiệu done = 1 thì sẽ ghi nhận được chu kì thực thi và số lệnh hoàn tất trên RV32IMP lần lượt là 67460 và 60641.

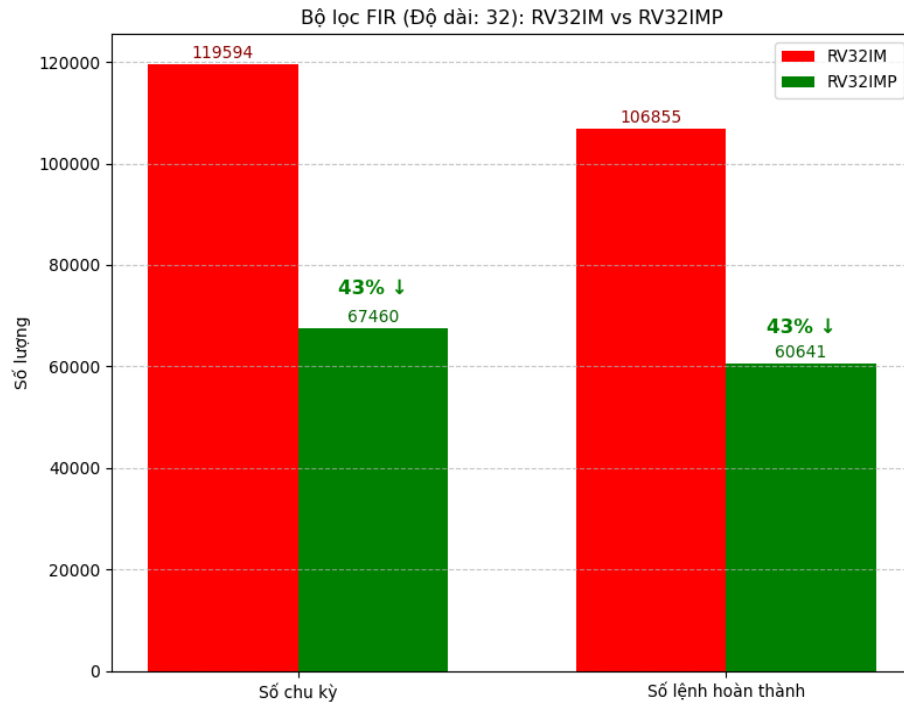
Hình 4.15 mô tả kết quả của bộ lọc FIR áp dụng lên tín hiệu đầu vào tổng hợp. Phần màu xanh lá là tín hiệu đầu vào gốc còn phần xanh dương là tín hiệu có nhiễu bao gồm thành phần tần số thấp 10 Hz cùng nhiễu 100 Hz và nhiễu trắng. Phần màu đỏ là tín hiệu sau khi lọc FIR, chỉ giữ lại thành phần tần số thấp mong muốn và loại bỏ hiệu quả các nhiễu tần số cao, thể hiện rõ khả năng lọc và độ trễ nhóm đặc trưng của FIR tuyến tính pha.

Độ trễ này được tính bằng $(\text{taps} - 1)/2$ với taps số hệ số của bộ lọc và độ trễ ở đây là 15.5 mẫu.



Hình 4.15: Tín hiệu gốc, đầu vào có nhiễu và đầu ra khi qua bộ lọc FIR

Hình 4.16 so sánh hiệu năng bộ lọc FIR 32-tap giữa phiên bản baseline và SIMD. Các cột màu đỏ đại diện cho phiên bản baseline là RV32IM còn màu xanh đại diện cho phiên bản SIMD là RV32IMP, thể hiện sự giảm đáng kể số chu kỳ và số lệnh hoàn tất khi sử dụng SIMD.



Hình 4.16: So sánh hiệu năng bộ lọc FIR giữa RV32IM và RV32IMP

Kết quả đánh giá cho thấy khi sử dụng SIMD, tổng số chu kỳ thực thi và số lệnh hoàn tất của chương trình bộ lọc FIR đều giảm so với phiên bản baseline. Tuy nhiên, mức cải thiện không cao bằng nhân ma trận.

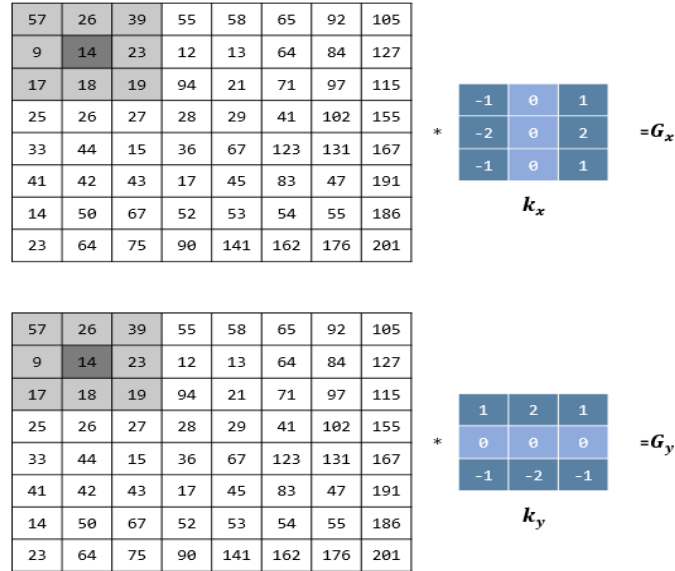
Dù các dữ liệu đầu vào được nạp liên tục trong bộ nhớ theo đúng định dạng bốn dữ liệu trong một ô nhớ. Tuy nhiên phần lớn các phép toán đều cần các dữ liệu nằm ở hai word liền kề, do đó cần phải dịch và ghép các byte để tạo dữ liệu 32 bit mong muốn trước khi thực hiện các phép nhân cộng dồn. Quá trình này tạo ra lượng tính toán bổ sung, khiến hiệu năng SIMD trong bộ lọc FIR chỉ giảm khoảng 43% so với baseline, thấp hơn so với mức cải thiện 73–74% quan sát được trong nhân ma trận.

4.2.3. Thuật toán phát hiện biên Sobel

Thuật toán phát hiện biên Sobel được sử dụng như một bài toán đại diện cho nhóm ứng dụng xử lý ảnh. Thuật toán này xác định biên ảnh dựa trên sự thay đổi cường độ sáng giữa các điểm ảnh lân cận. Trong thuật toán Sobel, hai thành phần gradient theo phương ngang và phương dọc lần lượt được ký hiệu là G_x và G_y . Độ lớn tại mỗi điểm ảnh được xác định theo công thức:

$$G = \sqrt{G_x^2 + G_y^2} \quad (4.3)$$

Hình 4.17 minh họa cách áp dụng kernel Sobel theo hướng ngang k_x và dọc k_y lên một ma trận dữ liệu 8×8 . Phép tích chập giữa từng kernel với vùng ảnh cục bộ 3×3 tạo ra các giá trị gradient G_x và G_y , thể hiện cường độ thay đổi theo từng hướng.



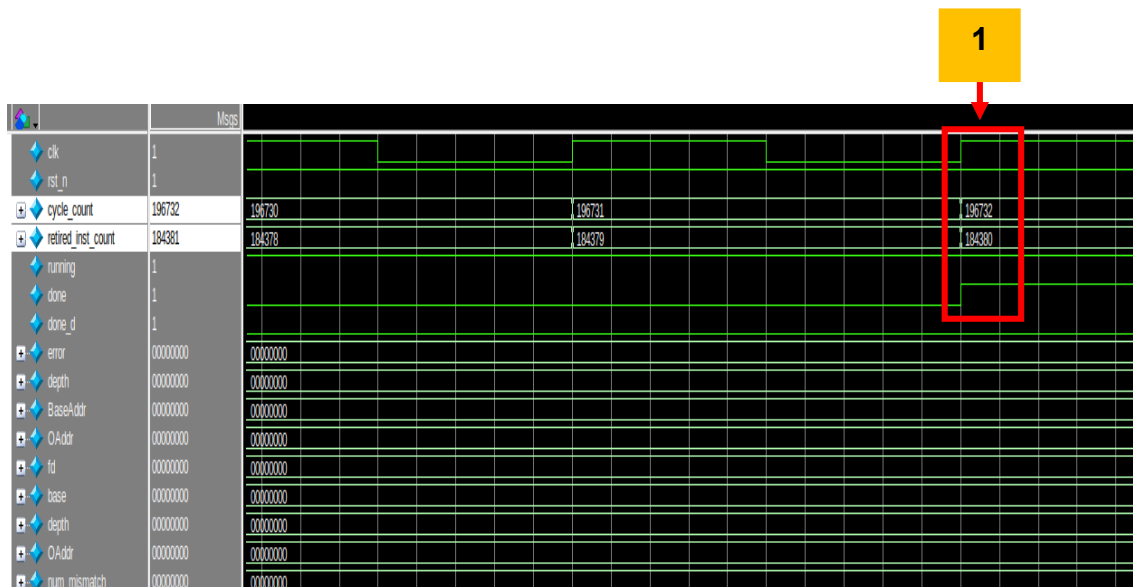
Hình 4.17: Tính gradient theo hướng ngang và dọc trong thuật toán Sobel

Tuy nhiên, phép căn bậc hai sẽ làm tăng chi phí tính toán khi triển khai trên bộ xử lý nhúng. Để giảm chi phí tính toán thì độ lớn gradient được xấp xỉ bằng tổng trị tuyệt đối của hai thành phần gradient, tức là $G \approx |G_x| + |G_y|$. Đối với phiên bản baseline, các phép nhân và cộng được thực hiện tuần tự trên từng cặp giá trị điểm ảnh và hệ số kernel.

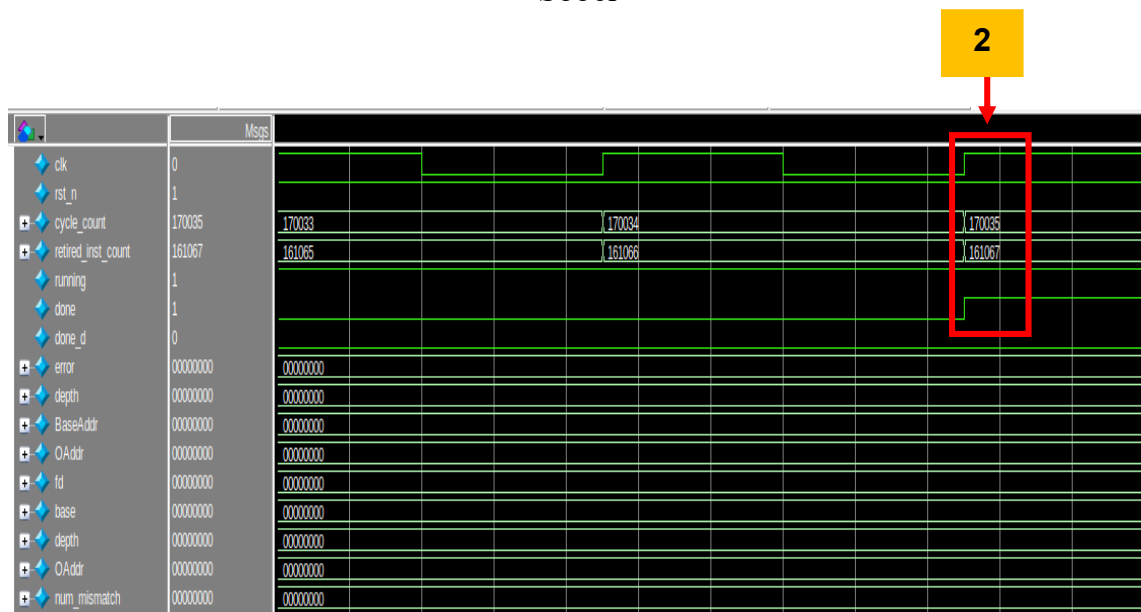
Trong khi đó với SIMD, các hệ số của kernel Sobel được sắp xếp thành từng nhóm bốn phần tử trong thanh ghi 32 bit. Do mỗi hàng của kernel chỉ có ba hệ số, phần tử thứ tư được gán bằng 0. Khi xử lý một điểm ảnh, dữ liệu từ ba hàng lân cận của ảnh được đưa vào tính toán. Lệnh PM4ADDASU.B thực hiện nhân cộng dồn giữa các điểm ảnh và hệ số Sobel tương ứng tạo ra hai giá trị G_x và G_y , sau đó lệnh PSABS.H tính giá trị tuyệt đối theo từng lane còn PADD.H thực hiện cộng hai thành phần gradient và lệnh PUSATI.H giới hạn kết quả trong phạm vi 0 – 255 để đảm bảo giá trị pixel hợp lệ [11].

Tương tự như hai ứng dụng trước đó thì sau khi hoàn tất được cái quá trình xử lý, chương trình cũng sẽ ghi giá trị 1 vào biến done_flag tại địa chỉ 0x8001FFFC. Khi testbench phát hiện done = 1, quá trình mô phỏng được xem là hoàn tất và đồng thời số chu kỳ thực thi và số lệnh hoàn tất sẽ được ghi nhận.

Hình 4.18 và 4.19 mô tả kết quả mô phỏng của thuật toán phát hiện biên Sobel trên hai phiên bản, ghi nhận số chu kỳ thực thi và số lệnh hoàn tất.



Hình 4.18: Dạng sóng mô phỏng của RV32IM trên ứng dụng phát hiện biên Sobel



Hình 4.19: Dạng sóng mô phỏng của RV32IMP trên ứng dụng phát hiện biên Sobel

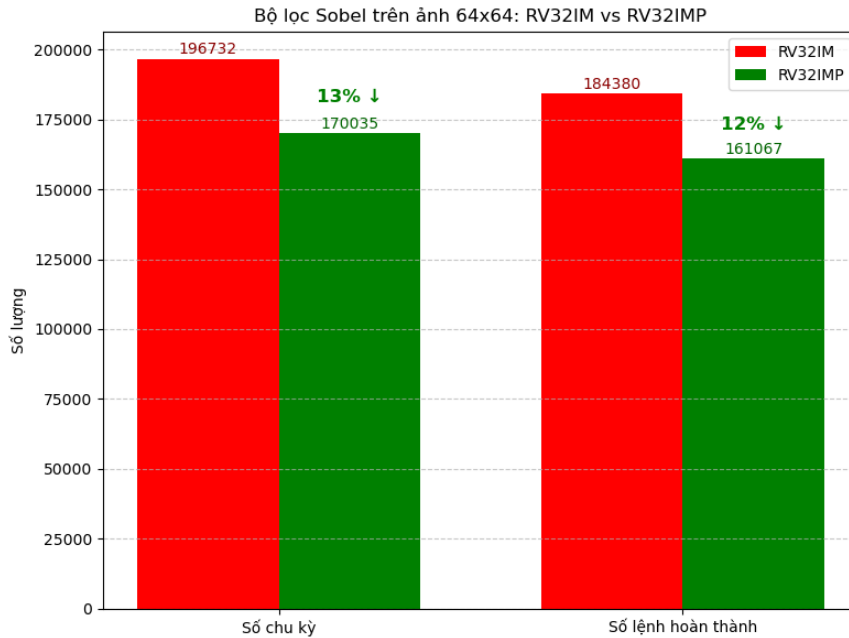
- ❖ Tại điểm số 1: Khi tín hiệu done = 1 thì chương trình sẽ ghi nhận số chu kì thực thi và số lệnh hoàn tất trên RV32IM lần lượt là 196732 và 184380.
- ❖ Tại điểm số 2: Tương tự thì khi tín hiệu done = 1 thì chương trình sẽ được kết thúc với chu kì thực thi và số lệnh hoàn tất trên RV32IMP lần lượt là 170035 và 161067.

Hình 4.20 mô tả kết quả phát hiện biên Sobel với hình bên trái là ảnh xám gốc, còn hình bên phải là ảnh sau khi áp dụng bộ lọc Sobel. Các biên và cạnh nổi bật được làm rõ, giúp nhận diện các vùng có sự thay đổi mạnh về cường độ sáng trong ảnh.



Hình 4.20: Ảnh trước và sau khi áp dụng bộ lọc Sobel

Hình 4.21 so sánh hiệu năng thuật toán phát hiện biên Sobel trên ảnh 64x64 giữa phiên bản baseline và SIMD. Các cột màu đỏ đại diện cho phiên bản baseline là RV32IM còn màu xanh đại diện cho phiên bản SIMD là RV32IMP, thể hiện sự giảm về số chu kỳ và số lệnh hoàn tất khi sử dụng SIMD.



Hình 4.21: So sánh hiệu năng thuật toán phát hiện biên Sobel giữa RV32IM và RV32IMP

Kết quả đánh giá cho thấy khi sử dụng SIMD, tổng số chu kỳ thực thi và số lệnh hoàn tất của chương trình Sobel giảm so với phiên bản baseline. Tuy nhiên, mức cải thiện không đáng kể thậm chí khá ít. Nguyên nhân chính là do bộ lọc Sobel 3×3 cần thu thập pixel từ nhiều hàng, do đó cần thực hiện thêm các thao tác chuẩn bị dữ liệu như nạp pixel, dịch bit và ghép dữ liệu trước khi thực hiện các phép nhân cộng dồn. Quá trình này tạo ra một lượng tính toán bổ sung, làm hạn chế hiệu năng tối đa mà SIMD có thể đạt được.

Sau khi tính xong giá trị G_x và G_y , phần hậu xử lý của phiên bản SIMD gần như không giảm số lệnh so với baseline. Ở baseline, trị tuyệt đối được tính bằng chuỗi lệnh `srai`, `xor` và `sub`, sau đó cộng biên độ, so sánh để giới hạn giá trị. Trong khi đó, phiên bản SIMD sử dụng các lệnh P-extension như `PSABS.H`, `PADD.H` và `PUSATI.H` để thực hiện lấy trị tuyệt đối, cộng biên độ và giới hạn giá trị đầu ra. Tuy nhiên, trước khi dùng được các lệnh này, chương trình vẫn cần thêm một số lệnh để đóng gói G_x và G_y vào cùng một thanh ghi. Do đó, số lệnh của phần hậu xử lý sau Sobel của SIMD gần như không giảm so với baseline và làm ảnh hưởng đến hiệu suất của lệnh MAC trước đó. Do đó, trường hợp này SIMD

chỉ giảm 13% đối với số chu kỳ thực thi và giảm 12% với số lệnh hoàn tất so với baseline.

4.3. ĐÁNH GIÁ TÀI NGUYÊN, TẦN SỐ HOẠT ĐỘNG, CÔNG SUẤT VÀ NĂNG LƯỢNG GIỮA RV32IM VÀ RV32IMP

4.3.1. Tài nguyên sử dụng

Bảng 4.3 mô tả tổng hợp các thông số tài nguyên phần cứng hai kiến trúc qua bộ lọc FIR sau quá trình tổng hợp bằng Quartus II. Các thông số được xem xét gồm Total Logic Elements, LUTs, FFs và số DSP Blocks.

Bảng 4.3: So sánh tài nguyên phần cứng của hai kiến trúc RV32IM và RV32IMP trên ứng dụng bộ lọc FIR

Thông số	RV32IM	RV32IMP	Chi phí phát sinh
Total Logic Elements	3934	6391	+2457 (+62.46%)
LUTs / Combinational Logic	3542	5838	+2296 (64.82%)
FFs / Registers	1510	1544	+34 FFs (+2.25%)
DSP Blocks	15	17	+2 DSPs (+13.33%)

Kết quả tổng hợp trên Quartus II cho thấy, việc tích hợp tập lệnh mở rộng P-extension làm gia tăng đáng kể mức độ tiêu thụ tài nguyên phần cứng với tổng số Logic Elements tăng 62.46%. Bóc tách chi tiết dữ liệu, ta thấy sự gia tăng này có tính chất phân hóa cực kỳ rõ nét.

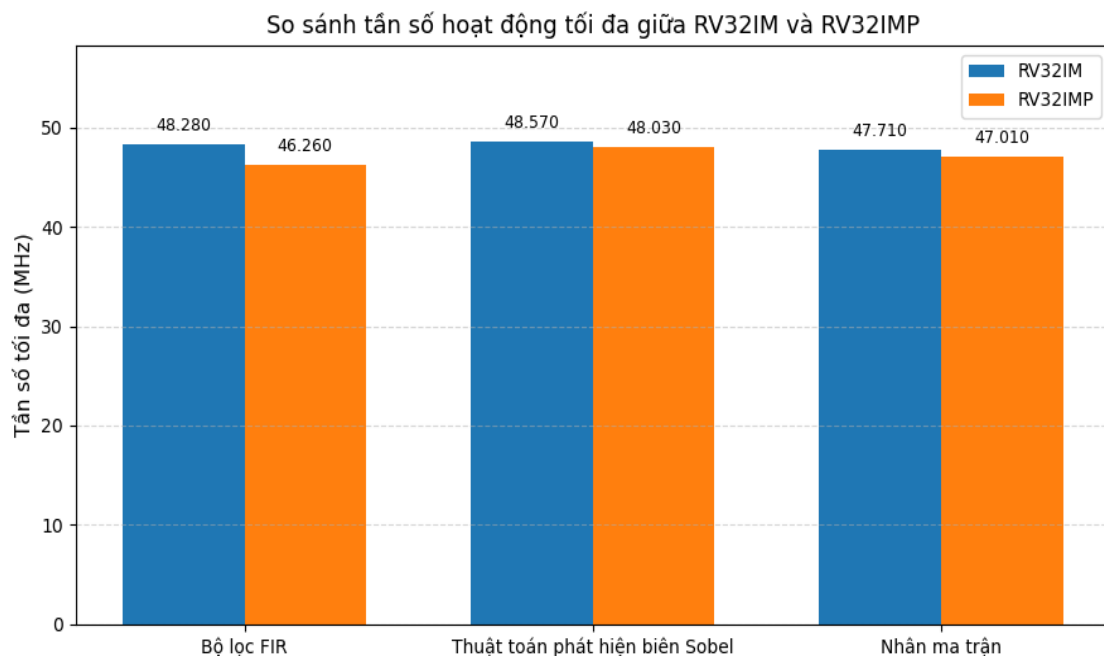
Phần lớn tài nguyên tăng thêm tập trung vào Combinational Logic. Cụ thể, số lượng LUTs tăng mạnh từ 3542 lên 5838 tức là 64.82%. Sự phình to này là điều tất yếu, bởi kiến trúc RV32IMP đòi hỏi việc thiết kế thêm bộ SIMD Pext phức tạp nhằm thực thi các lệnh tính toán song song, đồng thời cần thêm logic nhận diện, giải mã các trường lệnh và tạo tín hiệu điều khiển cho nhóm lệnh P-extension. Bên cạnh đó, hệ thống cũng tiêu thụ thêm 2 khối DSP do bổ xung khối xử lý liên quan đến lệnh nhân cộng dồn.

Ngược lại, phần Registers/FFs gần như không bị ảnh hưởng, chỉ tăng xấp xỉ 2.25%. Điều này chứng tỏ thiết kế lõi RV32IMP đã tối ưu rất tốt phần lưu trữ

trạng thái. Nó vẫn tận dụng lại toàn bộ cấu trúc 32 thanh ghi đa dụng và các tầng pipeline của RV32IM gốc mà không gây lãng phí tài nguyên bộ nhớ.

4.3.2. Tần số hoạt động

Tần số hoạt động tối đa của hai kiến trúc cũng được đánh giá trên ba ứng dụng gồm bộ lọc FIR, thuật toán phát hiện biên Sobel và nhân ma trận. Hình 4.22 mô tả kết quả so sánh tần số hoạt động tối đa giữa kiến trúc RV32IM và RV32IMP trên cả ba ứng dụng.



Hình 4.22: So sánh tần số hoạt động tối đa trên hai kiến trúc RV32IM và RV32IMP

Kết quả cho thấy sau khi tích hợp P-extension, tần số hoạt động tối đa của kiến trúc RV32IMP giảm nhẹ so với RV32IM ở cả ba ứng dụng. Nguyên nhân của sự suy giảm này là do việc bổ sung P-extension làm tăng độ phức tạp của đường dữ liệu, đặc biệt là các khối tính toán SIMD. Điều này có thể làm tăng độ trễ đường tới hạn, khiến tần số hoạt động tối đa của thiết kế giảm nhẹ. Tuy nhiên, mức giảm nhìn chung không đáng kể, trong khi P-extension giúp giảm đáng kể số chu kỳ thực thi ở các ứng dụng có khả năng khai thác song song dữ liệu.

4.3.3. Công suất tiêu thụ

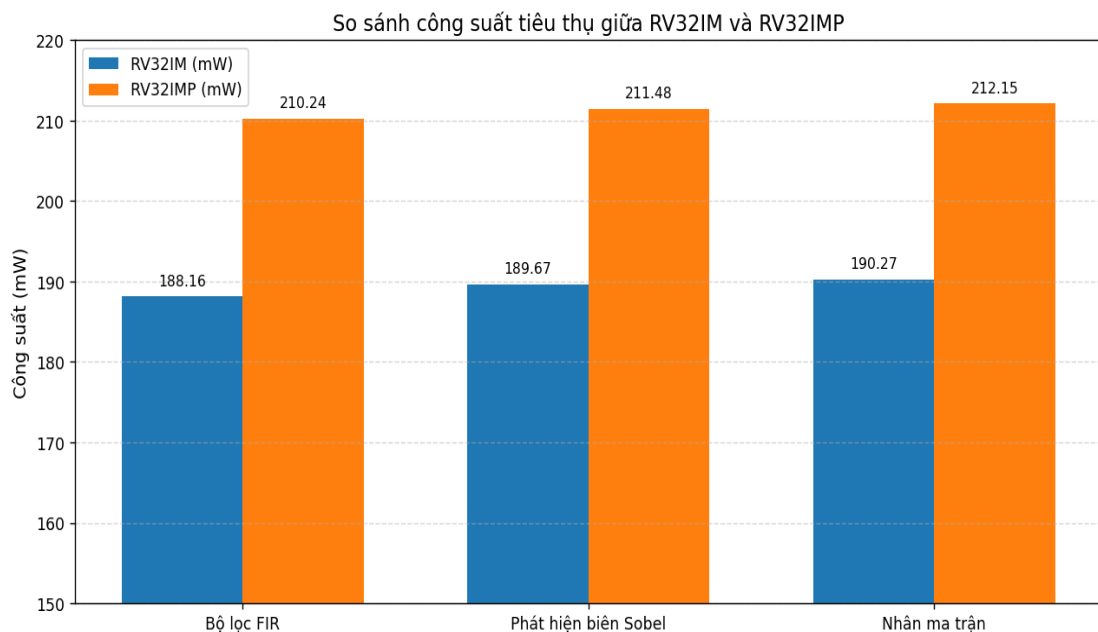
Công suất tiêu thụ của từng kiến trúc được đánh giá bằng công cụ Power Analyzer trong Quartus khi thực thi ba ứng dụng gồm bộ lọc FIR, thuật toán phát hiện biên Sobel và phép nhân ma trận.

Bảng 4.4 mô tả công suất tiêu thụ của kiến trúc RV32IM và kiến trúc RV32IMP khi thực thi ba ứng dụng.

Bảng 4.4: So sánh công suất tiêu thụ của kiến trúc RV32IM và RV32IMP

Ứng dụng	RV32IM	RV32IMP	Chênh lệch
Bộ lọc FIR	188.16 mW	210.24 mW	+22.08 mW (+11.73%)
Thuật toán phát hiện biên Sobel	189.67 mW	211.48 mW	+21.81 mW (+11.5%)
Nhân ma trận	190.27 mW	212.15 mW	+21.88 mW (+11.5%)

Để trực quan hóa mức độ chênh lệch cũng như so sánh tương quan giữa các thuật toán. Hình 4.23 minh họa chi tiết biểu đồ công suất tiêu thụ của cả hai thiết kế.



Hình 4.23: So sánh công suất tiêu thụ giữa RV32IM và RV32IMP

Kết quả cho thấy công suất tiêu thụ của kiến trúc tích hợp P-extension đều tăng trong cả ba trường hợp, với mức tăng nằm trong khoảng từ 11% đến 12%. Nguyên nhân là P-extension bổ sung các khối xử lý song song nhằm thực hiện nhiều phép toán trên dữ liệu 8 bit trong cùng một chu kỳ. Việc sử dụng thêm tài nguyên phần cứng làm công suất tức thời tăng lên.

4.3.4. Năng lượng tiêu thụ

Để có thể đánh giá hiệu quả của kiến trúc mở rộng, cần xem xét thêm năng lượng tiêu thụ dựa trên công suất và thời gian thực thi. Trên cơ sở công suất tiêu thụ và thời gian thực thi, năng lượng tiêu thụ được xác định theo công thức:

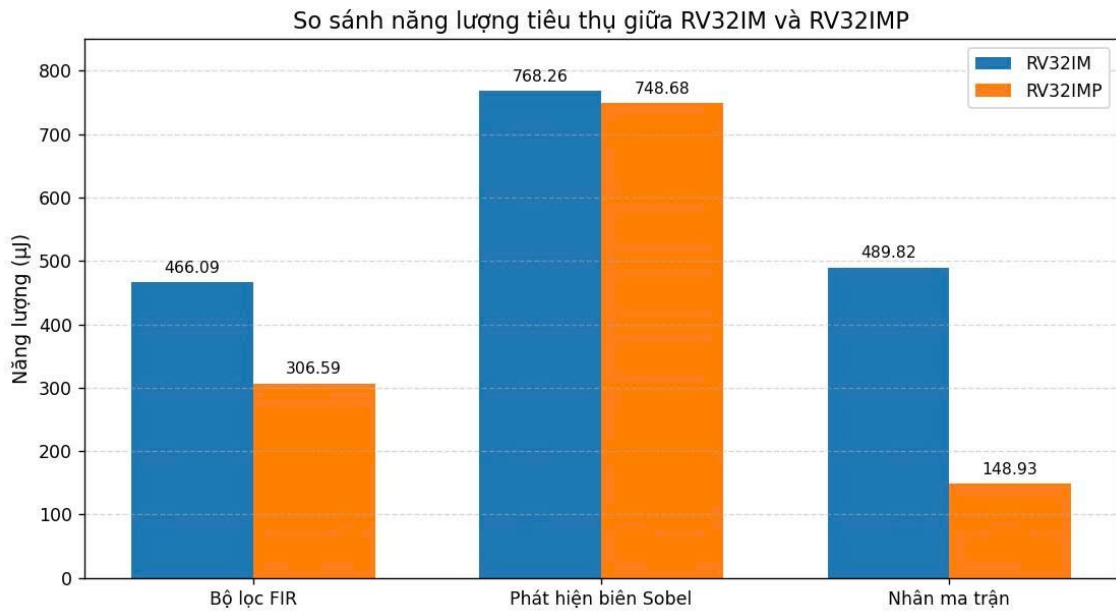
$$E = P \times t = P \times \frac{N_{cycles}}{F_{max}} \quad (4.4)$$

Trong đó, P là công suất, N_{cycles} là số chu kỳ thực thi và F_{max} là tần số hoạt động tối đa. Bảng 4.5 mô tả năng lượng tiêu thụ của kiến trúc RV32IM và kiến trúc RV32IMP trên ba ứng dụng.

Bảng 4.5: So sánh năng lượng tiêu thụ của kiến trúc RV32IM và RV32IMP

Ứng dụng	RV32IM	RV32IMP	Giảm
Bộ lọc FIR	466.09 μ J	306.59 μ J	34.22%
Thuật toán phát hiện biên Sobel	768.26 μ J	748.68 μ J	2.55%
Nhân ma trận	489.82 μ J	148.93 μ J	69.59%

Nhằm làm nổi bật mức độ tiết kiệm năng lượng của kiến trúc RV32IMP. Hình 4.24 dưới đây sẽ minh họa trực quan sự sụt giảm tài nguyên tiêu hao giữa phiên bản RV32IM và phiên bản RV32IMP.



Hình 4.24: So sánh năng lượng tiêu thụ giữa RV32IM và RV32IMP

Kết quả cho thấy mặc dù công suất tiêu thụ tăng 11–12%, thời gian thực thi giảm mạnh nhờ SIMD khiến tổng năng lượng giảm rõ rệt ở bộ lọc FIR 34.22% và nhân ma trận 69.59%. Riêng thuật toán Sobel, năng lượng giảm nhẹ 2.55% do thời gian thực thi giảm không nhiều. Nhìn chung, P-extension mang lại hiệu quả năng lượng rõ rệt cho các bài toán có mật độ MAC cao và dữ liệu song song tốt, nhưng bị giới hạn khi phải xử lý dữ liệu cho các phép toán SIMD.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. KẾT LUẬN

Đối chiếu với các mục tiêu đã đề ra ở Chương 1, đề tài đã hoàn thành tốt những công việc sau:

- Nghiên cứu và làm rõ kiến trúc RISC-V RV32IM, mô hình pipeline 5 tầng cũng như các đặc điểm của tập lệnh mở rộng P-extension.
- Thiết kế hoàn chỉnh ở mức RTL lõi vi xử lý RV32IM cơ sở và tích hợp thành công khối xử lý SIMD Pext vào hệ thống.
- Mô phỏng và kiểm chứng chính xác hoạt động của bộ vi xử lý thông qua phần mềm QuestaSim.
- Triển khai và đánh giá thành công hiệu năng vi xử lý thông qua ba bài toán thực tế: bộ lọc FIR, thuật toán phát hiện biên Sobel và phép nhân ma trận.
- Tổng hợp phần cứng thành công trên công cụ Quartus, làm cơ sở để đánh giá tài nguyên sử dụng và công suất tiêu thụ.

Từ các kết quả phân tích, đặc điểm của kiến trúc RV32IMP được đúc kết như sau:

- Tài nguyên, công suất và tần số hoạt động tối đa: Việc thêm khối SIMD Pext làm tăng diện tích mạch, dẫn đến tốn nhiều tài nguyên logic và công suất động/tĩnh hơn lõi cơ sở. Đồng thời, độ trễ của RV32IMP tăng khiến tần số hoạt động tối đa giảm nhẹ.
- Hiệu năng & Năng lượng tiêu thụ: Bù lại, P-extension mang đến mức tăng tốc mạnh mẽ cho các bài toán song song dữ liệu như bộ lọc FIR và nhân ma trận. Nhờ giảm mạnh số vòng lặp và số lệnh thực thi, năng lượng tiêu thụ tổng thể cho bài toán được tiết kiệm đáng kể. Mặc dù mức cải thiện của thuật toán phát hiện biên Sobel chưa cao như bộ lọc FIR và nhân ma trận nhưng kết quả này vẫn cho thấy P-extension đem lại

hiệu quả cải thiện nhất định về hiệu năng và năng lượng tiêu thụ đối với ứng dụng này.

Đề tài đã hiện thực hóa và đánh giá việc tích hợp P-extension vào kiến trúc RISC-V RV32IM. Dù có sự đánh đổi về tài nguyên phần cứng, kiến trúc RV32IMP vẫn cho thấy tiềm năng trong việc tăng tốc các bài toán có tính lặp và xử lý song song dữ liệu, tiêu biểu như xử lý tín hiệu số với bộ lọc FIR, xử lý ảnh với bộ lọc Sobel và phép nhân ma trận. Đây là các nhóm bài toán nền tảng trong xử lý tín hiệu số, xử lý ảnh và tính toán ma trận trên hệ thống nhúng hoặc thiết bị xử lý tại biên có yêu cầu tính toán cao nhưng khắt khe về bộ nhớ lệnh.

5.2. HƯỚNG PHÁT TRIỂN

Dựa trên các kết quả đạt được, nhóm nghiên cứu nhận thấy đề tài có thể tiếp tục được mở rộng và tối ưu hóa theo các hướng sau:

- Cải tiến Instruction Memory và Data Memory sang đọc đồng bộ theo cạnh xung clock để tương thích với các khối SRAM thực tế và ổn định hơn khi tổng hợp mạch.
- Phát triển hệ thống SoC bằng cách mở rộng giao tiếp của lõi RV32IMP thông qua các hệ thống bus tiêu chuẩn nhằm dễ dàng ghép nối với các ngoại vi. Đây là bước đệm cần thiết để đánh giá bộ xử lý RV32IMP trong các bài toán thực tế.

TÀI LIỆU THAM KHẢO

- [1] Girish M, Gaganashree M, Inchara H M, Likhitha B, and Moulya B N, "Design and Implementation of SIMD based RISC V Processor," *International Journal of Science, Engineering and Technology*, vol. 14, no. 1, pp. 1-5, 2026.
- [2] M. Arucu and T. Iliev, "Performance Evaluation of FPGA, GPU, and CPU in FIR Filter Implementation for Semiconductor-Based Systems," *J. Low Power Electron. Appl.*, vol. 15, no. 3, Art. no. 40, Jul. 2025.
- [3] S. L. Harris and D. Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Cambridge, MA: Morgan Kaufmann, 2022.
- [4] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture," Version 20250508, 2025.
- [5] A. Waterman and K. Asanović, Eds., "The RISC-V Instruction Set Manual, Volume I: User-Level ISA," Document Version 2.2, RISC-V Foundation, May 2017.
- [6] E. Cui, T. Li, and Q. Wei, "RISC-V Instruction Set Architecture Extensions: A survey," *IEEE Access*, vol. 11, pp. 24696-24711, 2023.
- [7] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition*. Cambridge, MA: Morgan Kaufmann, 2018.
- [8] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA: Morgan Kaufmann, 2019.
- [9] J.-H. Li, J.-K. Lin, Y.-C. Su, C.-W. Chu, L.-T. Kuok, H.-M. Lai, C.-L. Lee, and J.-K. Lee, "SIMD Everywhere Optimization from ARM NEON to RISC-V Vector Extensions," arXiv:2309.16509, Sep. 2023.
- [10] RISC-V International, *Preliminary in-progress RISC-V "P" Extension*, Version 0.20-draft, Mar. 25, 2026. [Online]. Available: <https://github.com/riscv/riscv-p-spec.git>. [Accessed: Apr. 1, 2026].
- [11] H. R. Sultanpuri, "Implementation of RISC-V P-extension ALU," M.S. thesis, School of Electrical Engineering, Aalto Univ., Espoo, Finland, 2025.

[12] A-Core, “P-extension ALU: pextalu.scala,” A-Core_Chisel/ACoreBase, GitLab. [Online]. Available: https://gitlab.com/a-core/a-core_chisel/ACoreBase/-/blob/dev/pextalu/src/main/scala/acorebase/alublock/pextalu.scala. [Accessed: Apr. 15, 2026].

PHỤ LỤC 1

CODE TOÀN BỘ ĐỒ ÁN

https://github.com/TrangThang-2k4/rv32imp_pipeline

PHỤ LỤC 2

VIDEO THỰC HIỆN CHƯƠNG TRÌNH

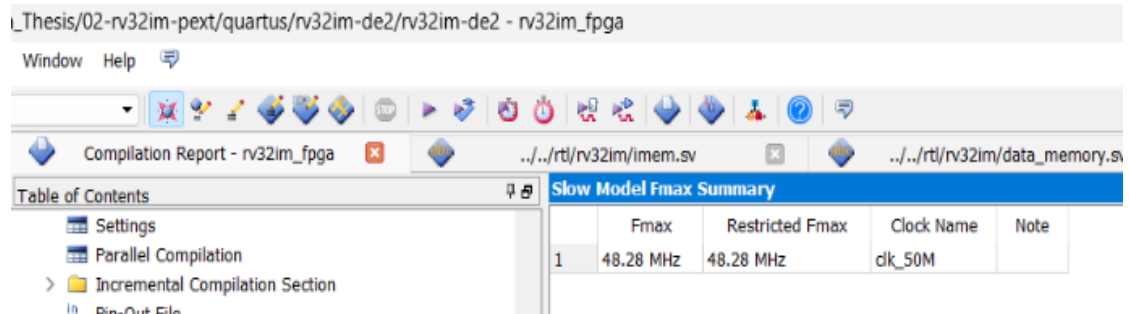
https://drive.google.com/drive/folders/18bsm07AwA6JqkvU-xPiDGhv5n4NAUuMC?usp=drive_link

PHỤ LỤC 3

MINH CHỨNG KẾT QUẢ

Tần số hoạt động tối đa trên bộ lọc FIR:

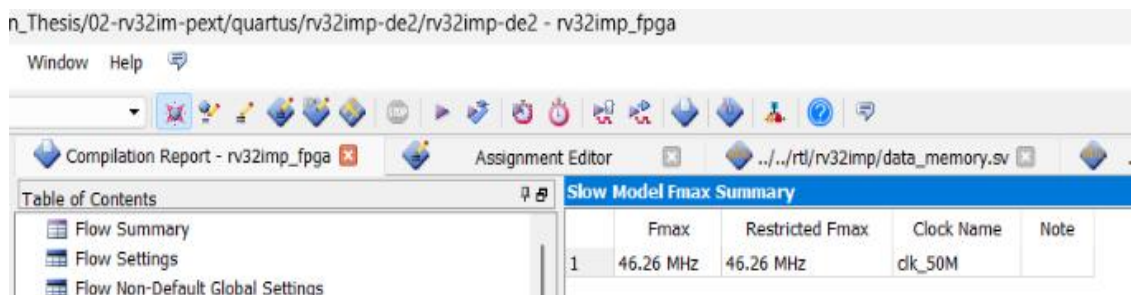
RV32IM



The screenshot shows the 'Slow Model Fmax Summary' table in the Quartus II compilation report for the RV32IM project. The table indicates a maximum frequency (Fmax) of 48.28 MHz, which is also the restricted maximum frequency, using a clock named 'clk_50M'.

	Fmax	Restricted Fmax	Clock Name	Note
1	48.28 MHz	48.28 MHz	clk_50M	

RV32IMP

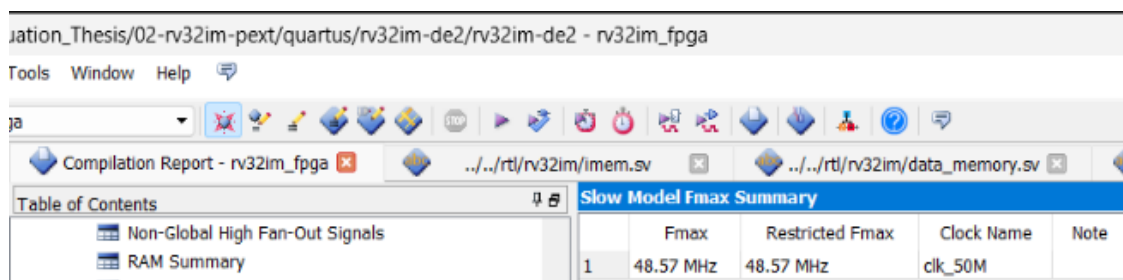


The screenshot shows the 'Slow Model Fmax Summary' table in the Quartus II compilation report for the RV32IMP project. The table indicates a maximum frequency (Fmax) of 46.26 MHz, which is also the restricted maximum frequency, using a clock named 'clk_50M'.

	Fmax	Restricted Fmax	Clock Name	Note
1	46.26 MHz	46.26 MHz	clk_50M	

Tần số hoạt động tối đa trên ứng dụng phát hiện cạnh Sobel:

RV32IM



The screenshot shows the 'Slow Model Fmax Summary' table in the Quartus II compilation report for the RV32IM project. The table indicates a maximum frequency (Fmax) of 48.57 MHz, which is also the restricted maximum frequency, using a clock named 'clk_50M'.

	Fmax	Restricted Fmax	Clock Name	Note
1	48.57 MHz	48.57 MHz	clk_50M	

RV32IMP

on_Thesis/02-rv32im-pext/quartus/rv32imp-de2/rv32imp-de2 - rv32imp_fpga

Window Help

Compilation Report - rv32imp_fpga Assignment Editor ../rti/rv32imp/data_memory.sv

Table of Contents

- Flow Summary
- Flow Settings
- Flow Map-Default Global Settings

Slow Model Fmax Summary

	Fmax	Restricted Fmax	Clock Name	Note
1	48.03 MHz	48.03 MHz	clk_50M	

Tần số hoạt động tối đa trên ứng dụng nhân ma trận:

RV32IM

luation_Thesis/02-rv32im-pext/quartus/rv32im-de2/rv32im-de2 - rv32im_fpga

Tools Window Help

rga

Compilation Report - rv32im_fpga ../rti/rv32im/imem.sv ../rti/rv32im/data_memory.sv

Table of Contents

- Flow Summary
- Flow Settings

Slow Model Fmax Summary

	Fmax	Restricted Fmax	Clock Name	Note
1	47.71 MHz	47.71 MHz	clk_50M	

RV32IMP

_Thesis/02-rv32im-pext/quartus/rv32imp-de2/rv32imp-de2 - rv32imp_fpga

Window Help

Compilation Report - rv32imp_fpga Assignment Editor ../rti/rv32imp/data_memory.sv

Table of Contents

- Flow Summary
- Flow Settings

Slow Model Fmax Summary

	Fmax	Restricted Fmax	Clock Name	Note
1	47.01 MHz	47.01 MHz	clk_50M	

Tổng hợp phân cứng trên ứng dụng bộ lọc FIR:

RV32IM

Flow Summary	
Flow Status	Successful - Sun Jun 21 13:29:11 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	rv32im_fpga
Top-level Entity Name	rv32im_fpga
Family	Cyclone II
Device	EP2C35F672C6
Timing Models	Final
Total logic elements	3,934 / 33,216 (12 %)
Total combinational functions	3,542 / 33,216 (11 %)
Dedicated logic registers	1,510 / 33,216 (5 %)
Total registers	1510
Total pins	12 / 475 (3 %)
Total virtual pins	0
Total memory bits	131,136 / 483,840 (27 %)
Embedded Multiplier 9-bit elements	30 / 70 (43 %)
Total PLLs	0 / 4 (0 %)

rv32im_fpga

Fitter DSP Block Usage Summary				
	Statistic	Number Used	Available per Block	Maximum Available
1	Simple Multipliers (9-bit)	0	2	70
2	Simple Multipliers (18-bit)	15	1	35
3	Embedded Multiplier Blocks	15	--	35
4	Embedded Multiplier 9-bit elements	30	2	70
5	Signed Embedded Multipliers	1	--	--
6	Unsigned Embedded Multipliers	12	--	--
7	Mixed Sign Embedded Multipliers	2	--	--
8	Variable Sign Embedded Multipliers	0	--	--
9	Dedicated Input Shift Register Chains	0	--	--

RV32IMP

Flow Summary	
Flow Status	Successful - Sun Jun 21 13:43:17 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	rv32imp_fpga
Top-level Entity Name	rv32imp_fpga
Family	Cyclone II
Device	EP2C35F672C6
Timing Models	Final
Total logic elements	6,391 / 33,216 (19 %)
Total combinational functions	5,838 / 33,216 (18 %)
Dedicated logic registers	1,544 / 33,216 (5 %)
Total registers	1544
Total pins	12 / 475 (3 %)
Total virtual pins	0
Total memory bits	131,136 / 483,840 (27 %)
Embedded Multiplier 9-bit elements	34 / 70 (49 %)
Total PLLs	0 / 4 (0 %)

rv32imp_fpga

Fitter DSP Block Usage Summary				
	Statistic	Number Used	Available per Block	Maximum Available
1	Simple Multipliers (9-bit)	4	2	70
2	Simple Multipliers (18-bit)	15	1	35
3	Embedded Multiplier Blocks	17	--	35
4	Embedded Multiplier 9-bit elements	34	2	70
5	Signed Embedded Multipliers	5	--	--
6	Unsigned Embedded Multipliers	12	--	--
7	Mixed Sign Embedded Multipliers	2	--	--
8	Variable Sign Embedded Multipliers	0	--	--
9	Dedicated Input Shift Register Chains	0	--	--

Công suất của nhân ma trận:

RV32IM

PowerPlay Power Analyzer Summary	
PowerPlay Power Analyzer Status	Successful - Sun Jun 21 14:10:25 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	rv32im_fpga
Top-level Entity Name	rv32im_fpga
Family	Cyclone II
Device	EP2C35F672C6
Power Models	Final
Total Thermal Power Dissipation	190.27 mW
Core Dynamic Thermal Power Dissipation	74.57 mW
Core Static Thermal Power Dissipation	80.19 mW
I/O Thermal Power Dissipation	35.51 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

RV32IMP

PowerPlay Power Analyzer Summary	
PowerPlay Power Analyzer Status	Successful - Sun Jun 21 14:17:22 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	rv32imp_fpga
Top-level Entity Name	rv32imp_fpga
Family	Cyclone II
Device	EP2C35F672C6
Power Models	Final
Total Thermal Power Dissipation	212.15 mW
Core Dynamic Thermal Power Dissipation	96.44 mW
Core Static Thermal Power Dissipation	80.26 mW
I/O Thermal Power Dissipation	35.45 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

Công suất của bộ lọc FIR:

RV32IM

PowerPlay Power Analyzer Summary	
PowerPlay Power Analyzer Status	Successful - Sun Jun 21 13:37:34 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	rv32im_fpga
Top-level Entity Name	rv32im_fpga
Family	Cyclone II
Device	EP2C35F672C6
Power Models	Final
Total Thermal Power Dissipation	188.16 mW
Core Dynamic Thermal Power Dissipation	72.46 mW
Core Static Thermal Power Dissipation	80.18 mW
I/O Thermal Power Dissipation	35.51 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

RV32IMP

PowerPlay Power Analyzer Summary	
PowerPlay Power Analyzer Status	Successful - Sun Jun 21 13:43:17 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	rv32imp_fpga
Top-level Entity Name	rv32imp_fpga
Family	Cyclone II
Device	EP2C35F672C6
Power Models	Final
Total Thermal Power Dissipation	210.24 mW
Core Dynamic Thermal Power Dissipation	94.48 mW
Core Static Thermal Power Dissipation	80.26 mW
I/O Thermal Power Dissipation	35.51 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

Công suất của thuật toán phát hiện biên Sobel:

RV32IM

PowerPlay Power Analyzer Summary	
PowerPlay Power Analyzer Status	Successful - Sun Jun 21 13:53:55 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	rv32im_fpga
Top-level Entity Name	rv32im_fpga
Family	Cyclone II
Device	EP2C35F672C6
Power Models	Final
Total Thermal Power Dissipation	189.67 mW
Core Dynamic Thermal Power Dissipation	74.03 mW
Core Static Thermal Power Dissipation	80.19 mW
I/O Thermal Power Dissipation	35.45 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

RV32IMP

PowerPlay Power Analyzer Summary	
PowerPlay Power Analyzer Status	Successful - Sun Jun 21 14:01:14 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	rv32imp_fpga
Top-level Entity Name	rv32imp_fpga
Family	Cyclone II
Device	EP2C35F672C6
Power Models	Final
Total Thermal Power Dissipation	211.48 mW
Core Dynamic Thermal Power Dissipation	95.71 mW
Core Static Thermal Power Dissipation	80.26 mW
I/O Thermal Power Dissipation	35.51 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data